面向系统与芯片架构工程师

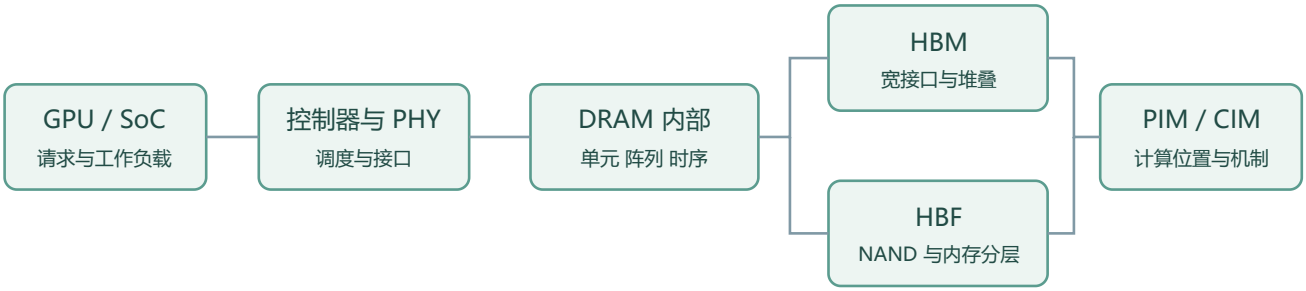
存储芯片架构学习手册

DRAM 内部结构 控制器 GPU 访存
HBM / HBF / PIM 与数字、模拟、混合存内计算

资料核对截至 2026 年 10 月 30 日 · 22 个学习模块 · 35 篇论文与技术案例 · 69 项一手资料

这份手册从存储单元和数据路径出发，建立容量、带宽、延迟、功耗、面积、可靠性与成本之间的关系。适合已有计算机体系结构基础、需要补齐存储芯片知识并准备技术面试的读者。

学习重点是能解释机制、估算数量级、比较设计方案，并识别论文结论的适用边界。每章包含通俗解释、专业细节、教学算例、面试问答和自测。所有教学数字与产品规格、论文测量和仿真结果分别说明。



图中连线表示学习关系，不表示统一的实际产品接线方案。

学习目录

1. DRAM 全景：从一个电容到 GPU 的内存通道
2. 1T1C 单元：电荷如何变成稳定的 0 和 1
3. 真实阵列：分块、译码、局部与全局 I/O
4. 命令与时序：把数字表变成资源状态机
5. 刷新与可靠性：弱单元决定了什么
6. 芯片架构取舍：性能、面积、能量、良率与可靠性
7. 论文变成面试能力：五个经典机制与反例
8. 控制器与 PHY：把请求变成合法、高效的 DRAM 命令
9. GPU 访存：从 warp 事务一直追到 DRAM 行
10. 存储产品地图：DDR、LPDDR、GDDR、HBM、SRAM 与 NAND
11. HBM：把DRAM阵列、通道与先进封装接起来
12. NAND基础：理解HBF前必须补齐的另一种阵列
13. HBF：高带宽Flash，到2026年究竟已知什么
14. GPU—HBM—HBF—SSD：权重、KV与MoE如何分层
15. PIM系统地图：计算放在哪里，究竟省了哪段搬运
16. 数字存算：从一位乘法到真正可用的整数 MAC
17. 模拟存算：电流真的会算乘法，但误差也会一起相加
18. 数模混合：把精度花在敏感位置，把并行留给合适阵列
19. 材料与落地瓶颈：不是所有存算路线都在等一种新材料
20. 论文训练营：从机制复述进阶到架构取舍
21. 架构案例：给 70B 推理设计一份能被评审的存储规格
22. 面试实验室：三个能诚实复现、能讲设计取舍的小项目
23. 论文案例比较与复述
24. 完整来源索引

如何安排学习

面试前一天 先完成 DRAM 层级、单元、时序、控制器、HBM、HBF 和 CIM 分类，挑两篇论文讲清机制与代价；以关键算例和问答为主。

三天补齐 第一天深入 DRAM 与控制器，第二天连接 GPU、HBM 和 HBF，第三天专攻 CIM、材料瓶颈与产品设计题。

完整研读 按目录读完，再挑一个可复现实验，记录版本、输入、假设、对照与失败案例。没有实施过的实验，应表述为阅读理解或拟议方案。

读论文时始终追问

作者改变了哪里？节省了哪条数据路径？增加了什么电路或软件开销？性能统计到哪一层？基线、精度、工作负载和制造条件是否可比？结果来自实测、仿真还是厂商目标？

模块 01 · 核心必会

DRAM 全景：从一个电容到 GPU 的内存通道

你已经熟悉 GPU/SoC 架构，这一章把 DRAM 从“一个固定延迟的黑盒”展开成共享资源层级。先回答三个问题：数据存在哪里、哪些工作能并行、哪些资源必须排队。随后读时序表和架构论文会自然得多。

学习目标

- 正确区分 Channel、Rank、Chip、Bank Group、Bank、Subarray 与 Mat
- 解释 row buffer 的物理含义及逻辑抽象限制
- 从容量、位宽和突发长度计算组织结构
- 把 GPU 请求拆成 DRAM 内部操作与资源争用

1. 把“存储器”拆成存储、搬运和控制

把 DRAM 想成一个有很多工作间的仓库，但不要让比喻替代电路。**存储单元负责保留电荷，感应放大器负责判定并恢复电荷，I/O 路径负责搬运，控制器负责安排时机。**这四种职责对应不同瓶颈：容量由单元数量决定，首次访问延迟受到电荷感测限制，连续传输速度受到接口限制，并发效率取决于地址分布和调度。

CPU/GPU 发出的是地址和读写请求；控制器要把物理地址映射成通道、Rank、Bank、行、列，再生成 ACT、RD/WR、PRE、REF 等命令。器件内部并不知道“这是一个矩阵的第几行”，只执行满足协议的命令。把软件张量行与 DRAM 物理行混为一谈，会使后续局部性分析全部错位。Ramulator 2.0 §II：层级与命令模型

面试时先画三条线：控制路径从调度器到译码器；数据路径从单元到 DQ；供电和刷新等维护路径横跨全芯片。然后说明优化落在哪条线。这个画法能避免把“加宽接口”和“缩短单元感测”混成一个问题。

2. 系统层级：Channel、Rank、Chip 的边界

层级	它组织什么	最容易误解的点
Channel 通道	一组可调度的命令/数据接口资源	加通道通常增加独立外部搬运资源，但还要看上游带宽和封装
Rank 秩	一组协同响应片选、拼成所需位宽的器件	同一通道的多个Rank通常共享数据总线，不等于多个通道
Chip / Die	物理芯片或硅片	一个封装未必只有一个Die；一个Rank也未必只有一颗芯片
Bank Group	一组具有部分共同访问约束的Bank	是时序和内部资源组织，不是软件缓存组

层级	它组织什么	最容易误解的点
Bank	对外可独立记录活动行状态的阵列集合	独立状态不代表独享DQ、命令总线和电源预算

经典非ECC DDR4 x64 Rank可由8颗x8器件拼成。控制器发一个Bank号/行号，8颗器件相应Bank共同动作，各自贡献8根DQ上的数据。因此8颗、每颗16个Bank并不意味着控制器能随意独立调度128个Bank；在这种锁步Rank组织下，逻辑上只有16组同步工作的Bank。此处仅为组织示例，不代表所有标准。SALP §2：Rank中的锁步关系

DDR5子通道、HBM Channel/Pseudo-channel要按各自协议重新确认位宽和共享范围。不能把“DDR4 Rank的x64”机械套到所有存储接口。Micron DDR5官方特性

3. Bank 内部：不是一个巨大的二维表格

教科书常把Bank画成一个整齐的大阵列，旁边放一条row buffer。这是**便于调度分析的逻辑视图**。真实电路必须切成较小的阵列块，否则字线、位线过长，电阻、电容和信号衰减都会恶化。小块中有局部字线、局部位线和局部感应放大器，再通过译码及全局数据路径连接起来。

Mat通常指较小的二维单元阵列与附近外围电路；Subarray常指一组共享局部访问结构的阵列区域。但**这两个名词的边界不是跨论文、跨厂商统一不变的**。SALP明确指出不同文献对Subarray的范围有不同用法。因此面试时不要硬背“一个Subarray必定有多少个Mat”，而要说明你采用的图中哪些局部感放和全局路径被共享。SALP §2.3及术语脚注

从版图看，距离本身就是性能变量：离字线驱动或感放较远的单元，信号传播条件不同。DIVA用真实器件表征说明结构位置能够带来系统性的时延差异；但内部地址扰乱和冗余修复意味着逻辑地址相邻不保证物理位置相邻。DIVA：分块组织与位置差异

4. Row buffer 实际是什么

Row buffer主要是正在保存整行判定结果的一组感应放大器，而不是另外放置的一块普通SRAM缓存。ACT把目标行连接到相应位线，感放判定并维持0/1；之后列命令只把其中选定的一小部分导向I/O。感放既是读取电路，也是保持已打开行的状态载体。

同一活动行的下一次访问可以省去重新激活，这叫row hit。Bank已预充电、没有活动行时，需要ACT再读；Bank打开了别的行时，通常需要先PRE再ACT。把“非hit”统称为miss有时会掩盖两者差别，因此汇报结果最好分别统计hit、closed-bank访问和row conflict。

物理上每个局部阵列区域有感放，并不意味着标准接口允许任意保留、任意选择多个活动行。共享行地址锁存器、全局I/O选择和协议状态可能限制这种自由度。SALP的价值就在于识别哪些限制可以用低代价改造解除。SALP：逻辑Bank与多个局部row buffer

对GPU而言，提高row hit率可能降低ACT/PRE次数，但如果为凑hit而延迟关键warp、阻塞其他Bank，最终吞吐和尾延迟仍可能变差。**hit率是解释变量，完成有用工作才是目标。**这条是资源调度推论，不是某篇论文的固定性能结论。

5. 容量与突发：把单位算清楚

教学假设：一个x8芯片有16个Bank，每Bank有65,536行，每个逻辑行1,024字节，暂不计冗余和ECC隐藏位。芯片可见容量为 `16 × 65,536 × 1,024 B = 1 GiB = 8 Gib`。8颗这样的芯片拼成x64 Rank，容量8 GiB。在这个假设下，一个Rank级逻辑行合计8 KiB；但单次RD不必把8 KiB全送出。

若突发长度BL=8，每根DQ传8个bit，x8芯片贡献 `8 × 8 / 8 = 8 B`，8颗共同贡献64 B。这样，一个64 B缓存行可以对应一次Rank级突发，而内部早已激活更宽的一整行。**容量单位Gb/GiB、接口位宽x8、突发长度8是三个独立概念。**

若接口每根DQ为3,200 MT/s，64位数据通道的理想带宽为 `3.2×10⁹ × 64 / 8 = 25.6 GB/s`。这是十进制GB/s，且只是外部连续传输上限。它没有减去刷新、换行、读写切换、总线空洞和无效字节。DDR4 8n组织及DDR双边沿传输的原始依据见Micron TN-40-40 第2页；本段所有组织组合和计算均为教学示例。

6. 一次 GPU miss 穿过了哪些资源

沿你熟悉的路径往下接：warp访存指令先合并为内存事务，经缓存和互连到内存控制器队列；控制器映射目标Bank并判断活动行；若需换行，就安排PRE/ACT；列命令选择数据，数据通过芯片I/O、PHY、控制器返回缓存，最后唤醒相关warp。整个过程只有一部分时间是单元感测时间。

教学拆账：假设一个请求总等待240 ns，其中上游与返回路径40 ns、控制器排队140 ns、器件服务60 ns。把器件服务缩短20%只节省12 ns，总体从240变228 ns，改善5%。若优化调度让排队降至80 ns，总体变180 ns，改善25%。实际系统中各部分可能重叠，这里是串行账本，用来训练先定位再优化的习惯。

因此看到“HBM带宽高”不能直接推出“随机访问首字延迟必定低”，看到“GPU有大量warp”也不能直接推出“内存延迟不重要”。只要并发请求不够、落点不均或依赖链太长，内部Bank限制仍然暴露。用可配置层级模型验证这些假设，比只记录总带宽更有解释力。Ramulator 2.0：控制器与器件分离建模

7. 面试白板的 90 秒组织顺序

1. 先标出外部接口：Channel，说明带宽预算和共享数据总线。
2. 再画Rank和芯片：说明通过多颗器件拼位宽，不把器件数量当并行Bank数量。
3. 进入芯片画Bank Group、Bank：说明独立活动行状态以及共享时序约束。
4. 进入Bank画局部阵列、感放、列选择和全局I/O：说明电路上的分块和协议上的约束不同。
5. 最后画一次ACT→RD→数据→PRE，分别标注首次感测、连续突发和换行成本。

不要一开始就罗列所有术语。面试官问“增加Bank是否能提高带宽”时，先回答它可能增加同时进行内部操作的机会，再追问数据口是否已经饱和、地址是否均匀、激活电流约束是否成为瓶颈。这样的回答同时连接器件和系统，也为后续SALP、HBM与PIM留下接口。

读任何存储论文，都可以在自己的图上圈出“改了哪里”：TL-DRAM圈位线，SALP圈局部/全局共享结构，RowClone圈感放到行的数据路径。图上圈不出修改位置，通常意味着尚未真正理解机制。

面试问答与追问

1 一个Bank到底有几个row buffer?

逻辑上，传统接口常把一个Bank描述为一个可见活动行；物理上，Bank分为多个局部阵列区域，各有感应放大器。不能把逻辑视图的一个row buffer误认为整Bank只有一组物理感放。可同时保留和访问几个活动行取决于器件结构、选择逻辑与协议。

继续追问

- 多个Subarray为何不能直接同时驱动全局I/O?
- SALP改的是哪种限制?

2 加Rank和加Channel区别在哪里?

加Rank通常增加容量和可交叠的内部工作，但同Channel的Rank仍共享外部总线，并有Rank切换约束。加Channel增加一组独立接口资源，通常能提高外部峰值带宽；收益仍取决于地址均衡、上游能力、引脚及功耗预算。

继续追问

- 为什么双Rank有时更快、有时更难跑高频?
- HBM的Channel该如何重新画?

3 x8、8n prefetch、8Gb分别表示什么?

x8是单颗芯片的外部数据位宽；8n prefetch是内部一次并行取得8倍外部位宽数据的组织方式；8Gb是芯片容量。三者不能互换。x8且BL8的一次完整突发贡献8字节，而不是8Gb或一个完整物理行。

继续追问

- 8颗x8组成x64时一次BL8传多少字节?
- 更长突发对小随机访问有什么代价?

4 row hit率越高，GPU性能一定越好吗?

不一定。hit可省激活/预充电，但为了hit而改变调度可能增加关键请求的等待、破坏公平或降低跨Bank并行。需要结合完成时间、队列时延、Bank/总线利用率以及GPU可运行warp数量分析。

继续追问

- 你会用什么微基准证明瓶颈?
- 吞吐和P99同时恶化时先看什么?

本章自测

1 8颗x8芯片锁步组成一个x64 Rank，每颗16个Bank，控制器通常可见多少组独立Bank?

A. 128组

B. 16组

C. 8组

D. 1组

参考答案 B 8颗器件对应Bank共同响应命令，贡献不同数据位片；不是128个可任意独立命令的Bank。

2 哪项最准确描述row buffer?

A. 额外放在DRAM里的普通SRAM L1

B. 保存被激活行判定结果的感放集合

C. 控制器请求队列

D. 每根DQ上的串行寄存器

参考答案 B row buffer与阵列感放直接关联；输出串行化缓冲是另一层资源。

3 一个x64数据接口速率为3200 MT/s，理论数据带宽为多少?

A. 3.2 GB/s

B. 12.8 GB/s

C. 25.6 GB/s

D. 51.2 GB/s

参考答案 C 3200×10^{6} transfer/s×64 bit/8=25.6×10^{9} B/s。MT/s已包含DDR双边沿，不能再乘2。

原始资料

1. Micron TN-40-40: DDR4 Point-to-Point Design Guide, Rev. H · 2020 · Micron Technical Note（授权分销商镜像）
2. A Case for Exploiting Subarray-Level Parallelism (SALP) in DRAM · 2012 · ISCA
3. Design-Induced Latency Variation in Modern DRAM Chips · 2017 · SIGMETRICS / POMACS
4. Ramulator 2.0: A Modern, Modular, and Extensible DRAM Simulator · 2024 · IEEE Computer Architecture Letters（2023预印本）

5. Micron DDR5 SDRAM Product and Feature Overview · 2026 · Micron官方产品页（访问于2026-10-30）

模块 02 · 核心必会

1T1C 单元：电荷如何变成稳定的 0 和 1

这是整个学习路线最关键的一章。把电荷分享、感应放大、恢复和预充电想明白，tRCD/tRAS/tRP、位线分段、刷新与存算融合都会变成同一套电路逻辑的不同应用。

学习目标

- 画出1T1C单元和差分感应放大器的作用
- 推导电荷分享后的位线电压
- 区分破坏性读取、恢复、刷新与预充电
- 解释为何数字接口时序由模拟电路约束决定

1. 一个开关、一个电容，但不是理想电路

典型DRAM单元用一个访问晶体管加一个存储电容，简称1T1C。晶体管的栅接字线WL，字线决定是否把电容连接到位线BL。电容节点的电荷状态表示信息，位线负责把微弱信息交给感放，也负责把写入电平送回单元。**电容不是持续主动输出0/1的逻辑门。**它只能贡献有限电荷，且会随时间泄漏。

示意图常用“高电压表示1、低电压表示0”。真实阵列还存在true/anti cell、内部数据扰乱等实现差异，因此外部逻辑1不必对应每个物理单元都充高电平。面试原理推导可以采用理想正逻辑，但需说明这是简化模型。

芯片设计困难在于：单元要小，能存的电荷有限；位线却连接许多单元，存在较大寄生电容。小电容向大电容分享电荷时，电压变化很小。感放必须在器件偏差、耦合噪声、电源波动和温度变化下可靠辨认这点差异。位线长度与感放开销的权衡见TL-DRAM §2–3。

2. 用电荷守恒推导：为什么信号这么小

在理想模型中，访问前位线预充到VDD/2，单元电容为Cc，位线等效电容为Cb。字线拉高、二者相连后，先忽略泄漏、晶体管压降与动态时序。由电荷守恒：

```
Vshare = (Cc × Vcell + Cb × VDD/2) / (Cc + Cb)
ΔV = Vshare − VDD/2
   = [Cc / (Cc + Cb)] × (Vcell − VDD/2)
```

教学假设：VDD=1.0 V、Cc=20 fF、Cb=200 fF。若Vcell=1.0 V，ΔV≈+45.5 mV；若Vcell=0，ΔV≈−45.5 mV。原本相差1 V的两种单元状态，落到位线后只表现为约±45.5 mV的扰动。若存1的单元因泄漏只剩0.7 V，ΔV降为约+18.2 mV。

这说明三个可推导结论：增加位线电容会压小初始信号；更长时间未恢复的单元通常具有更小判读余量；单纯提高外部DQ速率不能让这点模拟差异自动更快稳定。公式是本教程的理想电路推导，数值不是任何量产器件参数。原始电荷过程示意见AL-DRAM 图2。

3. 感放：把微小差异拉成全摆幅

差分感应放大器可以理解为一个具有正反馈的判决器。它比较一对位线上的微小差异，把稍高的一侧推向高电平、稍低的一侧推向低电平。这个过程是模拟再生，最终结果才表现为数字0/1。交叉耦合结构提供“越高越往高、越低越往低”的正反馈，但初始差异过小或偏置过大时也可能判错。

所谓“感放失调”是没有有效输入差异时，器件不匹配让电路偏向某一侧。可靠读出需要有效信号足以压过失调、噪声和耦合影响，并留出稳定时间。存储架构面试未必要求画完整晶体管级电路，但应解释清楚：**tRCD并不是任意插入的等待拍，而是内部信号到达可安全列访问状态的时间约束。**

真实阵列可能采用不同位线布局、参考方式和局部/全局感放层级。这里只讲共同功能，不把一张示意图当成所有DRAM的版图。关于真实内部位置和延迟之间的关系，可接着阅读DIVA：设计引起的时延差异。你可以把“距离、RC、初始信号、感放余量”作为一条因果链，而不是背四个孤立名词。

4. 破坏性读取到底破坏了什么

打开字线后，单元与预充到中间电平的位线分享电荷，原本完整的单元状态被扰动，因此这种感测具有破坏性。**不是说每执行一次RD列命令，数据就被清空一次。**整行的感测发生在ACT所启动的过程中；同一行已经打开时，后续RD从稳定的感放状态选择数据，不需要每次重新对单元做初始电荷分享。

感放判定后继续驱动位线，同时字线还开着，于是把单元重新拉到可靠状态，这叫restore恢复。恢复与感测是同一激活过程中的相连阶段；数据已经可以送往列路径时，单元可能仍需要继续恢复。为什么tRAS往往大于tRCD，正是这个问题的核心。

PRE不是“把row buffer写回单元”的主要阶段。安全发PRE之前应已满足所需恢复时间；PRE把字线关掉，并让位线回到适合下一次感测的均衡状态。标准还通过写恢复等约束防止刚写完就过早关闭。AL-DRAM §2.2：感测与恢复

刷新则是为了抵消长期泄漏而主动安排的维护工作。它会使用类似激活、恢复、关闭的底层机制，但动机不同：restore解决刚刚感测造成的电荷扰动，refresh保证长时间无人访问的数据也不丢。两者不能只翻译成同一个“回写”来含糊处理。

5. 从预充电到一次完整读写

1. **空闲预充电：**字线低，单元隔离；相应位线对处于均衡电平，感放未维持旧行数据。
2. **ACT：**行地址译码，选中字线升高，单元电荷影响位线。
3. **感测：**启用感放，微小差异再生成明确电平。
4. **RD：**满足tRCD后，列选择网络取出目标列组，通过内部I/O到输出路径；其余已激活数据留在感放。

5. **WR：**输入数据沿相反方向进入，被选中的感放/位线状态由写驱动更新，字线仍开着，因此新状态进入单元；未选部分按设计保持。
6. **安全关闭：**满足行激活最短时间以及读到预充电、写恢复等约束，才可PRE；均衡完成后才可激活下一行。

读路径与写路径共享很多结构，但控制条件并不完全对称。写入需要确保新的单元状态充分建立，因而“RD结束了可以PRE”不能直接推广到“WR发出去就能PRE”。RowClone正是利用感放已经持有稳定数据时能够驱动另一个目标行的观察，提出内部复制机制；这属于需要专门支持的扩展。RowClone §2–3：内部数据方向

6. 为什么预充到中间电平，为什么不能省

直觉上，如果旧位线停在高电平，新单元也为高，似乎不用预充。但下一次数据未知：若旧位线高而新单元低，小单元很难把大位线直接拉低；即使只检测电压差，也会留下强烈依赖上次数据的初始条件。回到中间参考使高、低两种状态都能朝相反方向产生可辨认的小扰动。

教学思想实验：仍取Cc=20 fF、Cb=200 fF。若位线保留1 V而新单元为0，共享后约0.909 V，很容易仍被当成高；先预充到0.5 V，则共享后0.455 V，相对于中点清楚落在低的一侧。这不是完整感放仿真，却足以说明为什么“旧行读完直接接新行”一般不成立。

再注意，PRE必须先使单元与位线隔离，否则把位线拉回中间电平时，也会把刚恢复好的单元再次拖向中间。这个先后关系把电路过程与命令约束直接连起来。预充电的原始电路解释见TL-DRAM：Precharging阶段。

因此“关闭row”不仅是清一个逻辑valid位；它是有能量和时间成本的物理动作。Open-page/close-page策略其实是在决定什么时候支付这笔成本，以及赌后续访问是否还会用到当前行。

7. 如何把模拟直觉变成架构判断

有GPU基础的人容易把DRAM理解成多Bank SRAM再加一个慢接口。区别是，DRAM核心延迟与电荷余量、泄漏、位线电容、感放偏差强相关。因此很多架构方案不是增加一级流水就能提高频率，而是改变物理共享程度、数据放置或安全裕量。

针对“能不能把时序都调小”这类问题，分三层回答：第一，原理上典型器件可能具有保守余量；第二，余量取决于工艺、温度、单元位置和数据状态；第三，产品需要覆盖受支持条件和故障率目标，有限测试不等于长期保证。AL-DRAM展示可利用典型条件余量的研究方向；SoftMC又提醒我们，某些由模型推测的可观察收益，在实物上未必能够直接测出。SoftMC：模型假设与真实器件实验

面试最后可以这样收束：“我会从Cc/Cb和初始电荷解释感测余量，再把它映射到tRCD、tRAS和tRP；如果提出降延迟方案，会明确它是在改电路、改控制器，还是只改变测试条件。”这比背出一个DDR速率更能证明你已经跨过芯片内部结构这一关。

面试问答与追问

1 DRAM为什么需要sense amplifier，而不能直接读电容电压?

单元可提供的电荷很少，连接较大位线电容后只造成微小电压扰动；感放利用差分与正反馈把扰动再生为可靠电平，同时驱动单元恢复。还要覆盖失调、噪声、温度及工艺差异。

继续追问

- 如果位线电容翻倍，初始信号会怎样?
- 为什么感放尺寸不能无限缩小?

2 DRAM读取是破坏性的，为什么row hit还能很快?

破坏性指ACT时单元与位线分享电荷；感放随后判定并恢复。行打开后数据已由感放保持，row hit的RD只做列选择和输出，不是每次重新破坏并重读单元。

继续追问

- tRCD与tRAS分别保护哪一步?
- 关闭行时发生了什么?

3 restore、refresh、precharge分别是什么?

restore是在激活感测后把单元重新充到可靠状态；refresh是周期性安排恢复以对抗闲置泄漏；precharge是在安全关闭单元后把位线均衡，为下一行建立初始条件。恢复主要在激活阶段完成，不能把PRE简单说成回写。

继续追问

- 为什么没RD也可能需要刷新?
- 持续访问一个行是否能免除其他行刷新?

4 用一个公式解释长位线的代价。

理想电荷分享$\Delta V = Cc/(Cc+Cb)\times(Vcell - VDD/2)$。长位线通常使Cb更大，初始扰动变小，也增加驱动/均衡负担；但它能让更多单元分摊感放面积。因此这是延迟、能量与bit成本的权衡。

继续追问

- 短位线为什么贵?
- TL-DRAM如何折中?

本章自测

1 破坏性读取主要发生在哪一步?

A. 每次CPU读取寄存器

B. ACT引发单元与位线电荷分享时

C. 数据离开DQ时

D. 控制器做地址映射时

参考答案 B ACT打开字线后，电荷分享扰动单元原有电荷，感放随后恢复。

2 Cc=20 fF，Cb=180 fF，VDD=1 V，Vcell=1 V，理想分享后位线电压为?

A. 1.0 V

B. 0.9 V

C. 0.55 V

D. 0.05 V

参考答案 C (20×1+180×0.5)/(20+180)=0.55 V。扰动是0.05 V，位线总电压是0.55 V。

3 哪句话正确?

A. PRE的主要作用是把未恢复数据首次写回单元

B. tRCD满足后就一定可立即PRE

C. 恢复通常在ACT引发的感测过程中进行，PRE负责关闭及均衡

D. 刷新只针对刚被读取过的行

参考答案 C 列访问就绪和单元充分恢复是不同条件；PRE还需满足tRAS及读写相关约束。

原始资料

1. Tiered-Latency DRAM: A Low Latency and Low Cost DRAM Architecture · 2013 · HPCA
2. Adaptive-Latency DRAM: Optimizing DRAM Timing for the Common-Case · 2015 · HPCA
3. RowClone: Fast and Energy-Efficient In-DRAM Bulk Data Copy and Initialization · 2013 · MICRO
4. Design-Induced Latency Variation in Modern DRAM Chips · 2017 · SIGMETRICS / POMACS
5. SoftMC: A Flexible and Practical Open-Source Infrastructure for Enabling Experimental DRAM Studies · 2017 · HPCA

真实阵列：分块、译码、局部与全局 I/O

已经理解单个单元后，要解决一个工程问题：如何把数十亿个单元组织成能制造、能寻址、能修复、还能快速搬运的芯片？这一章把简单Bank方框展开成真正的层级数据路径。

学习目标

- 解释Mat/Subarray分块为何同时影响延迟和面积
- 区分局部位线、全局数据线和外部DQ
- 理解行译码、列译码与prefetch的不同任务
- 从内部资源共享推导Bank Group和带宽限制

1. 阵列为什么必须分块

假设把所有单元排成一个超大的二维矩阵：每条字线要驱动大量晶体管栅，每条位线要连接大量单元。阵列越大，导线越长，RC和负载越大，远端延迟和近端延迟也越不一致。切成Mat或局部阵列后，线变短、信号更容易建立，但需要更多局部驱动器、感放、译码和布线空隙。

教学面积模型：每根位线连接N个单元，单元面积为Ac，感放及相关外围摊销为As，则单bit局部面积可近似写成`Ac + As/N`。若Ac=1单位、As=100单位，N=512时为1.195，N=256时为1.391，即仅这一简化项约增16.4%。缩短位线并不是免费提速。该模型忽略布线、译码、边缘和工艺规则，只用于说明摊销方向。

TL-DRAM讨论的核心正是感放摊销与位线负载之间的矛盾。真实芯片必须同时优化良率、密度和时序，不能只看一条曲线最低点。TL-DRAM §3：cells-per-bitline权衡

2. 行译码不是一个巨大的单级译码器

逻辑上，行地址选中一行；电路上常经过多级预译码、区域选择和局部字线驱动，避免让一个巨大扇出节点直接驱动全芯片。你可以把它理解成“先到楼栋，再到房间”，但关键电气原因是负载、距离和布线。局部字线真正控制访问晶体管，主字线/全局控制信号负责选择区域；具体层级依器件而异。

列译码处理的是另一件事：行已经由感放持有大量数据，列地址选择其中一组进入内部数据通路。因而一个列命令传出的bit数远小于激活涉及的bit数。读少量数据却要激活较大区域，是DRAM能耗分析里经常被忽略的放大效应。

教学假设：一颗芯片激活1 KiB逻辑行，而一次完整列突发只贡献8 B，那么内部感测数据量与这次外部贡献之比为128。它不是“能耗必定放大128倍”，因为感测、线驱动和I/O每bit成本不同，但说明随机小访问的利用率问题不能仅靠统计DQ字节数理解。内部层级可对照DIVA：Mat、驱动器与位置关系。

物理行与逻辑行也可能因冗余替换和内部重映射而不一致。所以分析器件的邻近效应时，物理映射是需要验证的假设，不能直接拿连续物理地址当版图坐标。

3. 局部位线、全局数据线、DQ分别负责什么

数据路径可以按如下顺序画：

```
单元电容 → 局部位线 → 局部感放 / row buffer
         → 列选择 / 局部I/O → 全局I/O与主感放
         → prefetch/串并转换 → DQ引脚 → PHY
```

最靠近单元的局部位线承担微弱电荷感测。经局部感放放大后，选中的数据再进入较长、较少的全局线路；全局级可能再有主感放或I/O放大器。文献有时把某段全局线路称为global bitline，有时强调global I/O，阅读时要沿着图检查它连接的对象，而不是只比较名字。

这里存在**宽到窄的层级复用**：一整行有成千上万bit，外部可能只有x8或x16 DQ。只有把少数选定数据依次送出，才能让存储密度和封装引脚预算兼容。因此Bank内部有很多感放，不代表所有感放可以同时驱动外部口。RowClone的FPM与PSM分别利用局部感放路径和内部共享总线，正好揭示这两种不同搬运粒度。RowClone §2–3

架构分析时要逐层问：哪个节点允许多个源同时保留状态？哪个节点一次只能选一个源？哪个节点有流水但不能增加总吞吐？这比笼统地说“存在带宽墙”更接近可执行设计。

4. 8n/16n prefetch不是缓存预取器

DRAM中的prefetch主要是**内部宽数据访问与外部高速窄接口之间的比例组织**。它不预测程序未来会访问哪个地址。CPU预取器根据访问历史提前请求数据，DRAM prefetch则在一个确定的列访问中并行取得多份外部数据字，然后以突发方式送出。

以DDR4 8n、x8为例：内部按64 bit组织一次列数据，外部每个传输拍送8 bit，共8拍，双边沿下占4个CK周期。8n中的n代表外部位宽，不是“预取8个缓存行”。DDR4维持8n并引入Bank Group组织的相关讨论见Micron TN-40-40：Prefetch与Bank Group。

教学推导：外部3.2 GT/s、x8，理想数据吞吐3.2 GB/s。若每次内部并行提供64 bit，等效需要每秒4亿次这样的64 bit数据供应。这里的“等效供应速率”不能简单宣称为所有内部电路统一运行在400 MHz：行激活、列流水、DLL和I/O使用不同电路与时间尺度。

更大的prefetch可以缓和阵列/外围相对外部接口的速率差，但也会改变最小传输粒度、内部布线和小访问效率。是否值得增加，要看真实事务大小和可用局部性，而不是因为数字更大就一定更先进。

5. Bank Group：共享资源在时序表里的投影

如果每个Bank都复制所有高带宽外围，面积和功耗会很高；如果共享太多外围，连续列命令就容易被内部通路节拍限制。Bank Group可以看作这种分组与共享关系在外部协议上的一种体现。对DDR4，同组与跨组的某些命令间隔不同，常用L与S后缀表示长、短约束。Micron TN-40-40 第6–8页

教学假设：BL8的数据占4 CK，而同组列命令间隔要求6 CK、跨组允许4 CK。如果连续读都在同组，即使行全命中，也可能每4 CK数据后空2 CK，理想化数据占用率只有4/6≈66.7%；交错到不同组可能填满这些空隙。这一算式忽略其他限制，目的是说明“row hit但带宽不满”完全合理。

还要注意，降低列命令间隔不能消除tRRD/tFAW等激活限制；不同Bank可以交叠内部感测，也不意味着DQ上能同时出现两份数据。把Bank Group、Bank并行和外部总线并行分开，才不会把纸面Bank数量误当可持续吞吐。

在GPU中，这个问题会转化为地址映射与请求合并：低位如何分配给列、Bank和Group，既决定连续事务是否共享活动行，也决定并发事务是否碰上同一内部节拍。

6. 映射没有唯一正确答案：做一组小例子

教学假设：有4个Bank，每个逻辑行容纳4个64 B事务。策略A让同一256 B范围内的4个事务都落同Bank同一行；策略B把连续64 B事务轮转到4个Bank。对于单线程顺序扫描，A容易摊薄ACT成本；对于并发请求很多而首次访问很多的场景，B可能更容易交叠内部延迟。

但不要马上给出“A适合顺序，B适合随机”的口号。顺序流若连续覆盖许多行，A也能跨行轮转Bank；B若低位交织设计得当，也能在多个Bank各自积累行命中。真正需要比较的是**在当前队列窗口中，有多少请求可同时成为合法命令**，以及命令间隔能否被其他工作填上。

建议自己手画8个请求：写出arrival time、Bank、row和读写类型。用两种映射分别圈出hit、conflict、可并行ACT以及DQ占用。这个小练习通常比只背“bank interleaving”有效。之后再用Ramulator将请求映射、调度与器件模型拆开做控制变量实验。Ramulator 2.0：独立地址映射和控制器组件

论文阅读中尤其要核对地址映射是否暴露Subarray号。一个需要知道内部Subarray位置的方案，如果没有获得映射信息的机制，就还缺产品落地的一环。

7. 连接 HBM 和存算：什么变了，什么仍需重新核对

学完内部路径，你可以把HBM视为在封装与接口组织上显著增加并行数据通路的DRAM系统，再继续检查其通道、伪通道、Bank和命令资源。不要因为它采用堆叠就把单元、位线、感放和刷新问题删掉；也不要把DDR4的所有命令约束逐字套过去。现代HBM还存在特定的行/列命令发射组织，必须按代际协议建模。Ramulator 2.1 §2：现代协议差异

存算则要问计算单元放在哪里。若操作直接利用感放和电荷分享，它依赖局部阵列内的数据布局；若在Bank附近增加算术逻辑，瓶颈可能变成局部到逻辑的数据口；若在堆叠底部逻辑Die计算，又要看垂直互连和跨Bank搬运。三者都叫“靠近内存”，数据路径却完全不同。

一个专业的结论应指出收益来自省掉哪段传输：省CPU↔DRAM外部通道，还是省Bank↔全局I/O，或只是提高并行度。能量和延迟要在相同工作量下统计整个路径；只展示阵列端算子峰值，无法回答端到端系统是否更快。

面试问答与追问

1 既然短位线更快，为什么不全部用最短位线?

更短位线减少单元数和寄生负载，却让更多感放、局部译码和布线开销分摊给更少bit，降低面积效率并增加外围复杂度。要比较可用容量、芯片面积、能量、良率与时延；TL-DRAM用近远段尝试折中。

继续追问

- 如何写一个最简单的面积模型?
- 为什么远段访问可能受隔离器件影响?

2 DRAM prefetch与CPU硬件预取的区别?

前者在已知列地址下，采用内部宽并行取数并通过外部窄口突发传输；后者预测未来程序地址并提前发请求。8n不是预测8个缓存行，也不保证应用会使用这些数据。

继续追问

- x16、BL8的一次突发贡献多少字节?
- prefetch增大有什么浪费?

3 为什么全是row hit却仍跑不满总线?

列命令间隔、同Bank Group约束、读写方向切换、命令总线、Rank切换和上游请求不足都可能制造空洞。应先检查命令轨迹及数据占用，再判断是tCCD还是其他资源限制。

继续追问

- 如何设计跨Bank Group对照实验?
- 为什么仅看hit率不够?

4 给存算架构选择一个计算位置，你先问什么?

先明确运算和数据布局，逐段画出输入、计算和结果搬运路径。阵列内、Bank附近、逻辑Die的并行粒度和共享口不同；还需考虑一致性、同步、可编程性及工艺限制。

继续追问

- 两个操作数在不同Subarray怎么办?
- 结果进入GPU缓存时需要什么协调?

本章自测

1 DDR4 x16且BL8的一次完整突发，单颗器件贡献多少数据？

A. 8 B

B. 16 B

C. 64 B

D. 128 B

参考答案 B 16 bit/拍×8拍=128 bit=16 B，不能把单器件和Rank混算。

2 哪项最能解释阵列分块的动机？

A. 让所有Bank共享同一根字线

B. 减轻长导线RC和负载，但付出外围摊销开销

C. 取消感放

D. 让程序地址等于版图位置

参考答案 B 分块的基本权衡是局部线更短与外围重复增加。

3 BL8占4 CK，同组tCCD=6 CK，忽略其他开销，连续同组读的理想数据占用上限约为？

A. 33.3%

B. 50%

C. 66.7%

D. 100%

参考答案 C 每6 CK能启动一次4 CK突发，故4/6。实际数值必须使用目标器件和模式。

原始资料

1. Micron TN-40-40: DDR4 Point-to-Point Design Guide, Rev. H · 2020 · Micron Technical Note（授权分销商镜像）
2. Tiered-Latency DRAM: A Low Latency and Low Cost DRAM Architecture · 2013 · HPCA
3. RowClone: Fast and Energy-Efficient In-DRAM Bulk Data Copy and Initialization · 2013 · MICRO
4. Design-Induced Latency Variation in Modern DRAM Chips · 2017 · SIGMETRICS / POMACS
5. Ramulator 2.0: A Modern, Modular, and Extensible DRAM Simulator · 2024 · IEEE Computer Architecture Letters（2023预印本）
6. Ramulator 2.1: A Composable Memory System Simulator for Modern DRAM Systems · 2026 · arXiv预印本

命令与时序：把数字表变成资源状态机

时序不是一串需要死记的缩写，而是“哪个资源刚做过什么、下一步至少要等多久”的集合。本章用状态机、时间线和三个计算例子训练你现场分析DRAM请求的能力。

学习目标

- 解释ACT/RD/WR/PRE/REF及其合法状态
- 区分Bank局部、跨Bank和数据总线约束
- 计算row hit、closed-bank、row conflict的简化延迟
- 用命令时间线识别tRAS、tFAW、写恢复等遗漏

1. 先看状态合法，再看计时是否满足

最小Bank模型有预充电就绪、激活中/活动、预充电中、刷新中几类状态。预充电就绪才能ACT；活动行满足时序后才能RD/WR；要切换到另一行，需先安全PRE，再等均衡结束。REF的要求取决于标准和刷新范围，不能把“任何时候都能刷一下”写进调度器。

控制器每拍通常先确定候选请求需要哪个前置命令，再检查命令在目标状态是否允许，接着检查所有相关时间约束和资源占用，最后做优先级选择。**Ready不只是“队列里有请求”，还包括状态、时间和共享资源同时允许。**命令改变状态，也会更新多个层级的下一次可发射时间。

Ramulator 2.0把组织层级、命令、前置条件和时序依赖明确建模，是读代码理解协议的好入口。它的DDR4命令轨迹验证使用厂商Verilog模型检查时序和状态；这证明的是协议层约束检查，不能等同于验证某个自定义电路在硅上可运行。Ramulator 2.0 §II–III

一个实用检查方法是给每条约束标作用域：同一Bank、同一Bank Group、同一Rank、同一Channel或某条数据路径。若你只给每个Bank记一个“busy直到某时刻”，会把能够交叠的阶段过度串行化；若只记录活动行号、不记录跨Bank计时，又可能产生非法激活密度。状态和计时需要分层保存。

2. Bank 局部约束：看懂 tRCD、tRAS、tRP、tRC

参数	连接的事件	要保护什么
tRCD	ACT→可发列读/写	感测达到可用状态；部分标准读写有分项
tRAS(min)	ACT→允许PRE	保持活动足够久，确保单元恢复等要求
tRP	PRE→下一次ACT	关闭、位线均衡完成

参数	连接的事件	要保护什么
tRC	同Bank ACT→下一次ACT	完整行周期；常与tRAS+tRP对应
tRTP	RD→PRE	读操作内部过程不被提前关闭破坏
tWR	写数据结束→PRE的恢复要求	新写单元充分建立，起算点不能误用WR命令

CL/RL描述列读命令到数据输出的延迟，CWL/WL描述写命令到写数据的关系；实际可能受模式和附加延迟影响。不要用一个“CAS延迟”代替端到端内存延迟。激活中感测和恢复的阶段区别可对照AL-DRAM 图2。

特别避免说“tRAS=tRCD+CL是物理定律”。tRAS是独立的最短活动约束，列命令可以在行打开期间不同时间发生。安全PRE取决于多条约束的最大值，而不是随意把几个缩写相加。还需检查器件规定的最长活动时间或其他维护要求，不能无限保持一行打开。

3. 跨 Bank 限制：为什么有空闲Bank也不能立刻ACT

tRRD限制相关范围内不同Bank的ACT间隔，tFAW限制滚动窗口内的激活次数，用来约束激活相关电流和资源压力。tCCD限制连续列命令的间隔；DDR4的同Bank Group和跨Bank Group存在L/S差异。它们不是同一种等待，改变地址分布可能缓解一项，却仍受另一项限制。Micron TN-40-40：时序约束与分组

教学假设：tRRD=4 ns、tFAW=32 ns，忽略其他限制。前四个ACT可发在0、4、8、12 ns；第五个不能发在16 ns，因为任意32 ns滚动窗口最多允许四次，最早要等到32 ns。后面再按同样规则滑动。此时“还有很多Bank空闲”也没有用。

读写切换还涉及共享DQ方向反转，控制器常把写请求成批排出以摊薄切换成本，但太长的写批次会推高等待读的尾延迟。Rank切换、命令总线占用、不同通道组织也可能加入约束。因此“多个Bank并行”更准确的意思是内部阶段可交叠，而不是所有命令和数据都能同拍无限并行。

tRFC表示一次相应刷新操作的恢复/忙碌时间；tREFI描述刷新命令安排的平均间隔要求。刷新范围不同，不能把tRFC一律当“全系统都停同样久”。

4. 一条严格说明假设的时间线

教学参数，非器件规格：tRCD=14 ns、CL=14 ns、tRAS=32 ns、tRP=14 ns、tRC=46 ns、tRTP=8 ns、数据突发持续4 ns。请求到达时Bank已预充电，且没有排队、总线冲突或刷新。

```
0 ns      ACT row A
14 ns     RD A.column0
28–32 ns  数据突发
32 ns     最早PRE：max(ACT+32，RD+8)=32
46 ns     最早ACT row B：PRE+14，同时满足tRC
60 ns     RD B.column0
74–78 ns  B的数据突发
```

首请求首数据延迟28 ns。第二行如果在32 ns就绪，最早首数据在74 ns，比当时起算42 ns；如果请求在0 ns就排队，总等待就是74 ns。**必须说明延迟从哪个事件开始计时。**论文中有请求响应时间、器件服务时间、命令间隔等不同指标，它们不能直接比较。

同一活动行且RD可立即发时，首数据简化为CL=14 ns；Bank已空闲时为tRCD+CL=28 ns；冲突且旧行可以立即PRE时为tRP+tRCD+CL=42 ns。如果旧行还没满足tRAS或刚写完，冲突延迟还要增加等待。基础命令过程参见SALP §2的命令/状态说明；上述完整时间线为原创教学推导。

5. 周期和纳秒：两条限制都要满足

Datasheet常同时用最少周期数和最少绝对时间表示约束，实际控制器应满足二者较严格的一项。**教学例：**某约束写为至少4 CK且至少5 ns，tCK=0.75 ns时，需要 `max(4, ceil(5/0.75))=7 CK`，对应5.25 ns；不能只配置4 CK。实际数据表的取整和允许组合仍按标准/器件定义处理。

DDR速率的MT/s已经计入每周期两次数据传输。若速率为3,200 MT/s，CK频率通常是1,600 MHz，对应0.625 ns；不能把3,200 MHz直接当CK频率，也不能在带宽计算里再多乘一次2。现代LPDDR、GDDR可能分离命令与数据时钟，必须明确参数属于哪种时钟域。

另一个常见误区是“时序周期数增加，所以延迟一定变大”。例如教学比较：18 CK×0.833 ns≈15 ns，24 CK×0.625 ns=15 ns；周期数增加但绝对延迟近似不变。反过来，外部频率翻倍时，若阵列绝对时间没减，内部行周期不会自动翻倍变快。

这也是为什么DRAM的带宽扩展与首次访问延迟改善是两条不同路线。现代协议的时钟与命令差异可参考Ramulator 2.1 §2，面试以具体器件表为最终依据。

6. 从时序推导吞吐上界，别只背峰值带宽

教学假设：4个可独立轮转的Bank，全部为不同活动行的小访问，每次换行服务一个64 B事务，tRC=50 ns。忽略所有其他限制，单Bank受行周期限制的吞吐至多 `64 B/50 ns=1.28 GB/s`，4Bank合计至多5.12 GB/s。即使外部口标称25.6 GB/s，内部供给也可能明显不足。

若每次ACT后能连续读8个有用64 B事务，同样的激活成本可分摊到512 B；但不能简单宣称性能翻8倍，因为列间隔和外部口会先成为限制。写成一组边界更稳妥：

```
可持续吞吐 ≤ min(
  外部DQ带宽,
  可供给的列操作速率 × 每次有效字节,
  Bank行周期可供给速率,
  tRRD/tFAW允许的激活速率 × 每次激活有效字节,
  上游请求生成与返回路径能力
)
```

这个min模型是本教程的分析框架；实际开销会交叠，不能把所有损失率随意相加。面试时先找到最小上界，再讨论如何用测量确认它：命令计数、活动行持续时间、DQ空洞、每次ACT传输字节数，比单独看带宽更有诊断力。

7. 用微基准把缩写变成证据

设计四组请求：同Bank同Row多列、同Bank不同Row轮转、不同Bank同Group、不同Group轮转。保持事务大小、读写比例和请求强度相同，依次观察列路径、行周期和分组约束。然后再加读写混合、刷新和不均匀热点，观察性能拐点。

固定请求速率扫负载，记录平均与P95/P99延迟、完成吞吐、row hit比例、ACT/PRE数、每Bank占用和DQ空洞。低负载测到的主要是服务时间；高负载测到的是服务能力加队列效应。不能把“满载延迟升高”直接解释成器件tRCD变大。

如果你真的做过这样的实验，面试可以说“我做了地址映射与命令约束的仿真比较”；在本教程阶段则应说“我会这样验证”，不要把学习文档包装成已完成项目。本网站的交互演示是教学模型，完整协议研究应固定模拟器版本、配置、trace和统计定义。2026年的Ramulator 2.1增加了命令轨迹可视化与更完整验证设施，可作为后续实践入口。Ramulator 2.1 §4

面试问答与追问

1 为什么tRAS比tRCD长?

tRCD对应达到可安全列访问的状态；tRAS限制行至少保持活动多久，涵盖单元恢复等要求。数据可以开始通过列路径输出时，单元未必已经充分恢复，因此不能立即PRE。

继续追问

- 如何计算最早PRE时刻?
- 写入会额外引入什么条件?

2 已经有tRRD，为什么还需要tFAW?

tRRD限制相邻激活的最小间隔，tFAW进一步约束较长滚动窗口内的激活密度。只满足两两间隔仍可能使连续多次ACT形成过高电流/资源压力；两者必须同时检查。

继续追问

- 给出0、4、8、12 ns后第五次ACT的例子。
- 更多Bank为什么不能突破该上界?

3 row conflict一定等于tRP+tRCD+CL吗?

仅在旧行已能安全PRE、无排队、无总线或其他约束时，这是到首数据的简化服务延迟。若旧行还受tRAS、tRTP、tWR限制，或REF/读写切换介入，还要增加对应等待。

继续追问

- CL是命令到首数据还是到完整缓存行?
- 请求端到端时延还包含哪些路径?

4 如何证明GPU带宽不满是DRAM行周期限制?

控制变量改变地址映射和每次ACT有效字节数，统计ACT/PRE、Bank占用和DQ空洞；用Bank数×每行有用字节/tRC给出上界，再检查tFAW、列间隔和上游请求数是否更紧。需要命令轨迹证据，不能只从总带宽猜。

继续追问

- 随机请求更多会不会只增加排队?
- 如何区分同组tCCD与tRC限制?

本章自测

1 某时序要求至少4 CK且至少5 ns，tCK=0.75 ns，最少需要多少CK?

A. 4

B. 5

C. 6

D. 7

参考答案 D ceil(5/0.75)=7，且大于4，所以至少7 CK。

2 教学假设tRRD=4 ns，tFAW=32 ns，ACT在0、4、8、12 ns，下一次最早为?

A. 16 ns

B. 20 ns

C. 28 ns

D. 32 ns

参考答案 D 第五次必须等最早一次离开四激活滚动窗口，同时满足tRRD。

3 Bank冲突请求的42 ns简化延迟最可能遗漏什么?

A. 旧行尚未满足tRAS或写恢复的额外等待

B. 容量单位换算

C. 电容不需要恢复

D. 缓存行永远只有1 B

参考答案 A tRP+tRCD+CL是旧行可立即PRE且无其他冲突时的简化，不是总响应时间公式。

原始资料

1. Micron TN-40-40: DDR4 Point-to-Point Design Guide, Rev. H · 2020 · Micron Technical Note（授权分销商镜像）
2. A Case for Exploiting Subarray-Level Parallelism (SALP) in DRAM · 2012 · ISCA
3. Adaptive-Latency DRAM: Optimizing DRAM Timing for the Common-Case · 2015 · HPCA
4. Ramulator 2.0: A Modern, Modular, and Extensible DRAM Simulator · 2024 · IEEE Computer Architecture Letters（2023预印本）
5. Ramulator 2.1: A Composable Memory System Simulator for Modern DRAM Systems · 2026 · arXiv预印本

模块 05 · 核心必会

刷新与可靠性：弱单元决定了什么

刷新不是后台免费保养。它消耗能量和可服务时间，又把温度、制造偏差、数据模式和错误防护带进体系结构。本章用RAIDR与真实器件论文解释：为什么一个看似简单的“少刷一点”方案，落地需要完整的可靠性闭环。

学习目标

- 区分retention、tREFI、tRFC和刷新窗口
- 估算刷新占用并识别估算范围
- 理解RAIDR的分箱与Bloom filter安全方向
- 解释VRT、数据模式依赖和扰动错误为何限制激进优化

1. Retention 是单元能撑多久，不是芯片统一常数

单元在没有被有效恢复时，电荷逐渐变化，直到无法可靠区分原来的数据状态。能够安全保留数据的时间称为retention time。这个时间依赖单元、条件和判定余量；一条行中的任意关键bit丢失都可能影响整行正确性，因此按行管理时常以最弱单元作为约束。

用小水桶漏水作比喻只适合记住“需要补充”，不能推导精确时间。真实泄漏与器件状态、温度、偏置和邻近数据有关。**温度升高通常让保持变难，但不应把一个固定倍数当作全器件物理定律。**某标准在温度区间内要求提高刷新率，是产品保证条件；它不等于每个单元的保持时间恰好按同倍数变化。

2013年的器件研究在真实DDR3上观察到数据模式依赖DPD和可变保持时间VRT：同一单元的表现可能受到其他单元数据影响，也可能随时间切换。它告诉架构师，可靠性不是一次短测试就能永久生成的静态标签。Retention study：实测DPD与VRT

2. tREFI 与 tRFC：间隔和忙碌时间是两回事

刷新窗口表达需要在多长时间内完成所要求的覆盖；tREFI表达刷新命令的平均安排间隔要求；tRFC表达一次对应刷新操作后资源忙碌多久。一次REF通常在芯片内部推进刷新地址计数，覆盖的行/组由具体设计和模式决定。**不能认为一次REF只刷新一行，也不能认为一次REF刷新整个芯片。**

教学假设：要求64 ms内执行8192次相应刷新命令，则平均间隔为`64 ms / 8192 = 7.8125 µs`。若每次使整个Rank不可服务350 ns，那么简化时间占用比例为`350 ns / 7.8125 µs ≈ 4.48%`。这里的64 ms、8192和350 ns只是一组计算假设，实际要查看器件、温度、模式和密度。

4.48%不是“系统性能必降4.48%”。刷新可能与其他Rank或通道的工作重叠，也可能打断关键依赖链并引起排队放大。若采用更细粒度刷新，还要明确阻塞的是哪些Bank。协议允许的推迟/提前次数也有限，不能用平均间隔掩盖长时间欠账。不同刷新覆盖范围在现代器件中需单独核对。Micron DDR5：刷新相关组织特性

3. 刷新调度：省时间与藏时间不是同一件事

降低必要刷新次数是在减少工作量；把刷新放到空闲间隙是在隐藏开销；把刷新粒度缩小是在释放其他资源的可用性。这三条路线可以组合，但评估不能混为一谈。一个方案把平均带宽损失藏住了，仍可能增加尾延迟；一个方案降低REF计数，也可能增加显式ACT/PRE和控制器工作。

教学案例：两个Rank共享DQ。Rank A刷新时可选择服务Rank B，从而减少总线空洞，但如果当前所有请求都落在A，B的存在无法帮忙。再把负载改成A有关键短请求、B有大量流式读，优先安排B虽然提高带宽，也可能让A的P99很差。

自刷新由器件内部维持保持，适合某些低功耗状态；正常工作中的控制器发刷新则还参与调度。进入/退出低功耗、维持数据与恢复可访问状态各有约束，不能把“处于自刷新”当作随时可零延迟唤醒。

实践上应同时统计刷新命令数、阻塞时长、被隐藏的时长、队列延迟与系统完成时间。用模拟器时先确认所选标准实现的刷新粒度，别以为把配置字段名改成DDR5/HBM就已自动建模所有维护机制。Ramulator 2.1：标准特定功能与验证

4. RAIDR：给弱行勤刷新，给强行少刷新

RAIDR的思路是按保持需求分箱：弱行仍频繁刷新，其他行采用较长间隔；控制器用Bloom filter压缩保存需要较快刷新的行集合。原论文报告的是其模型和仿真条件下的刷新、功耗与性能收益，并非一个现成内存条开关。RAIDR §3与评估

Bloom filter有假阳性而没有其数学模型中的假阴性：本来不在“弱行集合”的强行可能被判断为弱，于是多刷几次，损失效率但不因这个误判少刷。若一个真正弱行没被正确测出并放入集合，则属于**profile错误**，不是Bloom filter替你解决的问题。后来变弱的行也会造成类似风险。

原创教学推导：假设1000行中20行必须每64 ms刷新，80行每128 ms，900行每256 ms。在256 ms观察窗，统一64 ms刷新需4000次行恢复；理想分箱需 `20×4+80×2+900×1=1140` 次，减少71.5%。如果Bloom filter把另30个强行误判到64 ms档，会多出90次，变1230次，仍正确但收益下降。这只是分箱计数，不是RAIDR实测数字。

实现还必须考虑标准REF命令通常不能由控制器任意指定内部行；行级差异化管理可能需要显式ACT/PRE等机制及其额外命令代价。不能仅跳过某些REF，然后假定被跳过的恰好是强行。

5. 为什么一次出厂测试不够

如果保持时间始终不变，只需测量、分类、长期使用。VRT破坏了这个简单前提：某个单元在测试时处于较强状态，之后可能进入较弱状态；DPD又让测试图样是否覆盖不利情况变成问题。加长测试时间、更多数据模式和温度覆盖能改善信心，但不等于穷尽一切。Retention study：保持时间建档挑战

产品方案需要给出闭环：条件变化时如何回退保守刷新，如何重新表征，ECC事件如何触发告警或修复，极端条件如何覆盖，测试成本是否可接受。这里的“回退”不是让用户丢了数据再重试，而是在安全条件不再成立之前恢复保证范围。

教学反例：某行在三次测量中可保持300 ms，你把它设为256 ms刷新；第四次它切换到90 ms状态，在下一轮表征之前就可能失效。即使Bloom filter完美、调度完全按时，也不能弥补这个物理前提错误。所以论文方案评估应分开报告“机制在准确profile下的潜在收益”和“如何获得并维护profile”。

SoftMC这类FPGA平台的价值，是在可控温度、命令和数据模式下直接检验假设；它不是一套普通应用benchmark能替代的工作。SoftMC：可控器件表征平台

6. Retention、RowHammer、RowPress别混在一起

保持错误主要问“不干扰它，它多久需要恢复”；RowHammer问“反复激活附近行，是否会让它提前出错”；RowPress则提醒“让附近行长时间保持打开，也可能影响扰动风险”。它们都可能表现为bit翻转，但触发条件与防护统计不同。RowHammer原始实测；RowPress：行打开时长

对架构防护而言，仅统计访问次数可能遗漏不同驻留时间的风险；仅提高全局刷新又可能付出很大能量和性能代价。需要结合器件特征、监测粒度、刷新/限流机制和最坏情况分析。历史论文中的触发次数属于当年样品与实验设置，绝不能拿来当2026器件的安全阈值。

这里也能修正“row hit越多越好”：减少ACT可能改善带宽与激活能耗，但保持行打开的策略还需满足可靠性约束。性能策略和可靠性机制可能争用同一组命令资源，方案评估应同时计算开销。

面试讨论聚焦检测、防护与可靠性验证即可：你要体现的是错误机理会改变架构假设，而不是背某个攻击阈值。一个优秀回答会主动区分容量故障、保持故障、时序故障、扰动故障和链路故障，再说明哪个保护层覆盖它。

7. 有 ECC 并不意味着可以随意少刷新

ECC只在其编码、错误模型和保护范围内提供能力。单bit纠错对同一编码字内多个相关错误可能不够；一个机制减少平均错误率，也不保证最坏错误模式满足纠错能力。行级或区域级相关错误尤其不能被独立随机bit模型轻描淡写带过。

on-die ECC在器件内部纠正某些阵列错误，系统从外面看到的已经是纠正后的结果；它不能直接告诉你原始单元错误率，也不自动保护离开器件后的传输。EIN论文专门研究这种可见性变化，提醒我们不要把“外部测试没错误”解释为“内部完全无错误”。EIN：on-die ECC改变观察结果

设计减少刷新的研究实验，可以同时报告原始条件、外部可见错误、是否存在透明ECC、统计置信和覆盖范围。如果无法获知内部ECC，结论就应限于外部可见行为。面试中坦诚这一测量边界，比给出一个漂亮但无依据的零错误保证更专业。

你的最终判断框架可以是：先证明节省了哪些维护工作，再证明正确性前提可以被持续维持，最后把新增监控、纠错、回退与测试成本计入总收益。刷新优化真正难的通常是第二步。

面试问答与追问

1 为什么不能把tREFI翻倍来直接省一半刷新功耗?

必须先证明所有受支持条件下单元仍能保持数据；温度、DPD、VRT和扰动可能破坏这个前提。即使REF次数减半，系统总功耗也不一定减半，因为刷新只是功耗组成之一。需要可靠性闭环和器件保证。

继续追问

- RAIDR怎样减少一刀切保守性?
- 如何测量而不被on-die ECC遮蔽?

2 RAIDR为什么适合用Bloom filter?

它把需要更频繁刷新的弱行放入集合，假阳性会让强行被多刷，属于保守效率损失。滤波器不制造假阴性并不意味着profile不会漏判；VRT或建档错误仍可能让真正弱行未被保护。

继续追问

- 为什么不能只跳过某几次标准REF?
- 分箱越细是否越好?

3 tRFC/tREFI能否直接等于系统性能损失?

只能在明确刷新范围及阻塞模型下近似表示资源时间占用。系统可通过其他Bank/Rank/Channel隐藏部分开销，也可能出现关键请求与排队放大；最终要测完成时间、延迟和带宽。

继续追问

- all-bank和per-bank刷新如何改变分析?
- 如何判断刷新被隐藏了多少?

4 RowPress给性能调度带来什么启示?

可靠性风险不只由激活次数决定，行打开时长也可能重要。过分追逐row hit、长期保持行打开需要纳入器件限制与防护；应同时评估性能和可靠性，而不是把低ACT计数直接当安全证明。

继续追问

- 计数型防护需要增加什么状态?
- 如何设计不依赖某一历史阈值的论证?

本章自测

1 教学假设tRFC=400 ns，tREFI=8 μs，简化全Rank刷新时间占比为?

A. 0.5%

B. 5%

C. 20%

D. 50%

参考答案 B 400 ns/8000 ns=5%；这不是系统性能必然损失5%。

2 RAIDR弱行Bloom filter的假阳性主要导致?

A. 弱行被少刷

B. 强行被多刷

C. ECC自动失效

D. 刷新地址计数永远停止

参考答案 B 误判为弱行会更保守，多消耗刷新。真正危险的是保持时间建档漏判或后续变化。

3 外部读测始终正确，能推出内部单元完全无错吗?

A. 能，因为RD不经过ECC

B. 能，只要测试超过一分钟

C. 不能，on-die ECC可能已纠正部分内部错误

D. 不能，因为所有DRAM都必定损坏

参考答案 C 外部可见结果经过内部纠错变换，必须明确观察范围。

原始资料

1. RAIDR: Retention-Aware Intelligent DRAM Refresh · 2012 · ISCA
2. An Experimental Study of Data Retention Behavior in Modern DRAM Devices · 2013 · ISCA
3. Flipping Bits in Memory Without Accessing Them: An Experimental Study of DRAM Disturbance Errors · 2014 · ISCA
4. RowPress: Amplifying Read Disturbance in Modern DRAM Chips · 2023 · ISCA
5. Understanding and Modeling On-Die Error Correction in Modern DRAM: An Experimental Study Using Real Devices · 2019 · DSN
6. Ramulator 2.1: A Composable Memory System Simulator for Modern DRAM Systems · 2026 · arXiv预印本
7. Micron DDR5 SDRAM Product and Feature Overview · 2026 · Micron官方产品页（访问于2026-10-30）
8. SoftMC: A Flexible and Practical Open-Source Infrastructure for Enabling Experimental DRAM Studies · 2017 · HPCA

模块 06 · 深入研读

芯片架构取舍：性能、面积、能量、良率与可靠性

岗位里“架构设计、指标拆解、可行性与有效性”真正考的是这章：你能否把客户的带宽或功耗目标，拆成阵列、外围、接口、控制策略和验证工作的责任，并明确每个收益对应的代价。

学习目标

- 把客户指标拆成可验证的芯片级约束
- 区分单次访问能量、平均功耗和系统任务能耗
- 解释冗余修复、on-die ECC与系统ECC的边界
- 给出包含性能、良率、测试和兼容性的方案评估

1. 先定义目标：不是所有指标都能同时最好

DRAM架构一般同时面对容量、bit成本、首次访问延迟、持续带宽、平均与峰值功率、可靠性和量产测试成本。不能只写“提高性能、降低功耗”，因为这些目标可能互相冲突。更短位线可能更快却增加感放摊销；更多Bank可增加并行机会却增加外围；更激进时序可能改善典型性能却压缩最坏条件余量。

先把目标写成可验收形式：在指定容量、温度、电压和工作负载下，达到某个有效带宽及P99延迟；同时限制芯片面积、热功率、错误目标和接口兼容性。接着注明哪些是硬约束，哪些是可以加权比较的优化目标。

教学例：方案A吞吐提高15%，但可用容量下降20%；方案B只提高8%，容量不变。若用户的模型刚好能装入原容量，A可能导致频繁外溢，端到端反而更慢。你熟悉GPU模型部署，这正是把系统知识带进芯片架构判断的切入点。

TL-DRAM是一个很好的“先识别矛盾，再引入异构”案例：它并非宣称所有存储都同时变得又便宜又最快，而是创造不同延迟区域，再决定如何使用。TL-DRAM：密度与延迟折中

2. 面积不能只数bitcell，还要算外围与可售容量

芯片面积包括存储单元、感放、译码、全局布线、I/O、时钟与供电相关电路、测试、冗余与纠错元数据等。新增一小块逻辑的百分比开销看起来很低，但若它落在紧密阵列内部、改变版图规则或关键布线，落地复杂度可能远大于百分比本身。

一个简化bit成本模型为 `单位可售bit成本 ≈ 晶圆及相关制造成本 / (每片晶圆可切Die数 × 可售良率 × 每颗可售容量)`。封装、测试和产品分档也需进一步加入。面积更小会改变可切Die数；额外冗余可能增加面积，却提高可售良率；更大容量也可能更难通过全部测试。

教学假设：设计A每片产生100颗Die、良率80%、每颗8单位容量，可售总量640；设计B面积增大导致只有95颗、但修复策略使良率90%，总量684。忽略其他成本时，B尽管更大，单位可售bit反而更便宜。数值纯为说明“最小面积不一定最低成本”。

设计引起的局部速度差异也会影响分档和保守时序选择。DIVA说明物理位置会形成结构化慢区域；架构修改应考虑最慢区域而非只拿平均单元证明可行。DIVA：局部差异与可用时序

3. 功耗要按事件拆账，能量要按任务结账

粗略可把DRAM能量分为ACT/PRE事件、列读写、I/O传输、刷新和背景维持等部分。`总能量 ≈ Σ(事件次数×单次事件能量) + ∫背景功率dt` 是一个有用的账本，但事件能量应避免重复计入基线，并用合适器件模型或测量校准。Micron提供按系统访问活动估算功耗的工具，说明仅知道“容量和频率”不足以准确估算。Micron官方功耗计算器

教学例：相同任务原来10 ms完成、平均DRAM功率4 W，能量40 mJ；优化后8 ms完成、平均4.5 W，能量36 mJ。功率升了12.5%，任务能耗却降了10%。但更高瞬时功率可能超出散热或电流限制，仍不能只凭能量更低就采用。

再区分pJ/bit的分母：是物理总线传了多少bit，还是应用真正使用的有效bit？如果一次64 B事务只有8 B有用，按有效bit算的传输能耗可明显更高。压缩、合并、局部性和存算方案必须统一有效工作量才能比较。

对于堆叠存储，热还会反过来影响刷新和时序条件；“加算术逻辑使搬运减少”与“局部发热使维护增加”需要放进同一预算。把能量省在何处、峰值增加在何处标在路径图上，答案就具体了。

4. 冗余修复、PPR、ECC各自解决什么

冗余行/列通过地址替换绕过某些制造缺陷或已定位故障，通常需要备用资源和重映射信息；资源有限，粒度也有限。PPR（post-package repair）把某些修复能力延伸到封装后的阶段，具体额度、永久性和操作要求依器件。Micron DDR4技术说明给出了其PPR实现例子，不应把该例子当所有厂商统一额度。Micron TN-40-40：PPR

ECC通过冗余编码在读出时检测/纠正一定错误模式。它不是搬走坏行，也不能纠正任意多bit或任意相关故障。永久单元故障可以消耗纠错余量，若再叠加新的软错误就可能超过能力；因此修复、巡检和ECC常是协同层次，而不是三选一。

on-die ECC主要在芯片内部覆盖阵列数据路径的一部分；**系统/链路保护**覆盖的区间由编码放置决定。系统ECC通常通过控制器与额外存储校验位保护更长的数据路径，但地址命令错误、错误目标地址读取等情形是否覆盖，还取决于额外CA校验、CRC及完整端到端设计。不能把“ECC内存”当覆盖所有故障的总开关。

最重要的面试动作是画保护边界：编码在哪里生成、校验在哪里检查、哪些线和存储节点在边界内。EIN证明内部纠错还会改变外部看到的错误分布，测试解释必须带上这一层。EIN：内部纠错与观测边界

5. 测试不是最后补上的附件

新架构至少需要验证功能、时序、保持、功耗、电气条件和制造可测性。PVT表示工艺、电压、温度变化；此外还要考虑数据模式、地址位置、老化、反复访问和维修后的映射。平均benchmark通过，不能证明最弱单元、最不利图样和最坏供电条件通过。

分层验证能避免用错工具：电路仿真回答电荷/噪声和时序余量；器件级实验回答真实样品行为；协议模型检查命令合法性；系统模拟回答工作负载效果；量产测试方法决定这些性质是否能被成本可接受地筛选。每层支持不同结论，不能互相替代。

教学评审问题：有人说“我把tRCD缩短20%，模拟器没有报错，所以芯片能更快”。应回答：模拟器是否只把tRCD当输入常数？若是，它只说明调度按新的约束执行，并没有证明电荷能在更短时间可靠感测。必须补电路或器件证据。SoftMC论文的实验方法正好帮助理解这个断层。SoftMC：真实器件与可控命令

同样，某论文用SPICE证明一个单元操作可行，也不意味着完整芯片已经解决全局布线、测试时间、接口标准和软件协调。专业评审不是否定论文，而是把“已证明”和“尚需证明”拆开。

6. 把“有效带宽增加20%”拆成责任表

教学任务：客户说GPU读小块数据时有效带宽不够，要提升20%，功率不能超过预算。第一步先确认基线：有效字节定义、事务大小分布、读写比例、并发度、热点和温度；第二步把损失分到器件、控制器和软件层；第三步提出可测假设。

观察	候选机制	要补的证据
很多ACT，单次激活有效数据少	布局/映射、调度、更多内部并行	每ACT有效字节、Bank冲突、功率

观察	候选机制	要补的证据
row hit很多但DQ有规则空洞	跨Group交错、列路径改造	tCCD约束与命令轨迹
DQ满而有效字节少	更合适事务粒度、合并或压缩	物理/有效流量比与编解码代价
低负载就很慢	检查固定路径、器件服务和依赖	拆分排队与非排队延迟
高温下维护显著增加	热、刷新组织和负载安排	温度与刷新占用的联动

这份表就是架构岗位的工作方式：让“性能问题”变成可分工、可验证的技术问题。不要在缺少trace前直接决定加Bank或换HBM。

7. 面试方案评审：三种提速建议怎么选

方案A加倍Bank，方案B缩短位线，方案C只优化控制器调度。先判断基线限制：如果已被DQ带宽卡住，A/B的直接吞吐收益可能有限；如果大量Bank conflict，A可能有帮助；如果低并发的依赖链暴露首次感测，B更相关；如果请求分布良好但调度制造空洞，C可能最划算。

再问实施范围：A/B可能涉及芯片组织和版图，C可能只改控制逻辑；但C若需要内部Subarray映射或新命令，也并非纯软件开关。SALP提示我们，可以通过有限结构变化利用隐藏并行度，却仍需要明确共享资源和协议支持。SALP：释放子阵列并行的代价

最后给出实验矩阵而不是拍脑袋定案：保持容量与工作负载一致，比较平均/P99延迟、有效带宽、每任务能量、面积、可靠性条件和兼容性；至少覆盖顺序、随机、热点、不同并发度与读写混合。对未实现的硬件代价用估算区间表达，不伪装成精确流片结果。

推荐口头结尾：“我会先用trace证明瓶颈，再选择最小必要改动；如果触及阵列时序，必须给出物理可行性证据；最终以可售容量、可靠性和端到端工作量归一化比较。”这句话体现的是可审查的判断过程。

面试问答与追问

1 面积增加2%的架构改动一定值得拒绝吗？

不一定。要看可售容量、良率、性能、能耗和封装测试总成本，也要看新增结构落在哪里。少量外围逻辑与阵列内部插入新器件的实现难度不同；可以用单位可售bit成本和目标工作量收益比较。

继续追问

- 为什么论文面积百分比不能直接当量产结论？
- 多冗余如何同时增加面积和降低成本？

2 on-die ECC为什么不能替代系统ECC?

保护边界不同。on-die ECC在内部编码范围纠正阵列错误，离开该边界后的DQ/板级/控制器路径不自动被覆盖。系统ECC、链路CRC、CA校验和端到端设计各有作用，需画出编码与检查位置。

继续追问

- 地址错误是否一定被数据ECC发现?
- 透明ECC如何影响实验?

3 功率提高但任务能耗降低，能否接受?

可能接受，也可能不行。需同时满足峰值电流、热功率和温度约束；任务时间缩短可能降低总能量，但瞬时热点可能触发降频或更多刷新。应以相同工作量的总能量和受限条件比较。

继续追问

- pJ/bit的bit是物理传输还是有效数据?
- 怎样避免重复计入背景能量?

4 你如何回应“把Bank翻倍就把带宽翻倍”?

只有基线被Bank内部并行能力限制且新增请求能均衡分布时才可能明显改善；DQ、命令、tRRD/tFAW、Group列通路和上游并发可能先成为瓶颈。还需考虑外围面积、功率和良率。

继续追问

- 给一个带宽完全不变的反例。
- 给一个延迟降低但总带宽不变的场景。

本章自测

1 任务从10 ms×4 W变成8 ms×4.5 W，能耗如何变化?

A. 40 mJ变36 mJ

B. 40 mJ变45 mJ

C. 32 mJ变36 mJ

D. 无法用P×t估算

参考答案 A 平均功率乘时间：4×0.010=0.040 J，4.5×0.008=0.036 J。

2 哪个论证不足以证明缩短阵列时序的物理可靠性?

A. PVT下电路分析

B. 真实器件表征

C. 只改模拟器输入时序且模拟器没报协议错

D. 结合图样和温度的长期测试

参考答案 C 时序模型通常把物理时间当输入参数，不能自行证明更短参数在硅片上成立。

3 关于PPR和ECC，哪项正确？

A. 两者都是把坏单元数据搬到CPU缓存

B. PPR使用修复资源替换故障区域，ECC按编码能力纠错

C. ECC能纠任意数量错误

D. PPR资源无限且所有器件额度相同

参考答案 B 修复和纠错机制、粒度、资源及保护范围不同，常需要协同。

原始资料

1. Micron TN-40-40: DDR4 Point-to-Point Design Guide, Rev. H · 2020 · Micron Technical Note（授权分销商镜像）
2. A Case for Exploiting Subarray-Level Parallelism (SALP) in DRAM · 2012 · ISCA
3. Tiered-Latency DRAM: A Low Latency and Low Cost DRAM Architecture · 2013 · HPCA
4. Understanding and Modeling On-Die Error Correction in Modern DRAM: An Experimental Study Using Real Devices · 2019 · DSN
5. Design-Induced Latency Variation in Modern DRAM Chips · 2017 · SIGMETRICS / POMACS
6. Micron DRAM Power Calculators · 2026 · Micron官方设计工具（访问于2026-10-30）
7. SoftMC: A Flexible and Practical Open-Source Infrastructure for Enabling Experimental DRAM Studies · 2017 · HPCA

模块 07 · 深入研读

论文变成面试能力：五个经典机制与反例

不要求你背论文收益百分比。要求你能解释一个瓶颈、一条物理观察、一处结构修改、一组适用条件，以及你会如何证明它值得做。下面把SALP、TL-DRAM、AL-DRAM、RowClone和RowHammer训练成可复述的架构案例。

学习目标

- 按问题—观察—机制—代价—证据复述论文
- 区分电路模型、系统模拟、真实器件与真实系统证据
- 用相同目标比较多个优化方向

- 对每篇论文提出能改变结论的反例和验证实验

1. 每篇只记一张因果卡，然后训练反例

第一遍读摘要和图，只回答“作者要消除什么等待或搬运”；第二遍读背景和机制，把修改圈在自己的DRAM结构图上；第三遍读评估方法，确认是否修改容量、时序、映射和工作负载；最后读局限，自己再构造一个收益很小的场景。**面试官更容易通过反例判断你是否理解论文。**

用五句模板：原系统被X限制；作者发现Y没有被利用；通过Z结构/策略释放Y；代价是A，前提是B；证据来自C，所以只能支持D范围的结论。对精确数字，只有同时记得基线、配置、指标和证据类型时才使用。

例如“内存复制快十倍”可能指一个原语，而不是应用快十倍。**原创教学Amdahl例：**假设复制占程序时间30%，把该部分加速10倍，其余不变，总加速为 `1/(0.7+0.3/10)≈1.37×` 。若为布局和一致性新增5%原程序时间，则变为 `1/(0.7+0.03+0.05)≈1.28×` 。这条算式先于任何论文数字。

2. SALP：隐藏在 Bank 里面的并行机会

论文机制：Bank内部不同Subarray具有局部独立操作。SALP-1交叠一个Subarray的PRE与另一个的ACT；SALP-2增加局部行地址保持等支持，让另一Subarray的ACT能与当前写恢复交叠；MASA再将“保持活动”与“被选中驱动全局I/O”解耦，允许多个活动局部行并选择一个服务列命令。SALP，ISCA 2012，§4–5

你要能解释的矛盾：内部局部电路有独立性，但共享控制/数据路径让外部调度串行化。解决方法不是把一个Bank名字改成很多Bank，而是逐项解除不必要的共享限制，同时保留真正共享的数据口约束。

原创反例训练：如果所有请求都在同一个Subarray的两个不同Row间抖动，SALP的跨Subarray并行机会就很少；如果DQ已经满载，更多内部并行也不保证吞吐继续增加。若热行分布在多个Subarray，MASA保存多个局部活动行才可能明显减少重复ACT。

验证建议：准备同Bank同Subarray、同Bank跨Subarray、跨Bank三组trace；比较ACT/PRE数、关键路径等待、总线占用和尾延迟。再把Subarray地址未知作为实现约束，询问控制器怎样获得映射。论文的面积估计与系统模拟支持设计潜力，不构成已量产证明。

3. TL-DRAM：把长位线切成不同速度的区域

论文机制：在长位线上插入隔离晶体管，形成靠近感放的near segment和远端far segment。访问近段时隔离远端负载，获得较小等效电容；访问远段则经过额外器件，其时序代价不能按近段结果外推。再用硬件或软件管理近段数据。TL-DRAM，HPCA 2013

你要能解释的矛盾：长位线便宜，因为更多单元分摊感放；短位线更容易感测和驱动。分段并没有消除矛盾，而是给同一芯片提供一个小而快、一个大而相对慢的选择，再让数据放置策略决定收益。

原创教学判断：假设近段访问30 ns、远段60 ns，80%访问命中近段，平均服务时间为36 ns；若近段只覆盖20%，平均变54 ns。再加入迁移、查询与排队成本，就可能失去优势。这两个时延和命中率完全是教学假设，不是论文参数。

反例：热点快速变化、流式只读一次的数据、近段频繁抖动或迁移占用共享资源，都会削弱收益。面试不要把它等同于普通SRAM缓存：它依然位于DRAM阵列，受行、Bank、刷新和内部搬运约束；“更近”也不等于离CPU物理距离更近。

4. AL-DRAM：利用典型条件余量，但把保证范围说清楚

论文机制：不同模块和温度下的实际可用时序可能比统一最坏条件设定更短。AL-DRAM通过器件表征与条件适配减少部分时序等待，并在支持配置时序的真实系统上评估。它与前两篇不同，包含真实器件及系统实验，不只是设一个更小的模型参数。AL-DRAM，HPCA 2015

你要能解释的矛盾：标准时序要覆盖被承诺的广泛条件，而日常运行可能处于更温和条件。研究价值是找出可利用的余量；产品挑战是怎样知道当前余量、温度变化时怎样切换，以及怎样满足长期可靠性。

原创反例：在一个GPU系统中，若内存流量突然升高、热环境改变，而时序适配滞后，就不能仅凭开机时的测试结果维持激进参数。另一个反例是工作负载完全受计算限制，即使DRAM服务变快，应用收益也可能接近零。

面试延伸：把验证分成离线表征、运行条件监测、保守边界和回退机制，再讨论器件老化、测试覆盖和透明ECC。不能把论文“在测试中未出现错误”讲成对所有未来器件和全部条件的数学保证，更不能把个人超频经验当量产规格。

5. RowClone：复制数据为什么一定要经过 CPU

论文机制：FPM利用同Subarray共享感放：先让源行数据进入感放，再用稳定感放状态写入目标行；PSM通过内部共享路径在Bank之间搬运。目标是减少大块复制/初始化占用外部内存通道。该机制要求器件和系统支持，不能在普通DDR上任意发送两个非法ACT就宣称可用。RowClone，MICRO 2013，§3

你要能解释的矛盾：CPU参与复制时，数据从DRAM出来又写回DRAM，计算本身几乎不需要复杂算术，却消耗大量外部搬运。利用已有宽内部路径，可以在适合的布局与粒度下省掉这段绕行。

原创教学案例：复制4 KiB至少有4 KiB读流量和4 KiB写流量，忽略其他开销共8 KiB。如果内部完成且只发控制命令，可释放相应外部数据带宽；但若目的地不与源共享所需结构，就要改用较慢内部路径、搬迁或普通复制。不能把最快模式覆盖率默认设为100%。

系统追问：源数据若在CPU/GPU缓存中有脏副本，先处理一致性；目标缓存副本要失效或更新；地址翻译、权限、页分配和操作原子性也要有定义。面试时指出这些条件，才能把“存内操作很快”变成端到端可用方案。

6. RowHammer到RowPress：物理错误会推翻架构假设

论文事实：RowHammer在真实器件和系统上证明，重复激活一行可能使附近行发生错误，打破“访问某地址不会改变其他地址数据”的隔离假设。RowPress进一步表明，长时间保持行打开也能影响扰动，不能只以激活计数描述所有风险。RowHammer，ISCA 2014；RowPress，ISCA 2023

架构含义：可靠性已不只是内存厂商的单元设计问题。控制器决定激活频率与开行时长，系统决定访问负载，芯片决定物理脆弱性和内部防护；因此防护需要跨层分工。低平均开销也不等于最坏情况下安全。

原创防护取舍题：方案A增加全局刷新，状态简单但所有访问都承担额外维护；方案B精确跟踪每行活动，选择性强但状态开销与行数相关；方案C用压缩或概率结构降低状态，需要分析误判方向和失败概率。不要直接选一个“最先进”方案，先定义阈值不确定性、覆盖范围和可靠性目标。

将它与性能论文对照尤其有用：SALP释放内部并行，可能改变激活密度；open-page提高hit，却改变开行时长；减少刷新改变数据恢复机会。任何新机制都应该重新评估与防护的组合，不能默认原来单独验证的安全结论自动保持。

7. 用同一个需求挑论文，而不是按名气挑

观察到的瓶颈	优先联想到	先排除的反例
同Bank跨Subarray串行等待	SALP	请求全落同Subarray或DQ已饱和
阵列服务慢且热点较稳定	TL-DRAM	近段利用率低、迁移代价高
典型条件有可验证时序余量	AL-DRAM	无法监测条件或保障可靠性
复制/清零消耗大量外部流量	RowClone	粒度/布局不匹配、一致性代价高
密集活动造成邻近数据风险	RowHammer/RowPress防护	用历史阈值代替新器件证据

最后做一个两分钟口述练习：“我熟悉GPU侧访存，但最近把DRAM内部层级补齐。我用SALP理解隐藏并行，用TL-DRAM理解密度与位线延迟，用RowClone理解减少数据搬运。我的判断方法是先用请求轨迹定位瓶颈，再检查物理修改、数据布局和可靠性条件，最后评估有效工作量下的收益。”

这段应当只作为学习成果表述；若你随后真正完成模拟实验，再补充你实现了什么、配置是什么、观察到了什么。没有亲自复现的数字，就明确说是论文报告。**会分析论文已经有价值，不需要把阅读冒充项目经历。**

面试问答与追问

1 用一分钟讲SALP，并给一个收益不明显的负载。

它观察到Bank由多个局部Subarray组成，部分操作本可独立，却被共享控制与全局I/O限制串行化。SALP逐步交叠PRE/ACT、写恢复/ACT，并用MASA选择一个活动Subarray驱动全局口。若请求全落同Subarray或总线已经满载，收益可能有限。

继续追问

- SALP-2为何不能像MASA那样保留多个行服务列命令？
- 如何获得Subarray映射？

2 TL-DRAM为什么不是“所有访问都更快”的故事?

近段隔离远端电容而更快，容量小；远段仍有更大负载和隔离器件影响。最终依赖热点覆盖、放置/替换和迁移成本。应比较同容量、同工作量并计入远段与管理代价。

继续追问

- 近段应硬件管理还是软件管理?
- 热点变化时怎样降低迁移抖动?

3 RowClone需要哪些CPU/GPU系统支持?

除器件内部操作外，还需要可用的源目标布局与粒度，处理缓存脏副本和目的副本一致性，定义地址翻译、权限、完成通知和原子性。否则内部复制很快也无法保证系统看到正确数据。

继续追问

- 源目标跨Subarray如何处理?
- 对4 KiB页复制，最快模式覆盖率怎么测?

4 AL-DRAM的证据比纯仿真强在哪，又弱在哪?

它包含真实模块表征和真实系统时序调整，因此物理现象与系统收益不只是模拟器假设；但证据限于测试样品、条件和覆盖范围，不能直接担保所有新代际器件或长期最坏情况。

继续追问

- 你会如何设计安全回退?
- on-die ECC怎样改变误码观测?

5 如果一篇论文原语快10倍，应用中该原语占30%，理论应用加速是多少?

在其余部分不变且没有额外成本时，Amdahl定律给出1/(0.7+0.3/10)≈1.37倍。还要计入数据放置、一致性和管理开销，不能把原语倍数直接当应用倍数。

继续追问

- 如果新增5%原程序时间开销呢?
- 如何测量原语占比而不被重叠误导?

本章自测

1 MASA相对只允许重叠激活的机制，关键新增能力是?

A. 让所有感放同时驱动同一根全局线

B. 多个Subarray保持活动，但明确选一个驱动全局I/O

C. 取消刷新

D. 把DRAM变为SRAM

参考答案 B 保持活动和选中输出必须解耦，避免全局数据路径冲突。

2 RowClone FPM最关键的数据放置条件是?

A. 源目标在任意不同DIMM

B. 源目标共享适用的局部感放结构，通常同Subarray

C. 数据必须是浮点数

D. 只要CPU缓存命中即可

参考答案 B 最快复制利用局部共享感放，跨结构时需要其他模式或回退。

3 哪种面试表述最严谨?

A. 论文模拟提高20%，所以产品必提高20%

B. 读过论文就算参与了该项目

C. 在论文配置中模拟有效；我会检验映射、功耗与可靠性假设再判断适用性

D. 没有流片的论文都没价值

参考答案 C 区分机制价值、证据类型和外推范围，既不夸大也不无理由否定。

原始资料

1. A Case for Exploiting Subarray-Level Parallelism (SALP) in DRAM · 2012 · ISCA
2. Tiered-Latency DRAM: A Low Latency and Low Cost DRAM Architecture · 2013 · HPCA
3. Adaptive-Latency DRAM: Optimizing DRAM Timing for the Common-Case · 2015 · HPCA
4. RowClone: Fast and Energy-Efficient In-DRAM Bulk Data Copy and Initialization · 2013 · MICRO
5. Flipping Bits in Memory Without Accessing Them: An Experimental Study of DRAM Disturbance Errors · 2014 · ISCA
6. RowPress: Amplifying Read Disturbance in Modern DRAM Chips · 2023 · ISCA

控制器与 PHY：把请求变成合法、高效的 DRAM 命令

你熟悉 SoC/GPU 架构，可以把控制器看成同时懂业务优先级与 DRAM 时序的调度器。补齐的关键不是背命令缩写，而是能解释：同样的 DRAM，为什么换地址映射、请求次序或刷新策略，吞吐量和尾延迟会改变。

学习目标

- 画出系统请求—控制器—PHY—DRAM 的边界
- 解释 row hit、bank 并行、调度公平性的冲突
- 给出读写切换和刷新调度的量化评估方案

1. 三个角色：决定先做什么、如何传过去、芯片如何响应

控制器（memory controller）接收来自缓存/互连的读写请求，拆分地址、维护队列与 bank 状态，检查时序，安排 ACT、RD、WR、PRE、REF 等命令。还可能承担 QoS、写合并、错误处理与系统级 ECC。具体功能分布依 IP 而变。

PHY（physical layer）处理高速电气接口：时钟与采样、串并转换、驱动与接收、延时校准等。训练会联合控制逻辑、PHY、固件与 DRAM；不能一概说全部在 PHY 或全部在控制器。DFI 规定控制器到 PHY 的接口，不等于 GPU 到控制器的协议，也不等于 DRAM 引脚接口。DFI 官方说明

DRAM die执行阵列侧的译码、感应放大、恢复、列选择和刷新。在公开标准范围内，控制器知道发命令的合法条件；它通常不知道每个 cell 的真实模拟电压。架构提案必须明确是只改调度软件/控制器，还是要改芯片内部电路或接口。

```
请求 → 地址映射 → 读/写队列 → 调度器 + 时序状态
     → 命令/数据接口 → PHY → DRAM 引脚 → bank/子阵列
返回数据 ← 响应重排、错误检查 ← PHY ← DRAM
```

2. 地址映射：同一串字节，落到哪里很重要

物理地址需要拆成 channel、rank、bank group、bank、row、column、byte offset。排列不是全球统一公式，真实芯片还可能使用 XOR/hash。不要凭一张通用图声称某款 GPU 的 bank 位。

想象连续读取一大块张量。把较低位分给 channel/bank，能让相邻块分散到不同资源，扩大并行；把更多连续地址留在同一行，可以减少 ACT/PRE。两者不总同时最优：过度集中会产生热点，过度打散也可能失去行复用。stride 若恰好重复映射到少数 bank，会出现规律性带宽低谷。

面试分析顺序：先说明请求粒度和映射，再说明每个 bank 的打开行，再数 row hit/conflict，最后考虑共享命令/数据总线和时序限制。不同 bank 能并行准备，不代表它们可以同时占用同一条数据总线。Rixner 的经典工作说明了为什么 DRAM 结构感知的重排能改善实际访问效率。Memory Access Scheduling，§3–5

3. 一次读的时间：先把队列与阵列拆开

端到端读延迟可以拆为互连时间、控制器排队、命令等待、阵列操作、数据传输与返回。行命中只是其中一段变短。**hit** 表示目标行已打开；**closed-bank miss** 表示没有打开行，要先 ACT；**conflict** 表示开着另一行，要先满足关闭条件，再 PRE、ACT。

教学假设：令 tCL=tRCD=tRP=15 ns，忽略队列、burst 与其它约束；已满足可发命令条件。则首数据时间可粗算为 hit 15 ns、closed 30 ns、conflict 45 ns。若三者占比 60%/20%/20%，平均为 0.6×15+0.2×30+0.2×45=24 ns。这不是任何器件的规格，也不是完整 GPU load latency。

如果为了增加行命中，某请求在队列多等 100 ns，阵列侧节省 30 ns 仍可能得不偿失。反过来，让多个 bank 在一个请求等待 ACT 时准备其它请求，可提高吞吐量。**命中率、吞吐量、平均延迟、p99 延迟要同时看。**

4. FCFS、FR-FCFS 与 QoS：为什么“先服务行命中”不够

FCFS 倾向按到达次序服务，简单但可能出现队头阻塞。FR-FCFS 是 first-ready、first-come-first-served 思想：先利用已经可以执行的请求/命令，再按年龄打破平局；常见实现倾向可发出的行命中。**“ready”必须满足时序和资源约束**，不是看到相同行号就能发。不同实现对命令、请求、开行策略的组合会有差异。

假设流 A 不断访问当前打开行，流 B 的老请求需要换行。无限优先 A，平均带宽可能很好，B 却长期等待。可选补救包括年龄阈值、行命中批次上限、按来源限额、deadline 优先。代价是失去部分局部性、增加状态与仲裁逻辑；实时业务还需要可证明的最坏等待上界。

BLISS 提供一个很好的论文案例：用“同一来源连续被服务的次数”识别可能造成干扰的来源，暂时降低其优先级并周期恢复。它训练的是**性能、公平性、实现复杂度一起优化**的思路。GPU 多租户/多引擎可借鉴，但论文中的 CPU 工作负载和阈值不能直接移植。BLISS，§III–V

5. 读写切换与刷新：总线空转不都是程序问题

同一组 DQ 通常需要在读方向和写方向间切换，并遵守 turnaround 时序。频繁 R/W/R/W 会浪费可传数据的周期。控制器常把写请求暂存在队列中，达到阈值后批量 drain，再回到读优先。批次太小，切换多；太大，读请求尾延迟上升。写队列还要处理同地址相关性与可能的 forwarding。

刷新是保存电荷的必要操作，不能因为应用忙就无限延期。控制器可在标准允许窗口内调整时机，具体 all-bank、per-bank、same-bank 等支持取决于代际。**教学假设：**若每 7.8 μs 有一次 350 ns、完全不可重叠的暂停，简单占用率是 350/7800≈4.49%；实际损失还取决于流量、bank 并行、调度与温度，不能当作通用 DDR5/HBM 开销。

DARP 研究把刷新安排到更空闲的 bank 或写回阶段；SARP 进一步利用子阵列并行，但需要 DRAM 内部支持。这正好用来回答“只改控制器能做到哪一步”。DARP/SARP 原论文

6. 训练、验证与架构交付：不能只交一张带宽图

训练的目标是在工艺、电压、温度和走线差异下找到可靠收发窗口，例如调整采样相位、延迟、参考电压；支持项和流程随接口变化。训练成功解决“信号能否可靠收到”，不会自动解决低行命中、排队或 QoS。温度变化还可能触发重新校准或运行策略调整。

一个可评审控制器方案至少交付：地址映射、命令合法性状态机、调度优先级、刷新规则、队列大小/反压、请求顺序语义、超时与错误行为，以及工作负载和指标定义。读写同地址、队列满、刷新即将到期、极端 bank 热点等是边界场景。

Ramulator 2.0 将 DRAM 标准和控制器模块分开，适合研究策略变化。周期级模拟能检验时序/调度假设，但不会自动证明 PHY 信号完整性、晶体管电路或硅后良率。Ramulator 2.0

术语速记：MC=内存控制器；PHY=物理层；FR-FCFS=优先就绪再按到达次序；QoS=服务质量；write drain=集中处理积压写请求；backpressure=资源不足时阻止上游继续注入。

面试问答与追问

1 FR-FCFS 为什么可能伤害系统性能?

它通常能利用就绪请求和行局部性，但服务偏好可能让低局部性、低并行或延迟敏感流长期等待。我的评估会同时看吞吐量、每流 slowdown、p99 和饥饿上界；可用年龄优先或批次上限补救，再量化吞吐损失和仲裁成本。不能只拿总体 row-hit rate 证明方案更好。

继续追问

- 你如何定义公平性?
- 设置年龄阈值会不会让高带宽流抖动?
- 如果队列深度翻倍，收益和代价是什么?

2 提高 bank 数一定会提高带宽吗?

不一定。它增加可重叠的阵列操作机会，但需要足够独立请求、合适映射和控制器队列；共享数据总线、bank-group 时序、ACT 功耗约束、命令总线仍可能限制吞吐。应先看瓶颈是阵列占用还是总线已饱和。

继续追问

- 同 bank 不同子阵列能否并行?
- tFAW 为什么是调度器必须知道的限制?

3 客户报告低带宽，你如何区分 PHY 与控制器问题?

先区分数据错误/训练降速与无错误但低利用率。确认实际运行速率、错误计数和训练状态，再统计队列、命令/数据总线占用、读写切换、刷新、行命中和 bank 分布。最后用连续流、随机流、单 bank 热点分别隔离接口与调度瓶颈。

继续追问

- 训练通过是否意味着所有温度都稳定?
- 需要哪些硬件可观测计数器?

4 如何向面试官概括 DARP 与 SARP?

DARP 主要研究在允许刷新规则下优化刷新调度，利用 bank 间机会和写回阶段；SARP 更深入，利用同 bank 内子阵列的独立性让访问与刷新重叠，涉及内部组织支持。两者说明架构方案应明确控制器、协议和 DRAM 电路各改什么。

继续追问

- 如何证明不会漏刷?
- 是否能不改 DRAM 就直接部署 SARP?

本章自测

1 同一 bank 打开的是另一行，读取目标行通常先需要什么?

A. 直接发 RD，硬件总能自动换行

B. 在满足时序后 PRE，再 ACT 目标行，再 RD

C. 只提高 PHY 采样频率

参考答案 B 行冲突涉及关闭旧行与打开新行；还必须满足 tRAS、tRP、tRCD 等具体时序条件。

2 哪项最能识别“带宽提升却牺牲交互业务”的调度问题?

A. 只看平均 row-hit rate

B. 只看总完成字节数

C. 同时看每流吞吐、p99 延迟与饥饿情况

参考答案 C 总体平均数可能掩盖低局部性或低并行流的长期等待。

3 DFI 主要定义哪条边界?

A. 控制器逻辑与 PHY

B. GPU warp 与 L1

C. NAND page 与 FTL

参考答案 A DFI 是控制器—PHY 接口规范；其版本和覆盖的存储类型需要明确。

原始资料

1. Memory Access Scheduling · 2000 · ISCA 2000
2. DFI：控制器与 PHY 接口官方说明 · 2026 · DFI Group；2026-10-30 访问
3. The Blacklisting Memory Scheduler: Achieving High Performance and Fairness at Low Cost · 2014 · ICCD 2014
4. Improving DRAM Performance by Parallelizing Refreshes with Accesses · 2014 · HPCA 2014
5. Ramulator 2.0: A Modern, Modular, and Extensible DRAM Simulator · 2023 · 2023 arXiv 版本；IEEE Computer Architecture Letters
6. CMU-SAFARI/ramulator2 官方仓库 · 2026 · CMU SAFARI；2026-10-30 访问

模块 09 · 核心必会

GPU 访存：从 warp 事务一直追到 DRAM 行

已有 GPU 架构基础时，最容易犯的错是把“访存合并、L2 命中、DRAM 行命中、bank 并行”混成一个概念。本章把它们放在不同层级，用流量、并行数与时序把它们连接起来。

学习目标

- 区分合并事务、缓存行、DRAM burst 和 row buffer
- 用带宽—延迟积估算所需并发
- 用 Roofline 和 FlashAttention 解释软硬件取舍

1. 先画路径，再问“哪一级在等”

```
warp 的 load/store
→ 地址生成与转换（TLB 等）
→ 合并访问 / L1 路径
→ 片上互连 → L2 slice / memory partition
→ 内存控制器队列 → PHY → DRAM channel/bank/row
→ 数据沿返回路径唤醒依赖指令
```

这是通用学习框架；cache bypass、原子操作、异步 copy、具体 slice 与 controller 的连接依芯片变化。Hopper 官方架构展示了 SM、L2 与 HBM 控制器分工，但没有公开每个地址位如何映射到 bank。不要把微基准推测说成厂商规格。NVIDIA Hopper 架构说明

一条 load 慢，可能在 TLB、cache miss、互连拥塞、控制器排队、行冲突或数据返回上。先定位层级，才知道应改内核布局、缓存策略、分区映射还是 DRAM。软件能观察的只是其中一部分。

2. Coalescing：把很多小需求装进尽可能少的运输单元

warp 内线程的一次全局访存指令会合并地址请求。以 NVIDIA 文档的 32 B 事务口径为例，32 个线程各读 4 B，总有效数据 128 B；若覆盖四个对齐 32 B 段，只需四个事务。线程编号与地址无需严格单调，只要落在这些段内，置换也可合并。CUDA：Coalesced Global Memory Access

教学推导：若每个线程落在不同 32 B 段，32 个事务共 1024 B，当前指令只使用 128 B，有效比例 12.5%。若相邻线程连续读，但起点偏移 4 B，覆盖五段，则为 128/160=80%。这是事务利用率，不能直接断言执行时间变成 8 倍或 1.25 倍，因为缓存、并发与其它瓶颈会改变结果。

合并不等于行命中。合并发生在 GPU 请求侧；DRAM 行命中取决于更下游地址映射和打开行状态。一个 32 B sector 也不一定对应一次独立 DRAM 命令：L2 可能命中、合并或按器件 burst 粒度处理请求。

3. MLP、BLP、row locality：三种不同的“好访存”

概念	它解决什么	为何不能互相替代
MLP：memory-level parallelism	同时有多个独立访问在飞，隐藏延迟	若全部落到一个忙 bank，未必有高吞吐
BLP：bank-level parallelism	多个 bank 的阵列准备工作重叠	仍可能共享命令/数据总线，也需要足够请求
Row locality：行局部性	复用已打开行，减少切换	集中在少数 bank 可能降低并行或公平性
Coalescing：访问合并	减少无用事务字节	不保证下游有高行命中或高 BLP

教学假设：整个 GPU 想持续达到 3 TB/s，平均内存服务时间为 300 ns。稳定状态的在途数据量约为 BW×latency=900,000 B。若用每项 128 B 的等效请求计数，至少约 7032 项在途；这只是 Little 定律量级估计，不是某 GPU 的真实队列深度。缓存命中、请求合并和各层排队会改变口径。

单条指针链每次要先收到地址才能发下一次请求，依赖限制 MLP，即使峰值带宽很高也用不起来。多个独立 warp/访问链能帮助，但当接口已经饱和，再增加并发主要是延长队列。

4. Occupancy 是资源条件，不是“带宽使用率”

更多驻留 warp 提供更多可调度任务，某 warp 等数据时可以发另一个。但能否隐藏延迟，还取决于就绪指令数量、warp 内依赖、寄存器与 shared memory 占用、未完成请求额度、同步与 cache 行为。高 occupancy 的 kernel 也可能所有 warp 一起卡在同一个依赖上。

反过来，合理增加每线程工作量或使用较大的 tile，可能提高寄存器占用、降低 occupancy，却减少全局流量而更快。异步 copy/双缓冲让下一块数据搬运与当前计算重叠；前提是未来地址可知、缓冲足够、搬运资源有余量，并且同步正确。

调优时分别观察有效带宽、实际 DRAM 流量、L2 命中、活跃/就绪 warp、stall 原因。不要把 profiler 的某一个百分比当成全部事实。Nsight Compute 的 replay 与缓存清理会影响测量，比较时必须保持设置一致。Nsight Compute Profiling Guide

5. Roofline：先判断该加算力、加带宽，还是减少字节

选定同一个存储层级，令 I=运算次数/该层传输字节，计算上限 P，持续带宽 B，则性能上界为 `min(P, B×I)`。Roofline 是上界工具，不是保证达到的预测器；串行依赖、调度、同步等还能把性能压到更低。原始工作也强调用经过缓存过滤的内存流量，并允许建立 L2 等不同层级模型。Roofline 原始报告

教学假设：P=120 TFLOP/s，持续 HBM 带宽 B=3 TB/s，拐点 I*=40 FLOP/B。若 kernel 的 I=4，带宽屋顶只有 12 TFLOP/s；把计算单元翻倍基本无助。若通过数据复用将 I 提高到 20，屋顶升至 60 TFLOP/s；能否达到，还要看实际发射与并行。

注意单位与精度：FMA 通常计 2 FLOP；不能把 FP8 稀疏峰值放到 BF16 稠密 kernel 的屋顶上；也不能用“理论最低字节数”计算 I，却用含重复搬运的实测性能作不一致比较。报告应同时给理想流量、实际计数器流量和差距原因。

6. FlashAttention 与 PagedAttention：两种不同的节省

普通 attention 若把 QK^T 和 softmax 结果完整写回外存，会产生大的中间流量。FlashAttention 按块搬入片上 SRAM，用在线 softmax 的最大值/归一化统计合并分块结果；反向可以重算部分中间值。其核心是减少 HBM 往返，保持标准 attention 的数学结果，并不意味着把密集注意力的算术复杂度变为线性。FlashAttention，§3

教学推导：N=8192 时，仅一张 FP16 的 N×N 中间矩阵就占 128 MiB；32 个 head 若同时完整存储这类矩阵，就是 4 GiB。分块避免完整物化它，不代表 KV cache 也消失。tile 太大还会挤占片上资源，减少并发。

PagedAttention 管理的是不断增长的 KV cache：逻辑连续的 token 可以映射到不连续的物理块，减少预留浪费并支持共享。块小有利于减少碎片，块大可能利于执行和传输效率。它是软件分配/访问机制，不是 DRAM 行缓冲优化。PagedAttention，§4、§7

面试应说清三层区别：FlashAttention 减少算子的中间 I/O；PagedAttention 改善 KV 管理；DRAM 控制器负责最终到达设备的请求次序。三者可以组合，但某层提升不一定线性传到端到端。

面试问答与追问

1 访存已经 coalesced，为什么带宽还很低?

合并只说明同一条 warp 指令的事务利用率。还要检查工作集是否命中缓存、是否有足够 MLP、分区/bank 是否热点、读写和刷新开销、以及 load 是否受依赖或同步限制。先用计数器区分流量不足与服务效率低，再设计微基准隔离。

继续追问

- 连续访存是否保证高 row-hit rate?
- 如何区分 cache 带宽与 HBM 带宽?

2 为什么更多 warp 有时反而更慢?

增加并发能隐藏延迟，但也会增加寄存器、缓存和队列竞争。达到吞吐上限后，更多请求主要延长排队；如果工作集相互驱逐，还会放大流量。应该扫描 block size/驻留资源并观察性能与缓存流量，而不是追求最大 occupancy。

继续追问

- 你如何判断已经超过有用并发点?
- 为什么大 tile 降低 occupancy 却仍能加速?

3 FlashAttention 对存储芯片架构有什么启发?

先减少需要跨层移动的数据，再讨论提高接口带宽。它用片上空间、分块和部分重算换取更少 HBM 中间读写。对芯片设计的启发是联动片上 SRAM 容量、带宽、并发、异步搬运与外存配置，而不是只比较 FLOPS。

继续追问

- 它是否让 attention 变成 O(N) 运算?
- 它是否消除了 decode 的 KV 读取?

4 一块 3 TB/s GPU 是否能在 1 ns 内读完 3 KB?

3 TB/s 描述足够并发下的长期传输速率，不是一次请求首数据延迟。单请求仍有地址转换、排队、阵列和返回开销；只有大量独立访问流水起来，平均速率才可能接近带宽上限。

继续追问

- 带宽—延迟积在这里有什么用?
- 小粒度随机读为什么尤其需要并发?

本章自测

1 32 个线程各读 4 B，覆盖 5 个 32 B 段，按事务口径有效比例是多少?

A. 100%

B. 80%

C. 12.5%

参考答案 B 有效 128 B，传输 160 B，128/160=80%；这不是端到端性能百分比。

2 某 kernel 低于 HBM Roofline 很远，最合理的下一步是?

A. 直接判定需要更多 HBM

B. 检查依赖、MLP、发射和实际流量等额外上限

C. 认为 Roofline 公式失效

参考答案 B Roofline 是上界，不包含所有串行性和资源约束。

3 FlashAttention 的主要价值是什么?

A. 消除所有 KV cache

B. 把标准密集 attention 的运算量变成线性

C. 利用片上分块等方式减少 HBM 中间 I/O

参考答案 C 减少外存中间数据往返与存储是核心；标准 attention 仍有其算术量，KV cache 是另一类状态。

原始资料

1. CUDA Programming Guide：Writing SIMT Kernels · 2026 · NVIDIA 持续更新官方文档；2026-10-30 访问
2. NVIDIA Hopper Architecture In-Depth · 2022 · NVIDIA 官方架构说明
3. Roofline: An Insightful Visual Performance Model for Floating-Point Programs and Multicore Architectures · 2008 · UC Berkeley 技术报告 UCB/EECS-2008-134；2009 CACM 工作的原始报告
4. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness · 2022 · NeurIPS 2022
5. Efficient Memory Management for Large Language Model Serving with PagedAttention · 2023 · SOSP 2023
6. Nsight Compute Profiling Guide · 2026 · NVIDIA 持续更新官方文档；2026-10-30 访问

存储产品地图：DDR、LPDDR、GDDR、HBM、SRAM 与 NAND

面试中的“熟悉存储芯片架构或应用方案”通常会从选型切入。最好的回答是先识别数据的容量、访问与可靠性需求，再选择存储介质、组织、接口和封装，而不是背一个从慢到快的排行榜。

学习目标

- 区分介质、接口标准、封装与系统产品
- 根据工作负载选择内存类型并解释代价
- 正确计算容量、引脚带宽、有效带宽和功耗预算

1. 四层分类，避免把 SSD 与 DRAM 放错位置

介质层：SRAM、DRAM、NAND 决定基本存储机制。**组织层**：bank、row、page、channel 等定义访问并行和粒度。**接口/协议层**：DDR、LPDDR、GDDR、HBM 或 NAND 接口约定如何交互。**封装/系统层**：DIMM、堆叠器件、SSD 等把芯片和控制逻辑组合成产品。

DDR、LPDDR、GDDR、HBM 都属于 DRAM 家族路线，仍需面对感应、恢复、刷新和阵列时序。HBM 的“堆叠”和 NAND 的“3D 单元层数”也不是同一维度。HBF 基于 NAND，不能因为名字与 HBM 接近就把它当 DRAM。

类别	核心长处	主要约束	典型位置
SRAM	低访问延迟，适合高频小容量访问	每 bit 面积与漏电成本高	寄存器附近、cache、片上缓冲
DRAM	容量、成本、可重写性能较均衡	刷新、行操作与外部接口	系统内存、GPU 工作内存
NAND	高密度、非易失	页/块粒度、写擦除与寿命、ECC	存储、分层容量、HBF 候选数据层

2. DDR：通用主存是系统工程，不只是数据率

标准 DDR 常用于通用计算的系统内存，DIMM 生态方便配置容量和维护。但芯片、rank、module、channel 是不同实体：多个 x8 die 可以组成更宽的数据接口；增加 rank 不等于增加独立数据总线。控制器需在容量扩展、负载、速率与可靠性之间取舍。

教学计算：假设一个总数据宽度 64 bit 的 DDR 接口工作在 6400 MT/s，原始数据带宽是 $6400\times10^{6}\times64/8=51.2$ GB/s。6400 MT/s 已经表示每秒传输次数，不能再乘 2。若系统是两个 32 bit 数据子通道，把它们分别相加即可；ECC 额外位不应算成应用有效数据。

真实有效带宽还会扣除命令/时序空隙、刷新、读写切换、低局部性和不满载因素。可写为 `有效带宽≈原始带宽×利用率`，但利用率必须来自明确工作负载的测量或模型。厂商的标准分类资料有助于建立框架，其旧代际路线图不应用于当前选型。Synopsys 内存选型白皮书

3. LPDDR：功耗与系统形态优先，不等于“低性能 DDR”

LPDDR 面向严格功耗与紧凑系统设计，低功耗状态、供电、时钟及接口组织都很重要。判断优势要分活跃能耗、待机功耗、唤醒延迟、整板布线和封装。手机/边缘系统常关注每个任务能量与待机续航，服务器也可能采用 LPDDR 技术满足容量和能效需求。Micron 官方资料同时列出了移动和数据中心应用。Micron LPDDR5X

因此，“LPDDR 只用于手机”“LPDDR 一定比 DDR 延迟低”都不稳妥。需要指定代际、速率、拓扑、功耗状态和请求类型。进入更深睡眠可以节能，却可能增加恢复等待；持续流量高时，待机优势也不等于相同幅度的整机功耗下降。

教学假设：总数据宽 256 bit、8 GT/s，则原始带宽为 256 GB/s。扩大通道数可以提高带宽，也增加 PHY、封装引脚和布线资源。低功耗产品依然要计算真实业务需要多少吞吐量。

4. GDDR 与 HBM：提高单线速率，还是铺更多短连线

GDDR 常借助较高的每引脚数据率和板级多个器件供给图形/计算带宽；HBM 借助堆叠和宽接口、短距离封装互连供给高总带宽。两者都不是“无延迟内存”。HBM 仍包含 DRAM 阵列操作，GDDR 也可能满足某些 AI 系统的成本和容量目标。

教学对比，非当代产品规格：256 bit×20 Gb/s÷8=640 GB/s；1024 bit×6.4 Gb/s÷8=819.2 GB/s。第二种方案的单线速率低，总带宽却高。不能由这个算式直接得出成本或总功耗结论，还要加控制器/PHY、封装、堆叠数量、热和良率。

HBM 的高接口并行需要 GPU/加速器有足够通道、队列与请求；堆叠更高首先改变容量和热/制造约束，不会自动让带宽按 die 数同比增长。Hopper 的官方说明也把 HBM 控制器和 L2 明确列为独立架构资源。Hopper 架构资源

5. SRAM、NAND 与 HBF：容量层级为什么不能随便互换

典型 SRAM 单元用交叉耦合电路保持状态，在供电保持时无需 DRAM 式周期刷新，适合低延迟工作集；典型 DRAM 以电容存一位，密度更高但需恢复与刷新。不是所有 SRAM 都必须是 6T，也不是“片上”一定等于 SRAM：应分清常见实现和定义。

NAND 以页/块组织数据，读、编程、擦除的粒度和时间不对称，需要错误校正、坏块管理和寿命管理。SSD 还包含控制器、映射、缓存等系统机制，所以 NAND 芯片延迟不能直接等于 NVMe SSD 延迟。Microchip NAND 组织说明

HBF 的方向是让 NAND 提供更靠近加速器、更高并行的容量层；厂商将其面向 AI 推理。即使总读带宽很高，也不能推导细粒度随机首访问与 HBM 相同，更不能无条件把高频写入 KV 直接搬过去。可优先研究相对只读、可分块预测的权重；KV 的读写频率、生命周期和预取难度要单独评估。Sandisk HBF 技术简介（2025）

6. 从应用需求倒推产品规格：别只交“容量 + 带宽”

1. **容量：**常驻权重、KV/工作状态、临时 workspace、元数据、碎片和安全余量。
2. **吞吐：**每次任务实际经过接口的字节数×任务速率；区分读写和瞬时峰值。
3. **延迟：**平均与 p99、首次响应与稳态响应、最坏热点与混合业务。
4. **功耗/热：**活跃和空闲、I/O 与阵列、refresh，以及堆叠温度对运行的影响。
5. **成本/面积：**die、PHY、封装、测试、良率、容量扩展方式；没有工艺与设计数据时不编造 mm^2。
6. **可靠性：**ECC 覆盖范围、错误上报、修复、保持与寿命；on-die ECC 不能自动代替端到端保护。

教学能量预算：假设链路每传 1 bit 花 3 pJ，持续传 1 TB/s，则仅该口径动态功率为 $8\times10^{12}\times3\times10^{-12}=24$ W。这里没有计阵列、静态、刷新与控制器。不同论文的 pJ/bit 若边界不同，不能直接比较。

JEDEC 标准用于约定互操作所需的组织、命令、时序、电气和状态行为等；具体产品可能只支持其中部分等级，内部实现也不由一个接口名称完全决定。产品规格应同时给标准版本、器件料号/配置、目标工作负载和验证条件。不要把某标准最高数据率当所有器件的保证值。

面试问答与追问

1 客户要做 AI 推理，为什么不直接全部上 HBM?

先算能否满足容量、实际带宽和尾延迟，再比较功耗、封装成本和供应配置。HBM 很适合高带宽工作层，但容量成本与封装资源可能限制规模；若工作集可分层且访问可预测，可研究更大容量层。选型应给可验证的数据放置与迁移方案，不能只按峰值排名。

继续追问

- HBM 堆栈数减少会影响什么?
- 权重与 KV 是否适合相同存储层?

2 如何比较 DDR、LPDDR、GDDR、HBM 的功耗?

必须对齐系统边界和工作负载：同容量还是同带宽、活跃还是待机、是否包括控制器和封装、实际利用率多少。然后比较每任务能量和热约束。只抄不同资料里的 pJ/bit 不能得出整机能效结论。

继续追问

- 工作量低时哪类能耗占比会上升?
- 带宽翻倍但 pJ/bit 降低，总功率一定降低吗?

3 HBM 与 HBF 最根本的差别是什么?

HBM 属于 DRAM，HBF 基于 NAND。接口总读带宽相似不意味着访问粒度、首数据延迟、写入/擦除、寿命或可预测性相同。需要把 HBF 看成待评估的新层级，并明确哪些数据驻留 HBM、哪些可提前搬运。

继续追问

- 为什么小粒度随机读是关键测试?
- KV cache 更新会带来什么问题?

4 DDR 接口 64 bit、6400 MT/s 为什么是 51.2 GB/s?

每次传 64/8=8 B，每秒 6.4×10^9 次，相乘为 51.2×10^9 B/s。MT/s 已计传输次数，不再乘 DDR 的 2；这也只是原始数据接口带宽，应用有效带宽会更低。

继续追问

- 若包含 ECC 位怎么办?
- 若模块拆成两个 32 bit 子通道怎么办?

本章自测

1 下列哪一组都属于 DRAM 路线?

A. DDR、LPDDR、GDDR、HBM

B. SRAM、HBM、NVMe

C. NAND、GDDR、HBF

参考答案 A NVMe 是协议层，HBF 基于 NAND，SRAM 是不同介质。

2 HBM 堆叠 die 数翻倍，可以直接推出什么?

A. 带宽必然翻倍

B. 首数据延迟必然减半

C. 应重新计算容量与组织；带宽需看接口等约束

参考答案 C 容量、接口并行、速率和内部组织是相关但不同的维度。

3 1 TB/s 链路按 3 pJ/bit 估算的动态功率是多少?

A. 3 W

B. 24 W

C. 24 mW

参考答案 B 1 byte=8 bit，所以 $1\times10^{12}\times8\times3\times10^{-12}=24$ W；未包含其它能耗。

原始资料

1. NVIDIA Hopper Architecture In-Depth · 2022 · NVIDIA 官方架构说明
2. Choosing the Right DDR SDRAM Memory for Your Application · 2019 · Synopsys 官方白皮书（历史分类资料）
3. Micron LPDDR5X 官方产品与应用说明 · 2026 · Micron 持续更新页面；2026-10-30 访问
4. NAND Flash Organization · 2024 · Microchip 官方在线技术文档
5. Sandisk HBF Fact Sheet · 2025 · Sandisk 2025-07 技术简介

模块 11 · 核心必会

HBM：把DRAM阵列、通道与先进封装接起来

你已经理解GPU侧带宽需求，这一章补上另一半：HBM内部仍是DRAM阵列，带宽来自大量可调度的并行资源及宽接口，容量来自die密度与堆叠。不要把“层数、通道数、总线宽度、带宽”混为一个指标。

学习目标

- 画出GPU到HBM cell的完整路径
- 分别计算每stack容量与峰值带宽
- 解释pseudo-channel、TSV和base die的不同职责
- 用热、良率、路由及利用率评价增加stack的方案

01 | 一笔GPU读请求到底经过什么

把路径拆成两段：**GPU内部**是SM发起load、合并访存、查缓存、进入memory partition/控制器；**存储一侧**是PHY与封装互连、HBM底部接口及垂直互连、目标通道中的bank、子阵列、感应放大器，最后回到数据总线。不同GPU会有不同的缓存分片和地址映射，不能认为“一个SM固定对应一个HBM stack”。

HBM的DRAM核心die仍保留字线、位线、cell、行缓冲和刷新。堆叠并没有取消ACT、PRE及行冲突，只是给系统提供更多并行服务路径。GPU内几千个线程可以提供大量未完成请求；控制器必须把它们分散到不同通道和bank，才有机会把这些路径用满。

面试画图时标清边界：controller负责策略与时序约束，PHY负责物理收发；base die提供具体产品需要的接口、互连和辅助逻辑，不能一概写成“完整GPU内存控制器就在base die”。HBM4及定制base die扩大逻辑设计空间，也不意味着所有产品都内置通用CPU。Micron HBM4

02 | channel、pseudo-channel、bank、die不是同一种划分

用仓库类比：stack是一栋楼，die是楼层，channel是独立的服务线路，bank是能独立准备数据的工作区。**逻辑服务线路和物理楼层不必一一对应**，具体映射取决于器件组织。

概念	主要决定	常见误解
DRAM die数量	总容量、堆叠高度、热与制造复杂度	12-high就等于12通道
channel/pseudo-channel	可调度请求流、数据接口分区	切成两份就凭空带宽翻倍
bank/subarray	激活及行缓冲并行性	有很多bank就无时序约束
TSV	穿过硅的垂直电连接	TSV自身就是存储单元

以HBM3为明确例子：总数据宽度1024 bit，可分16个64-bit channel，进一步形成32个32-bit pseudo-channel。后者配合8拍burst提供32 B数据粒度。pseudo-channel增加调度分区，但仍有共享资源和命令时序；不要把它当作完全互不影响的芯片。Synopsys HBM3通道说明

03 | 两条公式分清容量和带宽

```
stack容量 = 每die容量 × DRAM die数量
峰值数据带宽 = 数据引脚数 × 每引脚数据率 ÷ 8
可用带宽 = 峰值带宽 × 综合利用率
```

这里的数据率已经是Gb/s或GT/s口径时，**不要再乘DDR的2**。如果给的是时钟频率，才要按具体接口机制换成数据率。容量的Gb与GB也差8倍。

教学假设：一个stack有12个24 Gb DRAM die，容量为12×24÷8=36 GB；1024个数据引脚、9.6 Gb/s每pin，峰值为1228.8 GB/s，即1.2288 TB/s。8个这样的stack得到288 GB和9.8304 TB/s峰值，但这是假设组合计算，不是某GPU产品宣称。若有效利用率70%，应用能用约6.88 TB/s；70%是本例假设，不能当通用常数。

作为真实产品锚点，Micron官方列出HBM3E的24GB 8-high及36GB 12-high产品，1024 I/O与超过1.2TB/s；HBM4官方页面则列2048 I/O及不同堆叠配置。产品值必须绑定厂商、型号/代际和日期，不能拿HBM3的channel数套HBM4。Micron HBM3E；Micron HBM4

04 | 为什么既叫3D堆叠又常用2.5D封装

描述的层级不同。HBM stack内部把多片DRAM die垂直堆叠，是3D结构；GPU和多个HBM stack通常并排放在高密度互连之上，是系统层面的2.5D集成。把HBM直接放在GPU顶部则涉及另一类垂直集成，热路径和供电条件会改变。

TSMC CoWoS家族提供不同互连实现：CoWoS-S使用硅interposer，CoWoS-R使用RDL interposer，CoWoS-L结合RDL和局部硅互连。因此“HBM一定连接一整块相同结构的硅中介层”说得过死。TSMC CoWoS结构

宽接口适合短而密的封装内走线。在同样带宽目标下，更多线可减轻单线速率压力，但会增加I/O面积、路由密度和封装约束。**HBM的价值是一组协同取舍**，不能简化成“单个DRAM cell读得快十倍”。比较GDDR或DDR时，需要把芯片/模组/整卡的总位宽和器件数拉到同一口径。

05｜高峰值为什么跑不满：从GPU一直问到cell

1. **GPU没有足够独立请求：**指针追踪、同步、低occupancy使内存级并行度不足，带宽还没用满就被延迟卡住。
2. **请求落点不均：**stride或地址映射让少数channel/bank拥堵，空闲通道帮不上忙。
3. **粒度浪费：**搬回32 B或更大系统事务，程序只使用其中少量字节；链路很忙而有效数据少。
4. **阵列准备不及时：**row miss、PRE/ACT、刷新以及激活功率约束导致数据总线没有可发送数据。
5. **方向与队列切换：**读写转换、QoS及返回网络阻塞影响利用率。

因此调优要同时测**应用有用字节、实际传输字节和时间**。前者除时间是业务有效带宽，后者除时间是设备服务带宽；两者的差距能暴露访存放大。若增加SM后性能不变，先定位是哪一级饱和，而不是直接要求更快HBM。

06｜加高、加宽、加stack，哪个代价更大

教学假设的良率直觉：若每个待组装die独立合格概率0.99，完全不筛选也无修复，12片全合格概率0.99^12≈88.6%，16片约85.1%。这只说明串联系统的风险会累积，*不是商业HBM良率估计*；实际有known-good-die筛选、冗余修复、TSV测试和工艺相关性。

加高可能提高容量而不同比例增加外部带宽，导致“每GB可用带宽”下降；加stack可提高容量和总接口带宽，却扩大封装面积、PHY占用及供电需求。提高pin速率可增加带宽，但眼图、时钟、信号完整性和功耗验证更紧。热设计还要考虑GPU热耦合、die间热阻和热点，并非看整包平均温度即可。

架构回答模板：先问工作负载缺容量、缺带宽还是缺低延迟，再提出stack数/密度/速率组合；分别检查地址并行性、性能收益、每token能耗、封装可制造性和成本。一个“容量够、峰值大”的方案，仍可能因热点或利用率差输给更均衡的配置。

面试问答与追问

1 HBM为什么带宽高，却不保证单次随机访问延迟同比降低？

带宽主要靠宽接口、短互连和多通道/bank并行；单个请求仍要经历排队、ACT/感应、列访问和返回。增多并行通道可以在同一时间服务更多请求，但不能消除同一个依赖链的等待。先看请求是否有足够并行度。

继续追问

- 如果负载是链表遍历，加一倍HBM通道有什么作用？
- 应分别测idle latency还是loaded latency？

2 12-high HBM3E改16-high，带宽会增加4/3吗？

不能仅由层数推断。层数首先影响容量和热/制造；带宽看外部位宽、pin速率及内部组织是否改变。若接口不变，峰值带宽可以基本不变，新增容量反而降低带宽/容量比。

继续追问

- 什么时候内部bank增加能提高可用带宽？
- 容量扩张后刷新开销如何评估？

3 32个pseudo-channel一定是32条完全独立总线吗？

是逻辑/接口上的细分服务资源，独立性要按相应代际规格的共享命令、时钟、电源及约束判断。它能增加请求调度灵活度，不等于每一条都无条件独立，也不改变总位宽守恒。

继续追问

- 如何设计地址映射让顺序访问分散而保留row locality？

4 让你为GPU把HBM带宽提高一倍，你先做什么？

先跑代表性trace确认应用是否被HBM数据服务限制，拆分有用字节和实际传输量，检查channel负载与row hit率；再比较加stack、提高pin速率、提高利用率或降低流量。用端到端收益验证，随后闭合功耗、封装、热和成本预算。

继续追问

- 若有50%运行时间完全不依赖内存，带宽翻倍的理想总加速比是多少？答案是1/(0.5+0.5/2)=1.33。

本章自测

1 1024-bit接口、每pin 8Gb/s，峰值带宽是多少？

A. 1024GB/s

B. 2048GB/s，因为DDR还要乘2

C. 8192GB/s

D. 只能由堆叠层数确定

参考答案 A 1024×8Gb/s÷8=1024GB/s。给定的是pin数据率，DDR效应已包含。

2 HBM stack内部3D堆叠、GPU与stack并排2.5D封装，这两句话是否矛盾?

A. 矛盾，2.5D中不能有3D组件

B. 不矛盾，描述的是不同集成层级

C. 只有HBM4可以同时成立

参考答案 B stack内部die垂直连接，与整个系统把GPU和stack并排互连，是两个层级。

3 8个channel中只有1个忙，最先应调查什么?

A. 把存储单元容量加倍

B. 地址映射、stride和请求分布

C. 只测芯片最高温度

参考答案 B 先确认瓶颈是否来自流量分布。热也重要，但不能解释所有通道失衡问题。

原始资料

1. What Designers Need to Know About HBM3 · 2022 · Synopsys technical bulletin
2. Micron HBM3E product information · 2026 · Micron（网页检索于2026-10-30）
3. Micron HBM4 product information · 2026 · Micron（网页检索于2026-10-30）
4. CoWoS advanced packaging · 2026 · TSMC 3DFabric（网页检索于2026-10-30）

模块 12 · 核心必会

NAND基础：理解HBF前必须补齐的另一种阵列

HBF的F是Flash。它不是把HBM的D改成F那么简单：cell读写物理机制、最小操作粒度、维护方法和软件契约都变了。先把普通NAND的因果链吃透，再判断高带宽Flash改进了什么、没改掉什么。

学习目标

- 画出string/page/block/plane/die层级
- 解释读页、编程页、擦除块的不同机制
- 把FTL、ECC、GC、写放大与延迟联系起来
- 避免把3D NAND层数和封装die堆叠混为一谈

01 | DRAM存电荷，NAND也存电荷，为什么行为完全不同

DRAM用电容上的电荷表示状态，访问晶体管接通后，与位线发生电荷共享，再由感应放大器恢复。NAND则将电荷保存在受绝缘结构约束的储电区域，**用阈值电压变化**来区分状态；浮栅和charge-trap是不同实现，不应把所有NAND都叫浮栅。读出时判断给定栅压下是否导通。KIOXIA NAND原理

想象DRAM是需要不断补水的小水杯，NAND是用能垒困住电荷的状态标记。这个类比只解释保持机制：NAND写/擦需要改变被困电荷，过程慢且会造成老化；读又要辨认阈值分布。因此**掉电保留、读写对称、随机小粒度访问**是三件不同的事，不能由其中一件推导另外两件。

SLC/MLC/TLC/QLC分别常指每cell存1/2/3/4 bit，对应2/4/8/16个可区分状态。状态更多意味着电压窗口更密，噪声、漂移和老化的容忍空间更小；是否采用更多bit/cell是容量、延迟、可靠性与成本共同决策，不能只看密度。

02 | 先认识string，再认识page到die

string是一串串联cell。读某个cell时，目标字线施加判读电压，其他未选字线用pass电压使路径可导通，通过位线感测整串电流。它并不是一个字节一个引脚的随机访问结构。多个string并排组织，字线横跨这些string。

层级	用途	不要照搬DRAM概念
Page	基本读/编程组织，附带纠错及管理开销	不等于CPU 4KiB虚拟页；多bit/cell时同一wordline涉及多个逻辑页
Block	擦除单位，包含很多page	不等于cache block
Plane	含阵列与局部数据路径，支持一定多plane并行	并行操作仍有地址和资源限制
Die/LUN	更高层执行/互连单元	LUN定义需看器件协议，不能永远与die画等号
Package/stack	组合多个die并接到控制器	堆更多die不自动使外部接口带宽等比例增加

数据路径：NAND array → sense/page buffer → cache/data register → 外部I/O。先感测一页与从buffer传走数据是不同阶段；缓存命令和多die交错可重叠阶段，正如流水线提高吞吐却不消除首个请求延迟。

03 | 读、编程、擦除：三种不同的账

读通常先把目标页从阵列感测进page buffer，再通过I/O传出需要的数据。即便接口允许只传其中一段，**阵列内部也可能已经付过整页感测成本**。要区分阵列读放大、总线传输放大以及应用请求粒度。

编程通过受控脉冲及校验，把cell推向目标阈值区间。通用NAND不能像DRAM一样任意反复原地覆盖；常见做法是写入新位置，旧页标无效。擦除则以block为单位重置状态。具体partial-page program次数、页写入顺序、多plane操作限制都要读器件datasheet，不能用一条经验代替协议。KIOXIA更新与回收流程

教学假设：页大小16KiB、阵列读20µs、输出接口2GB/s。单页输出约8.19µs；串行完成需约28.2µs，对应约0.58GB/s。若双buffer允许感测与传输重叠，长期上限受较慢的20µs阶段限制，约0.82GB/s，而不是2GB/s。再增加可并行plane才有机会继续提高。所有数值只用于练习，不能当HBF或现代NAND规格。

04 | FTL不是多余固件：它在修补物理介质与软件语义的差距

软件要“把逻辑地址A更新成新值”，介质却不擅长原地更新。**Flash Translation Layer**维护逻辑到物理映射：把新数据写到可用page，原子地更新映射，旧page成为垃圾；再由GC回收失效空间。坏块管理和wear leveling解决不可用位置与磨损不均。ECC校正一定范围的错误，read retry可能改变判读条件以恢复数据。KIOXIA L2P与GC；Micron ECC/错误处理

这些功能位于哪一层取决于产品：SSD控制器、managed NAND固件、host或定制HBF base die都可能承担一部分。**HBF不是必须原样搬入完整SSD FTL，也不是可以取消所有可靠性管理。**只读权重工作集可以简化随机更新、日志和GC路径，但仍要知道坏块在哪里、数据是否正确、何时迁移。

地址映射也有容量成本。教学假设1TiB介质用4KiB粒度映射、每表项4B，则表有2^28项，占1GiB；若按2MiB连续burst映射，表项约2^19个，只占2MiB。后者需要更强的连续布局契约，而且碎片、坏块和更新会使简单模型复杂化。

05 | 写放大和读放大：两个容易漏算的系数

```
写放大 WAF = NAND实际写入量 / 主机逻辑写入量
读放大 RAF = 实际读取或搬运量 / 应用有用量
有效带宽 ≈ 物理服务带宽 × 利用率 / RAF
```

教学GC例子：一个block有256页，回收时128页仍有效。先搬走这128页，再擦除，净获得128页可用空间。若随后承接128页主机新写，实际写256页，WAF=2（只计此循环，忽略元数据）。如果每个victim block有90%有效数据，回收会更不经济；选择更合适的victim、冷热隔离和预留空闲空间可以改善。

读放大也不能盲目追求最小：若读取一个更大连续块能显著减少命令数量、改善并行度和接口效率，多读一点可能反而更快。但若随机64B请求每次迫使取4KiB且无复用，RAF=64；1TB/s物理带宽最多提供15.625GB/s有用字节，还没扣时延与排队。

面试里解释性能异常时，应同时问**平均值与尾延迟**。GC、read retry、迁移等后台工作可能偶发阻塞前台，即使平均GB/s很好，P99也可能难以满足服务要求。

06 | “几百层NAND”与“16-high HBF”为什么不能相乘成通道数

3D NAND的很多层通常描述**一个die内部**沿垂直方向构造的存储结构；16-high描述**封装中**堆了16个存储die。前者主要提高阵列密度，后者增加封装容量，两者均不等于独立plane数，也不等于对外channel数。

高密度NAND要兼顾高深宽比结构的制造、字线/位线电阻电容、阈值分布和多层均匀性。CBA等阵列与CMOS分开制造再键合的方法，给外围电路的工艺及布局更多自由；这仍不是“每层都可独立以TB/s读出”的证据。

最后一个关键区别是**refresh的含义**：DRAM周期性恢复cell电荷；NAND虽无需这种持续刷新，仍有retention drift、read disturb及必要的数据迁移/重写。静态权重读得极频繁时，host没有写操作也可能产生介质维护写。Micron read disturb问答 这正是后面FLINT要专门处理的系统问题。

面试问答与追问

1 HBF如果是只读权重，为什么还要管理写耐久和刷新?

只读通常是应用语义。NAND的高频读会引起read disturb，保持状态也随时间和温度变化；纠错后迁移重写、模型重载都产生介质写。可以简化通用随机写FTL，但不能省略ECC、坏块及维护策略。

继续追问

- DRAM refresh和NAND refresh如何区分?
- 怎样把维护写从token关键路径移开?

2 NAND读4KiB请求，就一定只在阵列读取4KiB吗?

不一定。应用请求、接口传输和阵列page read可以是不同粒度。可能整页感测后只传部分数据，也可能多个请求复用page buffer。需分别测量阵列读放大与链路读放大。

继续追问

- 读放大高但带宽利用率也提高，如何判断总体是否收益?

3 为什么page不能一直原地覆盖?

编程只能在器件允许的方向和顺序调整阈值状态，恢复通常要block erase；任意覆盖会违反介质条件并影响可靠性。所以FTL常用异地写加映射更新，后续GC回收旧页。

继续追问

- 失电发生在数据写完但映射未提交时怎么办?

4 增加plane数就能等比例提高NAND带宽吗?

需要足够独立请求，阵列感测、page buffer、命令地址约束、外部I/O和供电都要配套。只增加plane却共享拥堵输出通道，收益会很快封顶。

继续追问

- 请画出感测与输出双buffer流水线。

本章自测

1 NAND最常见的擦除粒度是?

A. 一个CPU cache line

B. 一个page

C. 一个block

参考答案 C NAND通常page读/编程、block擦除，具体限制看器件。

2 主机写100GB，GC等使NAND共写250GB，WAF是多少?

A. 0.4

B. 1.5

C. 2.5

参考答案 C 250/100=2.5。WAF是总物理写入/逻辑写入，不是只算额外部分。

3 一颗232层NAND die堆叠16片，能推出有3712个独立channel吗?

A. 能

B. 不能

参考答案 B die内cell堆叠层数、die封装堆叠、plane及channel是不同维度。

原始资料

1. What is NAND Flash Memory? · 2026 · KIOXIA技术教育（检索于2026-10-30）
2. Managing Garbage Collection Operations Efficiently to Improve e-MMC and UFS Device Performance · 2024 · KIOXIA technical brief
3. Micron FAQs — NAND ECC and read disturb · 2026 · Micron technical support（检索于2026-10-30）
4. Sandisk HBF Fact Sheet · 2025 · Sandisk 2025-07 技术简介

模块 13 · 深入研读

HBF：高带宽Flash，到2026年究竟已知什么

截至2026-10-30，HBF已经有厂商公布的技术规范和系统研究，但公开规格、路线图、仿真架构与可购买产品必须分层讨论。本章的核心是：利用海量并行NAND读取得到高吞吐，同时承认它仍有较高访问延迟、粗粒度和维护成本。

学习目标

- 准确区分HBF的已公布规范、历史产品目标和未验证细节
- 用带宽时延积计算需要的并行度
- 解释HBF与HBM互补的条件
- 在面试中识别营销数字、模拟参数与产品指标

01 | 一张表把已知、目标、未知分开

证据层级	可说的结论	不能顺带推断
2026-08官方标准公告	Sandisk和SK hynix宣布经OCP发布首版HBF技术规范	所有合作方已量产装机
SK hynix公布的规格范围	8-high/16-high、容量最高512GB、约0.4–3.0TB/s分三档、UCIe连接	最高容量与最高速度必属同一现售SKU，或随机访问均达该速度
2025 Sandisk fact sheet	描述256Gb/die、16-die 512GB和1.6TB/s读带宽目标	这是2026首版规范唯一配置或独立实测
学术论文配置	可以研究给定页、时延、通道、维护策略下的收益	论文的2μs、SLC或FTL一定是厂商最终设计

以上规格分别来自Sandisk 2026公告、SK hynix FMS 2026、Sandisk 2025 fact sheet。标准公告回答“开始如何协作”，产品datasheet才回答“某个具体器件保证什么”。

02 | HBF究竟在改什么：让Flash更靠近计算，更并行

普通SSD路径包含主机I/O软件栈、存储协议、控制器、NAND接口和阵列。HBF的目标是让高容量Flash进入更接近加速器的层级，采用高带宽互连和大量阵列并行，减少传统存储访问路径的限制。**“靠近”主要改善搬运路径，并不把Flash cell改成DRAM cell。**

概念上可以拆成：NAND核心阵列提供容量，多个die/plane及局部buffer并行准备数据，base die或对应控制逻辑负责映射、排队、返回及维护，近计算接口把数据送给GPU/加速器。厂商公布了UCIe方向；但具体地址空间语义、最小读粒度、缓存一致性、错误上报和重试机制，仍需具体规范与实现证据。已公布接口范围

不要说“HBM接口完全兼容，插上就用”。物理互连能连接异构chiplet，并不自动赋予Flash低延迟、细粒度随机写或透明一致性。把HBF做权重对象层、显式DMA层或GPU可寻址层，会对软件、队列和容错提出不同要求。

03 | 高带宽为什么能与高延迟同时成立

超市很多收银台可以每秒完成很多订单，但某个订单仍需要完整结算时间。HBF利用并行资源提高长期完成率，无法保证一个存在数据依赖的load立即返回。架构师应首先写下**Little定律/带宽时延积**：

```
所需在途字节 Q ≈ 目标带宽 B × 单次服务延迟 L
所需独立事务数 N ≈ B × L / 每事务有用数据 g
```

教学假设：B=1TB/s，L=2μs，则在途数据至少约2MB。若每个有效事务4KiB，需要约489个同时有用的事务；如果只有64B，则约31,250个。真实装置还受bank/plane冲突、队列深度和读放大影响，公式是必要的量级检查，不是充分条件。2μs在FLINT中属于模拟配置，不能据此宣称某商业HBF延迟。FLINT Table 2

若只有一个串行依赖的64B请求，无论标称1TB/s多漂亮，在上述2μs假设下它最多提供32MB/s有用数据。并行度不足时，多通道闲着等；对规则权重流，预取和批量请求有机会填满流水线；对不可预测指针访问则难得多。

04｜页粒度、buffer与预取是架构的主角

HBF希望按较大块准备数据，GPU往往按cache line或tile消费。中间要回答四件事：**哪些小请求能合并？把完整页存在哪里？何时逐出？如果只用少量字节，谁承担放大？**

教学假设4KiB页中只有256B会用到，RAF=16；物理带宽1TB/s、服务利用率80%时，有用带宽最多50GB/s。若把相邻权重重排并使一页中75%字节都有用，RAF降至1/0.75，同样硬件可达600GB/s有用带宽。布局收益不能只算峰值，需要扣除重排成本、额外索引和地址转换。

预取提前量应覆盖“排队＋介质读＋传输＋数据就位”的关键路径。过早预取占用HBM/片上buffer、引起污染；过晚预取暴露停顿。MoE专家由token路由决定，下一步要读谁有时只有运行时才知道，因此“提前一层把全部专家拉来”可能浪费容量和带宽。

同样的原理能在普通Flash研究中看到：*LLM in a flash*用缓存复用与连续布局减少搬运成本；它是可迁移的方法论，不是HBF硬件实证。ACL论文第3节

05｜非易失不等于零维护，读路径也要有可靠性预算

官方资料说NAND无需DRAM式持续刷新，这是介质特征；但保持误差和read disturb仍可能要求纠错、重写和迁移。于是“只读权重”仍可能在后台产生program/erase，影响能耗、寿命和尾延迟。

一个合理设计需要计数器跟踪读压力，ECC发现错误趋势，冗余空间接收迁移数据，原子映射更新保证前台读始终取到有效副本。迁移还可能争用阵列、内部互连和base die资源。你不能只把它放进平均功耗，再假设性能完全不受影响。

FLINT提出额外维护plane及轮换机制，把维护与前台读拆开，并依据权重只读特性简化映射。这个例子提示**先围绕工作负载削减控制面复杂度，再为剩余可靠性责任安排资源。**它是预印本的架构提案，不代表最终HBF标准。FLINT第4节 若模型更新很频繁、权重在线训练或KV重写量很大，简化只读路径的前提就不成立。

06 | 怎样回答“性能距HBM只有2.2%，那还要HBM干嘛”

第一步翻脚注。Sandisk 2025 fact sheet的2.2%比较限定于**Llama 3.1 405B、8-bit预训练权重读取、逐kernel的xPU性能模型**，HBM容量在比较中被设成无限；文档同时承认HBF有更高延迟和更大page。原始fact sheet脚注

这能支持“特定可并行权重工作集在模型里能隐藏HBF代价”的论点，不能支持“小随机load、频繁写KV、训练和任何batch都等价HBM”。当算子有足够计算遮住传输，两个存储层可能表现接近；当依赖链直接等数据，延迟差会暴露。

另一个证据陷阱：本教程所读HBFSim v1在真实GPU上执行推理并注入模型时序，但该版本6.1明确其flash/thermal参考设备是普通NVMe SSD，并非HBF；GPU执行是真的，HBF行为仍是模型。某项校准误差小，不表示对未来HBF硅有同样误差。HBFSim方法与实验边界

07 | 面试该怎么讲路线图与尚未公开的信息

2025 Sandisk公告给出“2026下半年HBF memory送样、2027年初HBF推理设备送样”的目标。这里应使用**计划/目标**，不能因为日期临近就自动改写成已经实现。当前本包核验的是2026首版规范公告及公开论文，没有将目标当作量产证据。Sandisk路线图

对于具体产品的单请求延迟分布、最小传输粒度、读写干扰、耐久、WAF、热降频曲线、ECC重试、上电恢复及操作系统接口，需要进一步查实现级资料。回答“不知道”时同时提出验证方法：要求随机/顺序、不同并发度和读写混合的latency-bandwidth曲线，以及稳态热和老化后的P99。

> 可直接复述：HBF更适合作为近计算的高容量读密集层。我会用权重与KV的读写/复用特征决定放置，保留HBM承接低延迟和写密集热点，再通过并行度、读放大、热与维护模型检验它是否真正降低每token成本。

面试问答与追问

1 HBF能替代HBM吗?

不能笼统替代。二者容量、延迟、访问粒度、写行为和维护机制不同。大块、可预测、读密集权重可放HBF；低延迟热点、活跃KV及临时激活更倾向HBM。是否共享封装位置也重要，因为加入HBF可能挤掉HBM容量和带宽。

继续追问

- 如果模型本来完全装得下HBM，HBF的收益从哪里来?
- 你会优先报告tokens/s还是P99 TPOT?

2 1TB/s、2μs、4KiB事务，大约需要多少并行事务?

B×L=2MB在途数据，除4096约489个。此处2μs是教学/模型假设；该数只验证并行度量级，冲突和读放大可能要求更多资源。

继续追问

- 如果有效事务只有64B呢?
- 队列数足够但全落同一plane怎么办?

3 为什么HBF不能直接继承HBM的软件语义?

互连兼容只解决传输路径的一部分。Flash有页读、慢编程、块擦除、维护与错误重试；是否暴露为统一地址空间、由谁预取、如何一致性和故障恢复，需要明确系统契约。

继续追问

- 显式DMA层和透明cache层各有什么取舍?

4 如何检查HBF性能宣传是否能用于自己的方案?

核对年份和证据类型，确认单位是每stack还是系统、读还是写、峰值还是有效；再看模型、精度、batch、KV、容量约束及是否计入维护和互连。用目标trace重算，不能只套一个比HBM慢百分之几的结果。

继续追问

- 请说明2.2%比较的限定。
- 真实GPU运行为何仍不等于HBF硅实测?

本章自测

1 2026官方公布0.4–3.0TB/s三档HBF规格，能推出随机64B串行读也达3TB/s吗?

A. 能，高带宽就是低延迟

B. 不能，还受并发度、粒度、时延和实现限制

参考答案 B 总吞吐与单请求服务时延不是同一指标；规格范围也不是单SKU所有场景的保证。

2 2TB/s目标、5μs服务延迟，需要多少在途数据?

A. 10KB

B. 10MB

C. 10GB

参考答案 B 2×10^12B/s×5×10^-6s=10^7B，即10MB。

3 关于HBFSim，下列哪句准确?

A. 已经测量商业HBF完整芯片

B. 真实GPU执行结合HBF时序模型，普通SSD用于部分校准

C. 只要校准点准确就保证全系统预测准确

参考答案 B 真实GPU、模拟HBF与普通SSD校准应分别说明；覆盖范围和外推条件限制结论。

原始资料

1. Sandisk and SK hynix Advance Global Standardization of HBF with First OCP Technical Specification · 2026 · Sandisk 2026-08-03官方公告
2. SK hynix Unveils First HBF Standard Specifications at FMS 2026 · 2026 · SK hynix 2026-08-04官方公告
3. Sandisk HBF Fact Sheet · 2025 · Sandisk 2025-07 技术简介
4. Memory-Centric AI: Sandisk’s High Bandwidth Flash Will Redefine AI Infrastructure · 2025 · Sandisk roadmap blog
5. FLINT: Efficiently Leveraging High Bandwidth Flash for Capacity-Scalable LLM Inference Acceleration · 2026 · arXiv:2608.25062 v1，2026-08-25预印本
6. LLM in a flash: Efficient Large Language Model Inference with Limited Memory · 2024 · ACL 2024 Long Papers
7. HBFSim: Fast and Faithful Simulation of High-Bandwidth Flash Under Real GPU Execution · 2026 · arXiv:2609.09800 v1，2026-09-09预印本

模块 14 · 深入研读

GPU—HBM—HBF—SSD：权重、KV与MoE如何分层

这章把前面的器件知识变成架构面试答案。不是先问“能不能把显存换成HBF”，而是给每类数据写清大小、生命周期、读写比、复用次数和关键路径，再设计放置、预取及准入策略。所有手算例子的数值均为教学假设。

学习目标

- 手算权重、GQA KV容量与带宽下界
- 解释prefill/decode/batch怎样改变存储瓶颈
- 为MoE及KV提出能验证的分层方案
- 识别容量收益、尾延迟、耐久及封装机会成本

01｜先给数据分类，再谈哪层存什么

数据	主要特征	候选位置/策略
稠密模型权重	部署时写，推理中频繁读；通常可预测	装得下且延迟敏感时HBM；容量受限可考虑HBF顺序流与预取
MoE专家权重	总量大，单token仅用部分，热度不均	热专家HBM、冷专家HBF；调度与路由共同决定命中
活跃KV	不断追加，注意力反复读；增长可预测	优先HBM，并为即将生成的token留空间
可复用历史prefix KV	写一次后可能多次命中，价值取决于复用	选择性进入HBF/host层，按收益而非单纯LRU决定
临时activation/workspace	短命、频繁写	SRAM/HBM更合适；避免无意义地写入Flash
长期模型仓库/检查点	容量大、访问较稀疏	SSD作后备，按部署需求载入近端层

这是**设计起点而非固定规定。**HBF研究中既有偏权重容量层的FLINT，也有偏KV服务的工作，它们回答的问题不同。FLINT；HBF serving研究

02｜权重与decode：先用字节把“能跑多快”算明白

教学假设：70B参数，INT4权重，则原始权重约70×10^9×0.5=35GB；实际还要加量化scale/zero point、对齐、部分高精度层。若batch=1，每个decode step都完整读35GB，没有跨step缓存，且有效权重带宽1TB/s，则仅权重流下界35ms/token，对应不超过约28.6token/s。计算、KV、同步和kernel开销只会让实际更慢。

若batch=8能在一次权重加载中处理8个序列，则同样35ms的理想步时间可产8个token，总吞吐上界约228.6token/s；**单请求token间隔仍约35ms**，不能把228.6倒数当成每个用户的TPOT。batch增大还会增加KV读写和排队，因此不能无限摊薄。

```
共享HBM接口：T_HBM ≥ (D权重_HBM + D_KV_HBM + D其他_HBM) / B_HBM
对每个共享资源先合并字节/运算需求，再求各自服务时间
理想重叠下界：Tstep ≥ max(各资源服务时间，必须串行的依赖链时间)
```

重叠必须按物理资源分析：若权重和KV共用HBM接口，不能分别用满该接口带宽，再把两段时间取max；HBF填入HBM的流量也会占用目的端。读写方向切换、有效带宽和多个通道的分布应另行计入。只有资源确实独立且依赖允许，计算、HBF传输与HBM服务才可能交叠。固定一组服务操作、没有额外开销时，充分重叠的max下界与完全串行的服务时间之和构成理想比较区间；实际队列、同步、调度和新增搬运可使耗时超过这个朴素求和，不能把它当实际运行上界。

prefill更接近大矩阵乘，权重复用与算术强度通常较高；decode小batch常更像GEMV，权重搬运更显眼。但长上下文、大batch、稀疏注意力及MoE会改变主瓶颈，所以先测而不背“prefill永远compute-bound”。

03 | KV容量别用模型参数量猜，要从张量维度算

```
常规GQA/MHA KV字节数 =
2 × 层数 L × batch B × 上下文长度 S
  × KV头数 Hkv × 每头维度 D × 每元素字节 e
前面的2分别代表K与V
```

教学假设：L=80、Hkv=8、D=128、FP16即e=2，则每序列每token KV=327,680B=320KiB。S=32,768时约10GiB/序列；B=16时约160GiB。这个公式是常规KV布局；MLA压缩、量化、sliding window、稀疏保留、tensor parallel分片及元数据都会改变实际占用，不能直接照套所有模型。

decode不仅追加新KV，还会读取已有历史。新增写很小不代表总流量小；反过来，某层缓存写入了大量新KV，却几乎没有再命中，则Flash层的写入也不值。设计必须分别统计**新增字节、读回字节、读取放大、重复迁移次数和被丢弃的字节。**

若用全满HBM作为准入条件，新请求进入后继续生成会让KV溢出，被迫迁出再读回。可按最大新增token或置信区间留headroom，牺牲一点即时batch，换取少迁移、较稳定TPOT和较少Flash写。buffered cache-aware scheduling

04 | MoE不是只看“激活参数量”，还要看专家落在哪里

总参数决定模型要占多少空间，激活参数决定某token至少需要读/算多少；batch中的不同token可能选不同专家，所以一个step触及的专家集合可能随batch扩大。热点偏斜有利于缓存，但专家不均衡又可能使少数执行单元或存储通道拥堵。

教学假设：每步需读取10GB专家权重，70%字节命中HBM，剩余3GB去HBF；HBF物理带宽1TB/s、利用率80%、读放大1.5，有效带宽约533GB/s，HBF分支至少5.63ms。若只有一个独立请求链无法维持80%利用率，这个下界不再可达。若热专家集合漂移，预装缓存也要付更新流量。

可提出三种方案并比较：按历史热度常驻，按即将路由结果动态预取，或在受精度/模型约束允许时采用路由预测。评价命中率必须用**字节和关键路径**加权：命中很多很小专家、漏掉少数巨大专家，计数命中率会骗人。研究中的“离线已知专家查找/理想负载均衡”只是上界或受限实验条件。论文配置及局限

05 | 把写耐久变成可算的预算

```
简化理想寿命（天）≈
物理可轮换容量 × 可承受P/E次数
÷（每日NAND总物理写入量）
```

若WAF按前文定义为总物理写入/主机写入，分母可写为每日主机写入量×WAF，此时不能再加一次已计入WAF的维护写。只有明确将WAF限定为主机诱发写放大、排除独立维护时，才另加维护写入量。主机写入接近零的只读权重层应直接统计维护造成的物理写入，避免使用没有稳定分母的WAF。

纯教学假设，绝非HBF寿命规格：沿用320KiB新增KV/token，系统总生成速率1000token/s，若全部KV都落Flash，每天逻辑写约28.31TB。假设可轮换容量1TB、10,000次P/E、WAF=2且忽略维护，理想均匀磨损寿命约176.6天。若仅10%有复用价值的KV写入，其他条件不变则约1766天。这个数量级练习说明**准入策略可能比单纯提高读带宽更重要。**

真实寿命还受器件模式、温度、retention要求、误码裕量、坏块退役、热点与冗余容量影响。不要拿某论文假设的100k P/E当厂商保证；也不能用“读密集”标签忽略迁移写。

对于prefix缓存，一个简单价值模型是：预期未来命中次数×每次避免的重算/读回成本，减去写入、驻留机会成本及维护成本。阈值可随流量调整。频繁写入却从不读回的冷尾部，应该被拒绝入层，而不只是被更快写入。

06 | HBF不一定是白送容量：封装位置也有机会成本

比较至少要区分两种组织：**增加额外HBF资源**，与**在固定封装位置中用HBF替代部分HBM。**前者多了容量也多了面积/功耗；后者的HBM容量和带宽可能减少，热数据的性能损失可能大于远端读加速。

2026的KV-centric研究在生产trace衍生的系统模拟中发现，沿用SSD式KV卸载运行时直接改用特定HBF组织，可能使整体服务变差。论文自己明确：实验同时改变了近端容量/带宽，结论针对完整配置，**不证明单独HBF介质比SSD慢。**Full-Stack Characterization第9.3节

相反，另一个研究通过放置及调度改善高吞吐服务。但同一结果表中，某配置总完成时间下降时，平均TPOT仍可能上升；更多并发能够提升总吞吐却拖慢单用户。Characterizing HBF第5节 面试不能只说“加速百分之几”，要先说目标是吞吐、TTFT、P99 TPOT，还是满足SLO后的每token成本。

07 | 把方案写成能跑实验的五步

1. **固定问题：**模型/精度、上下文与输出长度分布、到达率、延迟SLO；记录热身和稳态时长。
2. **基线公平：**HBM-only、HBM+SSD、HBM+HBF；说明是否同GPU数、同容量、同成本或同封装预算，不能在不同图中悄悄切换。
3. **采集分项：**每层读写字节、命中率、RAF/WAF、page/plane并行度、排队、KV重算、跨卡通信和维护停顿。
4. **扫参数：**HBF延迟、有效带宽、页大小、HBM容量、专家热度、write budget、温度及P/E假设；用敏感性区间替代单个漂亮点。
5. **报告业务结果：**goodput（满足SLO的有效吞吐）、TTFT/TPOT分位数、每token能耗和成本，并保留失败反例。

最有力的反例包括：模型完全适配HBM且几乎不offload；小batch串行依赖；随机小粒度；KV冷尾部大量写且少复用；HBF挤占过多HBM；长时间热降频。能主动给出失败区间，比只会推销新器件更像架构工程师。

面试问答与追问

1 设计一个HBM+HBF推理系统，你第一句话会说什么？

先确定服务目标和数据特征：是容量装不下、带宽不够、还是延迟不达标；分别量化权重、活跃KV和可复用prefix。然后判断HBF是在新增位置还是替代HBM，用明确工作负载和SLO比较。

继续追问

- 权重和KV分别怎么放？
- 哪些数据绝不应该无条件写HBF？

2 为什么增加batch能提高总token/s，却可能降低体验？

批处理可复用权重并提高计算利用率，但单step工作和KV流量也增加，还可能增加排队。总token/s是所有序列合计，用户体验看其TTFT及连续token间隔。应按SLO goodput选batch。

继续追问

- 8个序列每35ms各出1token，总吞吐和单请求TPOT各是多少？

3 MoE只激活少量专家，为什么HBF仍可能卡住？

选中专家可能不在HBM、路由到达较晚、访问页不连续，且batch内专家并集可能很大；page读取放大和plane热点使峰值带宽难以利用。需联合缓存、布局、预取及调度。

继续追问

- 用专家数量命中率还是字节命中率？
- 预测错误时如何避免阻塞正确专家？

4 有两篇HBF论文一篇说更快，一篇说更慢，如何回答？

先对齐系统边界、基线、容量预算、HBM是否被替换、数据放置和SLO。权重只读层与KV写密集层不同；高吞吐目标与低TPOT目标也不同。用共同假设重建模型，找结论翻转点，而不是投票选论文。

继续追问

- 给出一个改变单一变量就让结论反转的例子。

本章自测

1 70B INT4原始权重约35GB，1TB/s有效带宽完整读一遍至少多久？

A. 0.035ms

B. 3.5ms

C. 35ms

参考答案 C 35×10^9/10^12=0.035s。量化元数据等会增加真实流量。

2 batch=8，每35ms各输出1token，哪一项正确?

A. 总吞吐约228.6token/s，单请求TPOT约35ms

B. 单请求TPOT约4.375ms

C. 每个请求都达到228.6token/s

参考答案 A 总吞吐摊到8条序列；每条序列仍要等下一step。

3 HBF层频繁接收KV，却很少读回，优先改什么?

A. 先提高写入准入门槛和复用预测

B. 无条件把更多KV写进去

C. 只比较读峰值

参考答案 A 写入未被未来复用偿还时，会消耗带宽、能耗和寿命；先减少无价值搬运。

原始资料

1. FLINT: Efficiently Leveraging High Bandwidth Flash for Capacity-Scalable LLM Inference Acceleration · 2026 · arXiv:2608.25062 v1，2026-08-25预印本
2. A Full-Stack Characterization of High-Bandwidth Flash for KV-Centric LLM Serving · 2026 · arXiv:2608.11668 v1，2026-08-12预印本
3. Characterizing High Bandwidth Flash for LLM Serving · 2026 · arXiv:2609.39131 v1，2026-09-30预印本
4. HBFSim: Fast and Faithful Simulation of High-Bandwidth Flash Under Real GPU Execution · 2026 · arXiv:2609.09800 v1，2026-09-09预印本

模块 15 · 深入研读

PIM系统地图：计算放在哪里，究竟省了哪段搬运

“存算融合”是大伞，不能把HBM旁的数字ALU、DRAM阵列内逻辑和模拟交叉阵列当成同一技术。本章先建立系统位置、数据路径和软件代价的框架；数字、模拟、混合及材料层面的细节继续进入对应CIM专题。

学习目标

- 按计算位置而不是宣传名称区分PIM类型
- 解释Ambit、SIMDRAM、HBM-PIM和UPMEM各自的数据路径
- 判断低算术强度是否足以让PIM获益
- 检查一致性、布局、跨bank通信和端到端加速

01 | 先问计算在哪里，名称留到后面

位置/机制	算什么	减少什么搬运	主要代价
Near-memory逻辑层或附近处理器	数字指令/专用算子	远距离CPU/GPU与内存传输	逻辑面积、热、接口与调度
Bank附近数字ALU	向量加乘、MAC等	row buffer数据无需全部出芯片	每bank算力有限、跨bank归约
DRAM阵列内bulk logic	整行位逻辑及组合运算	大量数据留在子阵列	特殊行/操作、布局、可靠性
模拟CIM阵列	电流/电荷域乘加	权重读到独立MAC的反复传输	转换、噪声、精度、非理想和校准

学界与厂商对PIM/PNM/CIM的命名并不完全统一。面试时用“**位置 + 数据流 + 运算域**”消歧：例如“bank附近数字SIMD单元”，比只说“数字存算”清楚。HBM是存储接口及堆叠体系，本身不等于HBM-PIM。

02 | PIM不是凭空产生算力，而是搬运账重新分配

以 `y=A×x` 为例，普通GEMV要把矩阵A不断送到计算单元；若A常驻各bank并在当地乘加，只需广播较小的x并汇总y，就可能减少外部流量。但本地仍需从cell经感应放大器和局部总线送到ALU，还可能在bank间归约。

```
T_PIM ≈ T放置/转置 + T启动 + T本地算子
        + T跨bank/跨die通信 + T结果回传 + T一致性
收益条件：避免的搬运时间 > 新增开销
```

教学假设：总任务100ms，其中80ms是适合下沉的部分，PIM将其缩为10ms，额外布局和同步花8ms。总时间20+10+8=38ms，加速2.63倍，而不是8倍。若每次只处理很小数据，启动成本就可能吃掉全部收益。

低算术强度只是候选条件。若操作依赖大量除法、分支或全局通信，弱小PIM核可能先遇到计算或互连瓶颈。迁到存储旁边后，原来在GPU上memory-bound的程序，完全可能在PIM上compute-bound。

03 | Ambit：借DRAM电荷共享做精确的批量位逻辑

Ambit利用同一位线上三行同时参与电荷共享，感应结果对应多数函数 `MAJ(a,b,c)` 。将第三行预置0得到AND，预置1得到OR；再配合NOT机制组合逻辑。它需要设计支持的专用操作与临时行，不是对任意现售DIMM发三次ACT就保证正确。Ambit第3–5节

原创推导练习：当c=0，多数为1必须a=b=1，所以是a AND b；c=1时，只要a或b为1就形成多数，所以是a OR b。每列都能同时进行这个判断，整行得到大量并行bit操作。

注意两点：一是恢复过程会把参与cell写成结果，必须考虑源操作数保护与复制；二是数据最好位于可共享该局部电路的范围，跨子阵列移动会削弱收益。它利用模拟电荷行为，却产出离散、精确目标的布尔结果；**不能因此称它为一个高精度模拟矩阵乘芯片。**

04 | SIMDRAM：把一个数竖着存，换来大量并行数据

常规布局把一个整数的多个bit放在同一访问字中；bit-serial布局让一个元素的不同bit分布在不同DRAM行，而一条bitline对应一个元素。这样一次整行逻辑能对很多元素的同一位并行计算，复杂运算再沿位次逐步完成。

SIMDRAM将目标操作转换成MAJ/NOT网络，分配计算行并生成DRAM命令序列，再由控制单元执行。论文属于**ASPLOS 2021**，系统评估用gem5等模型，不能当已量产CPU扩展。SIMDRAM机制及方法

你应提出的取舍是：位宽越大，串行步骤越多；乘除法比简单逻辑更难；输入输出的横竖转置、地址定位和跨子阵列复制要计入端到端时间。若一批bit-sliced数据能复用很多次，转换成本容易摊薄；若每次转置后只做一个小运算，专用ALU可能更合适。**功能可表达，不等于每种功能都高效。**

05 | HBM-PIM与UPMEM：仍是数字算，只是算在bank附近

Samsung HBM-PIM走bank附近可编程计算路线。其官方PIMSimulator展示了bank row buffer、向量/标量寄存器、指令寄存器及ADD/MUL/MAC等操作，利用内存命令触发计算。它省掉的是把大批权重搬到外部处理器的过程，不是取消阵列激活与局部数据传输。Samsung官方披露；官方PIMSimulator

UPMEM在DRAM芯片内集成与bank关联的DPU。已读真实系统研究中，每个DPU有专用DRAM bank、WRAM scratchpad和IRAM；MRAM在这里是**Main RAM，不是磁性MRAM材料。**研究发现简单运算、较少跨DPU通信的负载更合适，复杂运算和经host中转的DPU通信会限制收益。UPMEM实验研究

两者都提醒你检查本地算力/带宽配比。一个银行里存了很多数据，却只有很小ALU，就可能很快从外部带宽瓶颈变成计算瓶颈。对GEMV有利，不代表高复用GEMM也比GPU tensor core有利；比较前要对齐精度、batch和输入布局。

06 | 面试拉开差距的部分：一致性与软件契约

假设CPU cache里有A的最新脏副本，而PIM直接读取DRAM旧值，结果会错；若PIM修改了DRAM但CPU还命中旧cache，也会错。可选策略包括显式flush/invalidate、不可缓存区域、按阶段所有权转移或专门一致性协议。选择取决于共享频率和工程复杂度，不是“加个fence就能自动解决所有问题”。

还要处理虚拟地址到物理bank/子阵列的映射、页迁移、进程隔离、抢占、错误恢复与刷新协调。软件库或编译器必须知道哪些运算值得下沉，怎样组织输入，如何判断完成。**接口兼容、源代码不改、二进制不改、性能透明**是四个不同承诺。

模拟CIM则多一层数值契约：阵列电流/电荷需要与数字世界衔接，ADC/DAC、累加、校准和数据搬运都要计入；有效精度、模型精度损失与能耗不可分开报。后续进入数字CIM、模拟CIM、混合CIM和材料专题逐项拆解。对任何PIM方案，先问“数据从哪来、在哪算、怎么回去、谁保证正确”，再看TOPS/W。

面试问答与追问

1 PIM能否替代GPU?

应按算子和数据流划分。低复用、大流量且本地可算的部分可能受益；高复用GEMM、复杂控制和跨bank通信可能仍适合GPU/CPU。PIM增加一种执行位置，端到端需要异构调度和一致性，而非简单整机替代。

继续追问

- 小batch GEMV和大batch GEMM为什么可能选择不同位置?

2 Ambit为什么能做AND/OR，它是不是模拟CIM?

三行共享位线产生多数函数，第三行0得到AND、1得到OR；其物理操作有模拟电荷共享，但目标运算是精确bit逻辑，不是模拟多bit MAC。需要专门支持、临时行和源数据保护。

继续追问

- 结果恢复会改写哪些cell?
- 操作数跨subarray时怎么办?

3 SIMDRAM支持任意组合逻辑，是否意味着高精度浮点运算也很快?

不能。可表达性与性能不同；bit-serial步骤随位宽和操作复杂度增加，还要算转置、复制和命令控制成本。应选择结构规则、数据量大、低位宽且易复用布局的任务。

继续追问

- 用32bit乘法而非AND时，你预期哪个瓶颈先出现?

4 为什么UPMEM旁边带宽高，还会compute-bound?

本地计算核的吞吐和指令支持有限。原本主机上受内存限制的任务搬过去后，除乘浮点等可能需要很多指令，指令流水线先饱和；跨DPU通信还可能经过host。

继续追问

- MRAM在UPMEM文档中指什么?

- 怎样区分kernel加速和含传输的端到端加速?

本章自测

1 MAJ(a,b,0)等价于什么?

A. a OR b

B. a AND b

C. a XOR b

参考答案 B 三输入多数要至少两个1。第三输入固定0，只有a=b=1时输出1。

2 PIM执行前，CPU有输入的脏cache副本，应该关注什么?

A. 只增加PIM时钟即可

B. 数据一致性、所有权和可见性

C. 仅提高ECC强度

参考答案 B PIM读DRAM可能拿到旧值，需明确同步和数据可见性契约。

3 SIMDRAM中bit-serial的主要取舍是?

A. 无限位宽无额外延迟

B. 用更多逐位步骤换取大量数据元素并行

C. 完全不需要布局变换

参考答案 B 大量bitline能并行，但单元素的多bit操作要逐步构造，位宽和转置成本重要。

原始资料

1. Ambit: In-Memory Accelerator for Bulk Bitwise Operations Using Commodity DRAM Technology · 2017 · MICRO 2017
2. SIMDRAM: An End-to-End Framework for Bit-Serial SIMD Computing in DRAM · 2021 · ASPLOS 2021
3. Benchmarking a New Paradigm: An Experimental Analysis of a Real Processing-in-Memory Architecture · 2021 · arXiv:2105.03814；UPMEM真实系统研究
4. Samsung Develops Industry’s First High Bandwidth Memory with AI Processing Power · 2021 · Samsung 2021-02-17官方公告
5. Samsung SAIT PIMSimulator · 2021 · Samsung Advanced Institute of Technology官方开源实现
6. Hardware Architecture and Software Stack for PIM Based on Commercial DRAM Technology: Industrial Product · 2021 · ISCA 2021 Industrial Product

数字存算：从一位乘法到真正可用的整数 MAC

先把“存在哪里”和“怎么算”分开，再亲手把 4 位乘法拆成位平面。你会发现数字 CIM 的难题常常不是乘法本身，而是求和、连线、容量和数据调度。

学习目标

- 用两条独立坐标区分存储介质、计算域和计算位置
- 手推位切片、位串行、符号位及累加位宽
- 解释没有 ADC 之后仍存在的面积和系统瓶颈
- 读懂数字 CIM 论文的精度、算力和实测范围

1. SRAM、RRAM 是存储方式；数字、模拟是计算方式

把存算芯片当作厨房：存储介质决定食材怎样保存，计算域决定怎样加工。SRAM 用交叉耦合电路保持状态，DRAM 用电容电荷，RRAM 用可调电阻；这些名字本身不能决定乘加是数字逻辑还是模拟物理量。另一个独立坐标是位置：在阵列内计算、在阵列外围计算、在逻辑底座上靠近存储计算。项目介绍应把三件事分别说清。

数字路径通常把中间结果表示成可再生的逻辑 0/1；模拟路径让电流、电荷、电压或时间直接承载乘积与部分和。每个晶体管在物理上当然都有连续电压，但这不意味着所有 CMOS 加法器都应该叫模拟计算。分类看的是**信息表示和主要计算机制**，而非一句“底层都是模拟”。

反例很有用：CAP-RAM 用 6T SRAM 储存权重，乘加却在电荷域进行；数字 SRAM CIM 则可以采用逻辑门和加法器。RRAM 也可只保存二值权重，再由数字逻辑完成累加，不能看到 RRAM 就断言端到端模拟。CAP-RAM 原论文；SRAM CIM 分类综述

还有一类边界案例：Ambit 借 DRAM 电荷共享和感放再生实现精确的位逻辑目标。它说明“内部利用模拟行为”和“向上提供数字算子”能同时成立。面试中先声明自己讨论的是电路实现还是软件可见算子，避免分类争论。Ambit 作者说明

2. AND、XNOR、popcount：乘法只是第一小步

对无符号位 a,w∈{0,1}，一位乘积就是 AND：只有两者都为 1，乘积才为 1。若同时算 N 对一位输入，先得到 N 个 AND，再数其中多少个 1，即 population count（popcount，位计数）。它的输出范围为 0…N，至少需要 ceil($\log_2(N+1)$) 位。并行乘法很便宜，收集和累加这些结果往往更贵。

若二值神经网络用 {-1,+1} 编码，XNOR 判断符号是否相同，点积为 `2×popcount(XNOR(a,w))−N`。不要把这个公式直接套到无符号 0/1 数上；相同的门可以服务不同编码，数值含义由编码决定。XNOR 也可能出现在带补码处理的多位结构中，要看作者如何解释符号。

实现 popcount 可用全加器树、压缩树或逐周期累加。平衡树减少逻辑深度，但结果会越来越宽，布线和中间寄存器随之增加。把树放进阵列有利于减少读出搬运，却会挤压纯存储面积；放在外围更规整，但许多位线要汇到外围，可能受扇出和布线限制。Colonnade 的作者摘要明确描述数字位串行结构中的 XNOR 与全加器，以及串行换面积的选择。Colonnade 作者机构摘要

3. 完整推导：4 位 × 4 位的点积怎样拆开

教学例题，非某芯片实测。设输入向量 a=[3,5]，权重 w=[6,9]，均为 4 位无符号数。普通计算得到 3×6+5×9=63。将每个数按位拆开，$a_n=\Sigma_i 2^i a_{ni}$、$w_n=\Sigma_j 2^j w_{nj}$，所以 $\Sigma_n a_n w_n=\Sigma_i \Sigma_j 2^{i+j}\ \mathrm{popcount}(a_i\ \mathrm{AND}\ w_j)$。这就是位切片的数值依据。

位平面	输入两元素的该位	权重两元素的该位
bit 0	[1,1]	[0,1]
bit 1	[1,0]	[1,0]
bit 2	[0,1]	[1,0]
bit 3	[0,0]	[0,1]

对输入 bit 0，与四个权重位平面的 AND 计数依次是 [1,1,1,1]，按权重位移相加得到 1+2+4+8=15。对输入 bit 1，计数 [0,1,1,0]，得到 (2+4)×2=12。对输入 bit 2，计数 [1,0,0,1]，得到 (1+8)×4=36。bit 3 全零。最终 15+12+36=63，与直接乘加一致。

这里有 4×4=16 个位平面对。**位切片**指把位分到不同存储位置或子阵列；**位串行**指把部分位放到不同时间计算，两者可同时存在。如果四个权重位平面并行、每周期输入一位，基础计算需四轮；如果输入与权重位对都完全串行，需十六轮。实际周期还要加读写、流水线、符号处理和输出合并，不能只从“4 位”推断吞吐。

这个拆法不是某篇论文专有技巧，而是二进制展开。它也是比较数字和模拟 CIM 的共同语言：模拟阵列可能一次给出某个位平面的近似和，数字后端再移位合成多位结果。

4. 乘法精度与累加精度不是同一个数字

假设每个输入为 A 位、权重为 W 位无符号整数，N 项点积的最大值是 $M=N(2^A-1)(2^W-1)$，不溢出的最小输出位宽为 $\mathrm{ceil}(\log_2(M+1))$。常用保守上界为 A+W+ceil($\log_2$N)。注意 A、W 如果表示“位数”，绝不能写成 N×A×W 来估最大数值。

教学例题：N=64、A=W=4，最大和为 64×15×15=14,400。13 位只能到 8,191，14 位能到 16,383，因此需要 14 位累加器。你因此能理解“4 位输入、4 位权重、14 位输出”的配置：它没有把每个乘法变成 14 位，而是在保留求和动态范围。ISSCC 2022 paper 11.6 的官方简介确实给出这一精度组合，但此处 N=64 是教学推导，不能据简介反推该宏全部调度细节。ISSCC 官方 paper 11.6

对补码有符号数，最高位带负权。例如 4 位 -3 为 1101，解释为 -8+4+1。正负位平面组合会有加和减，必须处理符号扩展；直接把所有位乘积按正权加起来会算错。累加器还要涵盖最负输入相乘等极值，通常用 $A+W+\text{ceil}(\log_2 N)$ 位补码作为安全设计起点，再逐项证明范围。

若层间允许量化，内部仍可先宽位累加，再统一缩放、舍入和饱和；也可以提前截断节省面积，但这是明确的近似选择，必须测试误差。因此“数字”表示可精确实现所定义的逻辑，不自动等于对浮点原模型完全无损。

5. INT8、零点、浮点：外围会重新长出来

量化数并不总是直接等于真实值。常见表示为 $\text{real} \approx s(q-z)$，s 是尺度，z 是零点。因此一个点积需要处理 $\Sigma(a-z_a)(w-z_v)=\Sigma aw-z_v\Sigma a-z_a\Sigma w+Nz_az_v$，最后再乘 s_as_v。对称权重量化令 $z_v=0$ 可以减少修正项；逐通道尺度有助于精度，却增加尺度存取和后处理。这些都属于架构规格，不能只写“支持 INT8”。

浮点更复杂：尾数乘法只是其中一部分，还需要指数处理、部分和对齐、规格化、舍入和异常值策略。有的 CIM 方案把浮点块转换成共享指数定点；有的让敏感算子退回数字单元。这能降低开销，但必须说明格式覆盖范围和误差。若支持 BF16 输入而内部先量化为低位定点，面试中应说清转换位置，不要称其“原生无损 BF16 乘加”。

从 GPU 经验迁移时，想想 Tensor Core 指令为什么要同时说明输入、乘法、累加和输出类型。CIM 更需要这张表，因为阵列、位平面合并、激活输出可能各用不同位宽。一个完整指标至少应包含 A/W/accumulator/output 四项，以及饱和、舍入、符号和稀疏编码的规则。

6. 没有 ADC 以后，真正卡在哪里

- **存储密度**：加读端口、逻辑门和本地加法器可能使位单元大于普通 6T SRAM。每平方毫米能放的权重下降，更多权重需要换入，数据搬运反而回来。
- **求和与布线**：并行行数增加，部分和更宽、跨区连线更长。时钟树、寄存器、输出网络会吃掉面积和能量，先进工艺不意味着连线延迟按晶体管同比下降。
- **写与算竞争**：单端口或共享节点结构可能不能一边换权重一边算；双端口、双缓冲或专用位单元能缓解，但需要面积。不能把“所有 CIM 都不能同时写算”当定律。
- **形状与利用率**：阵列为 256 列而一层只用 40 列，剩余列不自然产生有效算力。小矩阵、深度可分离卷积、动态序列长度和稀疏模式都可能造成空闲。
- **供数与归并**：输入广播、跨宏部分和、激活、残差、归一化、输出写回仍然需要缓冲和网络。阵列不必搬权重，不等于整层没有搬运。

教学决策方法是先画每一步的数据量与周期，再找最大项。若换权重占总时长 60%，继续把阵列 MAC 加速两倍，整体收益最多约 $1/(0.6+0.4/2)=1.25$ 倍；应先评估权重驻留、批处理或双缓冲。这是 Amdahl 定律的时延形式，并非某芯片测量。

7. 用真实数字芯片学会读指标，而不是记冠军数字

TSMC 在 ISSCC 2022 的数字 CIM 工作很适合练“读标题后的追问”。原始摘要和官方资料支持：5 nm、专门的 12T 位单元、同时 MAC 与写入，且官方简介限定 4 位输入/4 位权重/14 位输出。标题中的 254 TOPS/W 是该宏特定配置结果，不能拿来直接击败 FP16 GPU 的系统能效。原论文摘要；官方精度口径

你应该追问：一个 MAC 算几个 operation？统计了外围和 SRAM 写入吗？是否算片外搬运？结果对应什么电压、频率、切换率和利用率？支持的精度变化后，周期数怎样增长？是否有完整模型映射？若只读到摘要，就明确说“我核对了结构方向，尚未获得完整 PPA 曲线”，这比编出内部细节专业。

Colonnade 则适合学习可配置精度：位串行让低位宽工作少跑一些位轮次，高位宽工作多跑一些。读它的意义是理解面积、精度与时延互换，不是用最低位宽的能效数字替代所有工作负载。它的公开作者摘要可作为起点，详细数据仍需完整论文支持。Colonnade 作者摘要

8. 面试用一段话把数字 CIM 讲清

> 数字 CIM 把权重留在 SRAM 等存储结构附近，以 AND/XNOR 形成一位乘积，再做 popcount、移位和累加。多位计算可以在空间铺开，也可以按位串行，用面积换周期。它能避免模拟 ADC 的精度和转换问题，但加法树、端口、长连线、权重换入和阵列利用率依然限制收益。我会先固定 A/W/累加位宽、目标算子和数据驻留方式，再比较完整层的能量与延迟。

如果继续追问“是否适合你熟悉的 GPU 工作”，不要直接说能替代 GPU。先找权重重复利用高、形状稳定、支持低位精度的子图；再验证权重是否放得下、动态数据如何进出、算子覆盖是否足够。如果目标是大模型 decode，权重容量和模型切换可能压过单宏 MAC 优势；如果是固定端侧模型，权重驻留和低待机需求可能更有吸引力。这些是待测的架构假设，应由工作负载验证。

词汇对照：bit slicing = 位切片；bit serial = 位串行；popcount = 位计数；reduction = 归约求和；accumulator = 累加器；weight stationary = 权重驻留；macro = 可集成的电路宏，不能自动等同完整 AI 芯片。

面试问答与追问

1 数字 CIM 为什么不等于普通 SRAM 后面挂个 ALU?

这是实现连续谱，关键在计算与存储的耦合粒度。若每次仍把完整权重经普通读端口搬到独立 ALU，主要是近存计算；若在位单元、局部位线或紧邻阵列的分布式逻辑中直接形成部分积和归约，能减少内部读出和传输。不能只凭命名判断，要看每个 MAC 的权重实际经过哪些节点，以及面积、时序和能量的统计边界。

继续追问

- 位单元内算与外围算分别牺牲什么?
- 能用数据路径而非产品名称定义边界吗?

2 64 项 4 位无符号点积需要多宽累加器?

最大值是 64×15×15=14,400，因此需要 14 位。应从最大数值推导 ceil($\log_2$(M+1))，不是把 4 位输入与 4 位权重相乘就说 16 位。带符号时另查极值，提前截断时另给误差与饱和规则。

继续追问

- 若 N=65 是否仍是 14 位?
- 如果只输出 INT8，中间还需要 14 位吗?

3 ADC 去掉后，数字 CIM 会天然比模拟 CIM 更好吗?

不会。数字减少模拟误差和转换成本，却可能用更多门、布线和位周期完成同样的乘加，降低权重密度。模拟可以并行累加但需转换、校准与精度约束。必须在相同任务精度、容量、有效吞吐和系统边界下比较，不能只比较峰值 TOPS/W。

继续追问

- 何种工作负载可能更偏数字?
- 高位宽和小矩阵分别如何影响二者?

4 所有 CIM 都不能一边写一边算吗?

不是。冲突由端口和电路共享决定。TSMC 的 2022 数字宏公开摘要明确支持同时 MAC 与写入，依赖专门位单元设计。系统还可用双缓冲或不同 bank 并行，但会花面积、控制和容量。面试时应说明同地址一致性、读写相位和并发条件，不能把宏的能力外推到任意访问。

继续追问

- 双缓冲如何影响有效权重容量?
- 怎样在仿真中定义写算冲突率?

本章自测

1 下面哪句话准确?

A. SRAM 一定是数字计算，RRAM 一定是模拟计算

B. 存储介质和计算域是两个不同维度

C. 只要用了晶体管，宏就属于模拟 CIM

参考答案 B SRAM 可以做电荷域模拟计算；RRAM 也可只提供数字权重。要看计算如何表示、累加与读出。

2 4 位无符号输入和权重的 64 项点积，最大结果是多少?

A. 64×4×4=1024

B. 64×15×15=14400

C. 15×15=225

参考答案 B 4 位无符号数最大是 15，不是 4；64 项求和需 14 位。

3 4 位输入按位串行，4 个权重位平面完全并行，忽略控制开销需几轮?

A. 4 轮

B. 16 轮

C. 1 轮

参考答案 A 一次处理一个输入位、同时处理四个权重位，共四轮；若两侧都串行才有十六个位对轮次。

原始资料

1. A Review of SRAM-based Compute-in-Memory Circuits · 2024 · arXiv review
2. A 5-nm 254-TOPS/W 221-TOPS/mm² Fully-Digital Computing-in-Memory Macro Supporting Wide-Range DVFS and Simultaneous MAC and Write Operations · 2022 · ISSCC 2022, paper 11.6
3. ISSCC 2022 Press Kit — Session 11, paper 11.6 · 2022 · IEEE ISSCC
4. Colonnade: A Reconfigurable SRAM-Based Digital Bit-Serial Compute-In-Memory Macro for Processing Neural Networks · 2021 · IEEE JSSC 56(7), 2221–2233
5. CAP-RAM: A Charge-Domain In-Memory Computing 6T-SRAM for Accurate and Precision-Programmable CNN Inference · 2021 · IEEE JSSC / author manuscript
6. Ambit: In-Memory Accelerator for Bulk Bitwise Operations Using Commodity DRAM Technology · 2017 · MICRO 2017

模块 17 · 核心必会

模拟存算：电流真的会算乘法，但误差也会一起相加

用欧姆定律理解阵列为什么能并行算点积，再把转换器、符号、噪声、失配和漂移放回电路。重点是辨别“有 8 位接口”和“得到 8 位可信结果”的区别。

学习目标

- 从 I=GV 与电荷守恒推导模拟乘加
- 说明符号、多比特权重和 ADC/DAC 的位置
- 定量分析 ADC 共享的面积、吞吐和能量代价
- 用真实芯片解释误差预算及宏到系统的差距

1. 交叉阵列为何能在物理上完成点积

最常见的教学图是电阻交叉阵列：行输入电压 V_i，交点器件电导 G_{ij} 保存权重，列收集电流。在器件近似线性、节点电压受控且互连电阻可忽略时，每个交点产生 $I_{ij}=G_{ij}V_i$，列端电流按 Kirchhoff 电流定律相加，得到 $I_j=\Sigma_i G_{ij}V_i$。输入向量与一列权重的点积因此同时发生，没有逐项发出乘法指令。

教学数值：某列电导为 [10,20,40] μS，三行电压为 [0.10,0.20,0.05] V，对应电流 [1,4,2] μA，列和为 7 μA。若用 $V=\alpha x$、$G=\beta w$ 映射数据，理想列电流为 $\alpha\beta\Sigma xw$，输出再除掉尺度。这里的线性器件、理想电压驱动和读出节点是明确假设，不是现实阵列永远满足的条件。

它省掉的是许多显式乘法与部分局部数据移动，并没有创造免费的计算：驱动电压要耗能，导通器件流过电流，列端要稳定和读出。加大阵列并行度会同时提高动态范围、总电流和互连压降，所以不能把行数无限增大后宣称吞吐线性增长。

2. 不止电流域：电荷域、时间域也能计算

电流域让乘积由电导与电压形成，汇总电流后读出；**电荷域**把输入或乘积编码成电容电荷，通过电荷共享或累积形成结果；**时间域**用脉冲宽度、延迟或振荡次数表示数值。三者都会遇到非理想，只是主要误差源不同。电荷域要关心电容失配、寄生和 kT/C 噪声；时间域要关心抖动、时间量化和长动态范围带来的等待。

SRAM 的数字存储节点可以只负责控制开关，计算发生在外接或局部电容上。CAP-RAM 就是重要反例：6T SRAM 保存位，电荷域完成乘加，并使用 ADC 和校准。论文实际测量也检查不同切片的增益、偏置与整体传递曲线，而非只证明单个电容能储电。CAP-RAM 电路与测量

NeuRRAM 又说明“RRAM CIM 必须是传统电流求和读出”过于狭窄：它采用电压模式的神经元读出，设计目标包括重配置与双向数据流。面试时可用 $I=GV$ 建立直觉，但解释某颗芯片时必须回到它真正采用的接口与电路。NeuRRAM 期刊原文

术语上，ADC 是把连续幅度量成数字码；TDC 是把时间量成数字码。使用时间编码并不意味着从此没有转换成本。PWM 可以实现输入编码，未必需要每行一个传统多位电压 DAC，因此也不能把某一种 DAC 拓扑当成模拟 CIM 的必选配置。

3. 负权重、多比特与转换器怎样拼起来

普通被动电导不能为负。常见差分表示把权重写成 $w\propto G^{+}-G^{-}$，分别读两路贡献后相减。这样可表示正负，却要额外器件、列或读出资源；两边都有较大电流但差值很小时，能量和噪声并不会随结果一起变小。也可以用正负输入相位、参考列或数字符号修正，不存在所有芯片统一的唯一实现。

多比特权重有两种基础路线：把一个器件编成多个电导等级；或用多个低位器件保存不同位平面。前者提高存储效率，但相邻电导窗更窄，对编程与读稳定性要求更高；后者读出更容易分辨，却增加器件、列和合并工作。多位输入也可用幅度多级、脉冲时长或逐位串行传入。

一次模拟读通常只是某一部分和。后端要转换、移位、累加，可能还要处理跨阵列拼接。若 N 个二值乘积相加，要“完全区分所有精确和”，就有 N+1 种输出：N=256 时至少 9 位数字输出。实际方案可限制激活行数、缩小数值范围或允许量化来采用更低位 ADC；这表示做了精度、并行度与面积取舍，并非绕开信息量约束。

权重写入也不是普通寄存器赋值。多级非易失器件可能需 pulse→read→compare→再 pulse 的写校验，把目标导电状态逼近到容许区间。这会增加部署和更新耗时，却可能被大量后续推理摊薄。读频繁、写稀少与在线训练应采用不同成本模型。

4. ADC 共享：面积减少，并不代表转换工作消失

教学假设：一个阵列有 128 个待读列，每个 ADC 不使用流水线，一次转换占用 10 ns，持续转换启动间隔也为 10 ns；这里的 10 ns 不只是输入到结果的延迟。互连稳定、复位、采样和数字合并时间暂忽略。若每列一个 ADC，128 列可在一次 10 ns 窗口转换完；若只放 8 个 ADC，通过多路选择器共享，需要 16 批，至少 160 ns。若输入有 8 个串行位轮次，则转换下界分别为 80 ns 与 1.28 µs。

这个例子只隔离转换服务能力，不是任何论文芯片的时延。真实设计可能有流水线、双缓冲和重叠，应使用转换启动间隔计算持续吞吐，并另外计入流水线首批延迟。例如结果延迟为 10 ns、启动间隔为 1 ns 的流水 ADC，16 批可在约 10+15×1=25 ns 内输出完毕，不能仍按 160 ns 估算。ADC 阵列每单位时间能处理多少个列结果仍给出吞吐上界。增加 ADC 数量会提高面积、瞬时功率和布线压力；减少数量则加长队列，并可能让计算阵列等待读出。

若每次转换动态能量相同，128 个列结果仍需 128 次转换。单纯把 128 个 ADC 换成 8 个，不会自动让动态转换能量缩为 1/16。漏电、时钟、复用开关和待机策略可能改变总能量，应该分项建模。共享可以省面积，也可能允许每个 ADC 做得更好，但要付出时间。

面试可把它与 GPU 执行单元类比：共享功能单元减少面积，却形成结构冲突。需要用每轮产生多少结果、转换器每秒服务多少结果，检查生产与消费是否匹配，再根据延迟目标决定并行度。

5. 把误差放到正确的位置：不是一个“噪声参数”包办

误差来源	直观机制	常见应对与代价
编程误差	实际电导偏离目标	写校验、冗余器件；多花时间和面积
失配与偏置	不同列的增益、零点不同	逐列或分组校准；需要系数与校准流程
读噪声	同一输入重复读结果波动	提高信号、平均采样；耗能或增加延迟
互连 IR drop	线阻使远端电压不同，且随数据改变	小阵列、低电流、更宽线；容量或面积代价
非线性与饱和	器件、驱动或 ADC 超出近似线性区	缩放、限制激活行数；降低有效动态范围或并行度
漂移与温度	状态随时间、环境变化	补偿、重校准、重编程；维护成本

教学压降估算：一段等效 100 Ω 线承载 20 μA，压降为 2 mV。若该处目标读电压只有 100 mV，这已占 2%。真实 crossbar 是分布网络，电流随输入与电导图案变化，不能把所有误差简化成一次全局乘 0.98。压降还可能与多个权重相关，使独立高斯噪声模型失真。

静态校准适合稳定的增益与偏置；随机噪声需要统计处理；数据相关非线性需要限制工作点或更复杂模型；漂移需要时间维度的维护。把它们都归结为“材料不稳定”会漏掉驱动、连线、采样和算法映射问题。

6. 8 位输入、8 位输出，为什么仍不是 8 位有效计算

8 位输入表示外部可给 256 个编码，8 位输出表示接口可返回 256 个编码。它们没有保证从输入到输出的物理链路能可靠区分相邻理想结果。若误差、非线性或噪声足够大，低位输出可能只是抖动。反过来，任务准确率接近量化软件基线也不能证明每个点积都实现了逐位精确整数计算。

至少分别报告：接口位宽、权重状态数、部分和与 ADC 位宽、数值误差分布、任务级质量。对 ADC 可在规定测试条件下使用 ENOB 等指标，但针对整个 MVM 更应报告归一化 RMSE、偏置、尾部误差、输入分布及测量范围。不要把 ADC 的某个理想正弦测试公式直接当成完整阵列的通用精度。

真实反例：NeuRRAM 的扩展数据检查过提高输入位宽的效果，特定配置下输入电压降低造成信噪比问题，因此位宽增加未必改善 RMSE。IBM PCM 论文也把权重误差、残余电路误差与数字量化分开分析；它的 8 位输入/输出不能被改写为“每个权重具有 8 位有效精度”。NeuRRAM Extended Data Fig. 5；IBM PCM 精度分析

优秀回答会进一步问误差在哪一层最敏感：层归一化前后的尺度、注意力 logits、输出投影与普通中间层可能不同。可通过逐层误差注入和任务验证分配精度，而不是强迫所有层使用同一个位宽。

7. 真芯片教你的三件事：校准、协同设计、系统边界

CAP-RAM：电荷域并非天生没有误差。它通过测量并校准偏置、增益和部分非线性来改善输出。读论文时应看整个计算链的统计，不能只截取理想电荷共享的一条直线。CAP-RAM 校准章节

NeuRRAM：器件、电路、映射与训练共同决定模型效果。噪声感知训练使用硬件特征，校准也与输入分布有关；“用了算法补偿”不是缺点，但算法步骤和部署成本必须纳入可复现性说明。NeuRRAM Methods

IBM 64 核 PCM：阵列外还包含数字处理和通信。公开预印本的演示明确说明层间激活传输经过测试板 FPGA，部分 max-pooling 和 embedding 不受芯片支持。因此可称为重要的多核混合信号演示，却不能把局部矩阵乘能效直接声称为完整自包含应用能效。IBM 论文 §V 与实验平台

读这些论文时，你的输出应是一个表：哪些工作在器件、阵列、片上数字逻辑、FPGA、主机完成；哪些数字实测、哪些来自仿真；模型是否剪枝或为硬件重新训练；是否含映射、编程和校准时间。面试官会从这张表判断你是否具备产品架构视角。

8. 怎样建立一个诚实的模拟 CIM 分析模型

先用精确整数点积作为参照，接着分别加入输入量化、权重编程偏差、列增益/偏置、随机读噪声、ADC 量化与饱和。每次只增加一类误差，记录 MVM 误差和任务精度；然后测试误差相关性、长时间漂移和温度角落。没有真实器件数据时，只能称为参数敏感性分析，不能称为对某芯片的精确复现。

面积与能量侧分别估阵列、DAC/驱动、ADC、数字合并、缓冲和通信；时延侧分别估建立时间、位轮次、转换批次、跨宏归并。最常见的虚假乐观来自只算阵列核心，最常见的虚假悲观则是假设所有 ADC 必须高精度、每行必须独立高成本 DAC。两者都应回到实际拓扑和任务需求。

> 我理解模拟 CIM 的优势是用物理并行完成乘积与局部归约，代价是数值误差和外围读出。我的设计顺序是先确定可容忍的任务误差，再分配阵列规模、输入编码、权重位切片与 ADC 精度，最后把校准、通信和容量放进系统模型。材料改进很重要，但我不会把整个问题简化为器件电导更稳定就一定成功。

面试问答与追问

1 模拟 crossbar 为什么能算矩阵乘法？假设有哪些？

以电压编码输入、电导编码权重，在近线性器件和受控节点电压下，每个交点按 I=GV 形成乘积，列上按电流守恒相加。理想公式依赖线阻可忽略、驱动和读出维持工作点、器件不过饱和。现实需要考虑线阻、非线性、噪声及转换器。其他芯片还可能用电荷或时间域，不能一张示意图概括全部。

继续追问

- 输出电流怎样变成负数结果？
- 阵列行数增大为什么不总能提高有效性能？

2 128 列共用 8 个非流水 ADC，每个 ADC 的转换占用周期和持续启动间隔均为 10 ns，一轮读出至少多久？

按题设每个非流水 ADC 每 10 ns 只能完成一次转换，至少分 16 批，转换部分是 160 ns，未含采样、建立、复位、复用和数字归并。若 8 位输入串行，转换服务需求达到 1.28 μs。若换成流水 ADC，必须另知持续转换启动间隔；仅凭 10 ns 结果延迟不能推得 160 ns。共享减少 ADC 数量，不会自动减少需要转换的结果数。

继续追问

- 面积、瞬时功率和总动态能量怎样变化？
- 若 ADC 更快，阵列是否一定同步加速？

3 厂商说支持 8 位输入输出，怎样验证有效精度？

要求说明权重编码、累加和 ADC 位宽，测不同输入分布下的误差、偏置、饱和、重复读噪声和尾部，以及跨列跨芯片差异。然后比较任务准确率与同量化软件基线。接口位宽只是数据格式，不能证明端到端 MVM 每一位都可靠。

继续追问

- 为什么增加输入位宽可能不降低 RMSE?
- 任务准确率相同能证明数值误差为零吗?

4 静态校准能否消除模拟 CIM 所有误差?

不能。稳定偏置和增益通常适合校准，但随机读噪声、随数据变化的压降、时间漂移和温度变化需要不同策略。平均采样用时延换噪声；限制工作点用并行度换线性；重校准和重编程增加维护成本。应先做误差分解，再选择补偿方法。

继续追问

- 独立高斯噪声模型漏掉了什么?
- 如何把误差预算传到模型映射与训练?

本章自测

1 电导 20 μS、读电压 0.2 V 时，理想电流是多少?

A. 4 μA

B. 100 μA

C. 0.1 μA

参考答案 A $I=GV=20\times10^{-6}\times0.2$ A=4 μA。

2 减少 ADC 数量并复用，最可靠的直接结论是什么?

A. 动态转换能量必然按数量同比下降

B. 通常省转换器面积，但增加转换批次或吞吐压力

C. 接口精度自然提高

参考答案 B 每个列结果仍需转换；总能量还取决于转换次数、复用、漏电和工作频率。

3 哪项最能说明“SRAM 不是必然数字 CIM”?

A. SRAM 需要时钟

B. 6T SRAM 可保存权重并控制电荷域乘加

C. SRAM 不需要刷新

参考答案 B 存储状态是数字不妨碍计算中间量由模拟电荷表示，CAP-RAM 是实际例子。

原始资料

1. A Review of SRAM-based Compute-in-Memory Circuits · 2024 · arXiv review
2. CAP-RAM: A Charge-Domain In-Memory Computing 6T-SRAM for Accurate and Precision-Programmable CNN Inference · 2021 · IEEE JSSC / author manuscript
3. A compute-in-memory chip based on resistive random-access memory · 2022 · Nature 608, 504–512
4. A 64-core mixed-signal in-memory compute chip based on phase-change memory for deep neural network inference · 2023 · Nature Electronics 6, 680–693; public preprint 2022
5. Compute in-Memory with Non-Volatile Elements for Neural Networks: A Review from a Co-Design Perspective · 2023 · Advanced Materials 35, 2204944

模块 18 · 深入研读

数模混合：把精度花在敏感位置，把并行留给合适阵列

“混合精度”和“数模混合”经常被混成一个词。这里分清算法、位平面、计算电路和系统异构四层含义，并用 2025 年真实处理器和训练研究练习性能、精度与容量取舍。

学习目标

- 区分混合精度、混合信号和异构存储三个概念
- 推导 MSB/LSB 分工与误差放大
- 理解浮点对齐和数字梯度残差的必要性
- 按相同精度与系统边界比较方案并设计验证计划

1. 先问“混合的是什么”，避免概念错位

说法	混合对象	一个例子
算法混合精度	不同数值格式或位宽	一些层 INT4，敏感层 INT8，归一化用浮点
电路混合信号	模拟与数字计算过程	模拟阵列给部分和，ADC 后数字移位累加
位平面混合	同一个数的不同位交给不同电路	高位精确数字计算，低位允许模拟近似
系统异构	多种存储/计算引擎	SRAM 数字 CIM、RRAM 模拟 CIM 和常规向量核协作

它们可以重叠，也可以独立存在。完全数字的 GPU 也能做混合精度；固定 INT4 的模拟宏也可能属于混合信号，因为 ADC 后有数字归并。不能看到“mixed precision”就断言该论文把 MSB 放数字、LSB 放模拟，这只是一种可选实现。

给架构图加四个标注就容易清楚：数据格式、计算域、存储介质、执行位置。随后问精度配置在编译时静态决定，还是运行时可以切换；切换是否需要重新编程、重排数据或重新校准。所谓灵活性要用具体配置与切换代价证明。

2. MSB 数字、LSB 模拟：先把算式算明白

教学例题：把一个 8 位非负权重 W=173 拆成 $W=16W_h+W_l$，$W_h=10$、$W_l=13$。输入 x=7 时，xW=16×70+91=1,211。若高位乘积由精确数字路径完成，低位模拟路径输出 91+δ，最终结果是 1,211+δ。若反过来让高位路径产生同样绝对误差 δ，误差会被位权放大到 16δ。

当 δ=3，低位错误导致结果 1,214；高位部分和错误导致 1,259。这个算例解释为什么某些方案优先保护高位，但不是证明所有网络只保护高位就足够：低位有很多项时误差仍可累积；符号位、抵消严重的输出、层内异常值也可能敏感。低位阵列输出范围较小，尺度和噪声分布必须实际验证。

如果输入也拆成 $x=16x_h+x_l$，则乘积包含 $256x_hW_h+16x_hW_l+16x_lW_h+x_lW_l$ 四项。不能只算“高高”和“低低”而漏掉两个交叉项。符号补码拆分还要明确高位部分如何解释为有符号数。

最简单的误差预算为 $\Delta y=16\Delta y_h+\Delta y_l$；若误差相关，不能把方差直接按独立噪声开平方。可先用最坏界 $|\Delta y|\le16|\Delta y_h|+|\Delta y_l|$，再用实测统计做更紧估计。设计应同时问“误差有多大”和“这种误差对目标任务有什么影响”。

3. 模拟部分和 + 数字累加，是一种普遍分工

模拟阵列不一定承担完整多位 MAC。它可以只完成少量激活行或某个位平面的局部和，转换后再由数字单元精确处理位权、符号和跨阵列归约。减少一次模拟计算的动态范围，可能降低 ADC 难度；代价是更多轮次、更多数字累加和数据暂存。设计目标是在误差、周期与面积之间找到合适分界。

2025 年 Nature 的 memristor/SRAM 混合处理器公开扩展图说明：memristor 路径采用 5 位 ADC，随后数字累加跨权重位与输入轮次的贡献，SRAM 路径则采用数字计算结构。这是一个具体实现，不能推广成“所有混合芯片都用 5 位 ADC”。主文需要订阅，本课程证据范围是作者摘要与公开扩展图注。2025 Nature Extended Data Fig. 4

一个看似反常但合理的情况是：更多数字后处理反而改善总体能效，因为它让模拟前端不必追求很高精度。反过来，如果每个微小部分和都需转换、搬运和累加，模拟并行优势也可能被外围吃掉。应画出每个输出经历多少次 ADC、多少次移位加法、多少字节通信，再做比较。

4. SRAM 与 RRAM 混合，要回答权重与层放在哪里

异构系统可以让写入频繁、精度敏感或形状不合适的工作进入 SRAM/数字路径，把稳定、容量较大且能容忍误差的权重放入非易失模拟阵列。这样利用两种技术的长处，但也多了一套映射、缓冲、地址空间、结果归并与调度问题。

2024 年 Science 和 2025 年 Nature 的原始研究都展示了 memristor 与数字或 SRAM CIM 的融合方向。前者本课程仅核读作者摘要；后者公开资料进一步说明按层/核粒度考虑数值类型与硬件配置，并利用误差敏感性决定配置。可以据此学习协同设计方法，不能把论文名字简化成一种固定“高位数字、低位模拟”规则。Science 2024 作者摘要；Nature 2025 摘要与扩展数据

教学映射问题：如果某层只占 2% 运算但精度特别敏感，把它放在精确数字单元可能代价较小；若该层恰是串行依赖链的瓶颈，2% 运算也可能占很大时延。应基于测量的延迟、能量和通信，而非仅按 FLOPs 比例分配。若两个引擎输出格式不同，还需量化转换和尺度管理。

接回你熟悉的 HBM/HBF：它们讨论的重点是容量、带宽与数据供给；CIM 讨论的是计算在数据附近如何发生。即使权重驻留在 RRAM，动态 KV cache、激活和不支持的算子仍可能需要其他存储及 GPU。层级存储方案与 CIM 方案可以组合，但不是互相替代的概念。

5. 为什么“支持浮点”仍要追问指数对齐

浮点累加必须把不同指数的数对齐。**教学例题：**$12=1.5\times2^3$，$2.5=1.25\times2^1$。以 2^3 为共同尺度，第二项尾数应变成 0.3125，得到 $(1.5+0.3125)\times8=14.5$。若只有很窄的定点小数位，较小项可能在右移时被舍掉；指数跨度越大，这个问题通常越突出。

共享指数或块浮点能减少每个元素独立处理指数的成本，但需要决定分组粒度、异常值如何处理、数据是否重排以及误差能否接受。逐输入、逐向量、逐 kernel 选择共同指数会给出不同的硬件与精度开销。即使阵列只看到定点电压，外部输入仍可声明某种浮点格式；两者不矛盾，但不能混淆“能接收浮点”与“每步严格遵循完整浮点语义”。

2025 年论文公开扩展图讨论输入与 kernel 的对齐方式，并给出相关能量分析，其中相应对比是**模拟结果**。阅读时需将实测芯片任务、设计流程和模拟组件收益分开标记。Nature 2025 Extended Data Fig. 8

面试可以这样追问：支持哪些指数范围？小项被截断到何种阈值？累加发生在哪个域、多少位？NaN/Inf/非规格数是否覆盖？硬件论文的 AI 工作格式不一定承诺通用处理器所有浮点角落语义，规格必须明确。

6. 训练的混合，与推理的混合不是同一难题

推理主要读取已编程权重，偶尔部署或更新；训练则反复读取、反传和更新，还要维护优化器状态。若每个细小梯度都直接变成一次不稳定器件脉冲，小更新可能消失，正负更新不对称可能产生偏置，写能量与耐久也更突出。因此训练对更新路径要求更高，但并不意味着一定要等待完美对称器件。

Nandakumar 等的混合精度训练方案让模拟存储承担主要加权和，同时在数字端高精度累积尚未写入的更新，累计到器件可实现的步长才发脉冲。论文包含硬件/软件结合实验，也包含行为模型与系统建模；不能把全部结果称为端到端训练芯片实测。Mixed-precision deep learning 原文

教学例题：器件平均可写步长设为 0.01，连续三次目标更新为 0.003。若每次直接舍入为零，三次信息全丢；数字残差保留到 0.009 后仍未必发脉冲，再来 0.003 达到 0.012 时发一次近似 0.01 脉冲，保留残差 0.002。实际器件更新随机且依赖状态，因此残差机制并不神奇消除所有误差，还需实际训练验证。

这说明数字精度可以用在对收敛最关键的状态上，模拟并行用在高吞吐的矩阵操作上。代价是额外数字存储、更新计算和通信；大模型优化器状态可能很大，要把这些容量计入系统，而不是只数模拟权重阵列。

7. TOPS/W 之前，先把比较条件放在同一张表

至少对齐六件事：输入/权重/累加/输出精度；允许的任务误差；算子与矩阵形状；有效容量与驻留方式；稀疏统计口径；能量统计范围。1 位逻辑操作、4 位 MAC、BF16 浮点运算不是可直接互换的 operation。一个 MAC 按一次还是两次运算也要说明。

按层次列数：**阵列**只看存储计算核心；**宏**通常还含局部外围，但具体定义需读论文；**芯片**包含数字缓冲、网络及控制；**系统**还涉及外存、主机、供电和数据输入输出。四个数都可以有价值，但必须标明边界。校准、写入和唤醒是否摊入，取决于每次部署后运行多少次推理。

教学摊销例题：假设编程与校准耗能 1 mJ，每次推理 10 μJ。部署后只跑 10 次，平均每次总耗能为 110 μJ；跑 1,000 次，则为 11 μJ。固定权重长期运行与频繁模型切换会得出截然不同的结论，不能只报稳态单次推理。

稀疏也需谨慎：若某些零没有真正关闭对应行、列或运算周期，按“等效稀疏算力”放大 TOPS 并不等于功耗同比下降。阵列利用率低、形状不匹配和不同引擎间同步都可能让系统吞吐远低于峰值。

8. 一份能送评审的混合 CIM 设计提案

1. 固定目标：模型、batch、输入尺寸或上下文长度、质量阈值、延迟/吞吐、功耗及模型切换频率。
2. 建立纯数字与量化软件基线：把量化误差和器件误差分开，避免把算法本身的精度变化归因给芯片。
3. 做逐层敏感性实验：分别注入权重偏差、读噪声、ADC 饱和、输入量化；测误差相关性与异常值。
4. 划分层与位平面：说明什么进入数字，什么进入模拟，跨域需要多少转换、缓存和归并。
5. 核算容量、写入、编程校验、校准、通信和利用率，给出平均与尾部延迟，不只报 MAC 峰值。
6. 做边界压力测试：小矩阵、频繁换模、温度变化、长时间漂移、极端输入、引擎拥塞。

完成这些分析后才能回答“混合是否值得”。你可以诚实地说：我复核了公开资料并建立教学模型，计划用公开器件误差数据做参数扫描；若还没有运行，就不要声称已经复现该芯片。真正有价值的项目不是堆论文名字，而是能解释在哪些条件下你的方案会输，以及下一步要测什么。

面试问答与追问

1 混合精度一定意味着数字和模拟电路混合吗?

不一定。混合精度指不同数值格式或位宽，也能完全数字实现；混合信号指计算路径中存在模拟与数字部分。位平面混合和 SRAM/RRAM 异构又是其他维度。讨论具体论文应分别标注格式、计算域、介质和执行位置。

继续追问

- ADC 后接数字累加是否算混合信号?
- 同一层的高低位分工与不同层分配引擎有什么区别?

2 为何有方案把高位交给数字路径?

同样绝对误差在高位会被更大位权放大。例如 W=16W高+W低，高位部分和误差 1 会变成输出误差 16。保护高位可能高效，但要考虑交叉项、符号、抵消和层敏感性；不是所有场景都可忽略低位。应先建立误差预算，再测任务质量。

继续追问

- 输入和权重都拆成两部分时有几项?
- 误差相关时如何估最终方差?

3 非易失器件更新不够线性，训练一定做不了吗?

不能一概而论。数字端可保留高精度更新残差，累积到可写步长才触发器件更新，算法也可适应部分非理想。但器件噪声、耐久、偏置和数字状态容量仍有成本。已有混合训练研究的硬件/软件实验与仿真应分开看，不能据此宣称所有大模型训练问题已解决。

继续追问

- 数字残差占多少额外存储?
- 推理是否同样频繁更新权重?

4 如何判断一篇混合 CIM 论文的高能效能否用于产品?

先对齐任务质量、数值格式、矩阵形状和有效容量，再确认数字是阵列、宏、芯片还是系统边界。检查编程校准与换模是否摊销、通信和不支持算子在哪里执行，以及实测与模拟分别是什么。最后用自己的工作负载测利用率和端到端延迟，无法从标题的 TOPS/W 单独得出产品结论。

继续追问

- 频繁模型切换会改变什么?
- 稀疏等效 TOPS 和实际省电如何区分?

本章自测

1 W=16W高+W低，高位部分和误差为 2、低位精确，最终误差是多少?

A. 2

B. 16

C. 32

参考答案 C 最终误差为 16×2=32，高位的误差会被位权放大。

2 哪种情况属于算法混合精度，但未必属于模拟计算?

A. GPU 不同层使用 INT4 与 BF16

B. RRAM 列电流经 ADC 输出

C. 电容电荷共享

参考答案 A 不同数字格式可在纯数字硬件上混用；计算域是另一维度。

3 编程校准 1 mJ，推理 10 μJ，运行 1000 次，平均每次总能量为多少?

A. 10 μJ

B. 11 μJ

C. 1010 μJ

参考答案 B 固定开销摊到每次为 1000 μJ/1000=1 μJ，加 10 μJ 得 11 μJ。

原始资料

1. A mixed-precision memristor and SRAM compute-in-memory AI processor · 2025 · Nature 639, 617–623
2. Fusion of memristor and digital compute-in-memory processing for energy-efficient edge computing · 2024 · Science 384, 325–332; DOI 10.1126/science.adf5538
3. Mixed-precision deep learning based on computational memory · 2020 · Frontiers in Neuroscience 14, 406; author manuscript
4. A 64-core mixed-signal in-memory compute chip based on phase-change memory for deep neural network inference · 2023 · Nature Electronics 6, 680–693; public preprint 2022
5. Compute in-Memory with Non-Volatile Elements for Neural Networks: A Review from a Co-Design Perspective · 2023 · Advanced Materials 35, 2204944

材料与落地瓶颈：不是所有存算路线都在等一种新材料

把 SRAM、DRAM 和多种非易失器件放在同一个架构框架中：材料决定可用的状态和可靠性，电路与系统决定这些状态能否转化成有价值的计算。核心问题是哪些瓶颈能靠材料解决，哪些必须换架构。

学习目标

- 比较七类存储介质的状态机制及与 CIM 相关的限制
- 区分耐久、保持、漂移和读稳定性
- 从单器件跨到阵列、工艺、封装和良率分析落地难题
- 用定量能量分解回答材料改善的系统收益

1. 七种介质，先看“用什么保存状态”

介质	状态机制与机会	与 CIM 相关的主要代价
SRAM	交叉耦合逻辑保持位，快速读写，便于与 CMOS 逻辑集成	面积与漏电，CIM 逻辑/多端口降低存储密度；也能做模拟电荷计算
DRAM	电容电荷，存储密度高，局部感放与逻辑底座可利用	漏电刷新、破坏性读与恢复、时序限制；增加计算不应破坏存储可靠性
RRAM	离子/缺陷分布或导电路径改变电阻，具非易失多态潜力	具体机制不同；状态分布、随机更新、写校验、选择器与阵列干扰
PCM	相变材料晶态/非晶态改变电阻，可保存多级状态	编程加热、漂移、写能量和耐久；热与状态历史需建模
FeFET	铁电极化改变晶体管阈值，利用阈值或读电流表示状态	极化与陷阱耦合、阈值分布、保持与循环可靠性，需匹配工艺
MRAM	磁隧道结磁化相对方向改变电阻，非易失、适合稳定二态	模拟多态与读出裕量受限，低电阻可能带来大电流，写机制有面积/电流取舍
Flash	浮栅或电荷俘获改变阈值，成熟高密度非易失存储	高压编程/擦除、更新粒度与寿命、阈值漂移和读干扰；专用模拟设计不等于普通 NAND 任意计算

这张表给出机制和设计关注点，不给每类材料统一的寿命、位数或速度上限。不同材料体系、单元结构、工艺节点和操作条件会相差很大。综述适合建立地图，实际规格必须回到器件与芯片原论文。非易失存算协同设计综述

“非易失”只表示断电后状态可保留，并不表示读电路、ADC、数字网络和控制器不耗电；“多级”只表示存在多个可区分状态，并不直接保证这些状态能提供线性、稳定的模拟乘法权重。

2. 材料进步可以直接改善什么

状态一致性：同样编程条件得到更窄分布，可减少校验次数或冗余。**读稳定性**：噪声、漂移和温度敏感性减小，可降低校准频率和误差。**写行为**：更低电压、更少能量、更小可控步长，利于部署和训练。**电阻工作区**：合适的高阻态能降低电流与 IR drop，但也可能降低信号幅度，因此要与读出噪声匹配。

训练常希望连续更新更线性、正负更对称，让给定数量的增强/减弱脉冲产生可预测效果。推理通常更关心写入后状态是否稳定、读误差能否控制、部署写校验多久能完成。两类需求有重叠，但优先级不同：固定推理权重可接受较慢写入，频繁在线更新则未必。

材料研究还可能改善热预算、与 CMOS 工艺的兼容性和晶圆内分布，这些直接影响集成与生产。但“某个单器件电导可调 256 级”并不等于“百万个器件都能在所有温度下可靠提供 8 位模拟权重”。后者要求状态间距、噪声、漂移和尾部分布同时满足约束。

架构师要把材料指标翻译成系统指标：更稳定的电导能否少用一次读取平均？更大的可用电阻是否允许更大阵列？更快的写入是否降低模型切换时延？如果无法给出这样的因果链，材料优势可能停留在器件测试曲线上。

3. 材料进步不会自动消除什么

- **转换器**：只要连续量需要进入数字控制或后处理，就存在采样、量化或时间读出的成本。更好器件可能降低精度需求，但不会自动删除转换工作。
- **容量和利用率**：模型比片上权重大很多时仍需分块与换入；不规则矩阵不会因为电阻更稳定就填满阵列。
- **通信与缓冲**：输入广播、跨宏归约、层间激活与动态状态仍需移动。NoC 拥塞、SRAM 缓冲不足属于系统问题。
- **算子覆盖与编译**：矩阵乘快不代表归一化、Softmax、索引、动态控制和布局转换都快。映射、调度、运行时与软件接口要实际实现。
- **供电和热**：阵列大并行产生瞬时电流，模拟参考与数字开关噪声可能互相影响；封装、IR drop 和温度梯度仍需设计。
- **可验证性**：数字逻辑要验证功能，模拟与非易失状态还要验证统计与时间行为，生产测试和失效诊断不能省略。

对于 SRAM 数字 CIM，许多工作已经主要面对常规 CMOS 的密度、逻辑、布线和调度问题，根本不以新型电阻材料为前提。即使使用 RRAM 或 PCM，也可通过数字残差、校准、冗余、分块和异构映射缓解部分非理想。材料与架构应共同推进，而不是串行等待。

4. Endurance、retention、drift：三件不同的事

Endurance（耐久）问经过多少写擦或更新循环后仍满足规定指标。**Retention（保持）**问某个已编程状态在给定时间、温度和误码要求下能维持多久。**Drift（漂移）**问状态在时间中怎样变化；模拟计算即使尚未跨越数字判决阈值，也可能因电导逐渐变化而误差增加。**Read disturb（读干扰）**则问反复读取本身是否改变状态。

它们可能互相取舍：更强写脉冲可能扩大状态间隔，却加剧损伤；更高温度可加速某些失效，不能简单把所有材料都套进同一加速模型。数字存储可借 ECC、刷新或重读提高可用性，模拟乘加还要考虑分布式小误差怎样影响数值和任务。

2026 年 FeFET 可靠性论文研究自然老化器件及后续操作，提供脉冲工程、循环和保持证据。读它应特别区分“器件自然存放多年”“编程态在条件下保持”“通过模型外推长期保持”。论文中的某批器件、某温度的循环数不能写成整个 FeFET 领域的寿命上限。FeFET 原论文与实验条件

教学提问：模型一小时换一次与每微秒更新权重，对耐久的需求显然不同；而部署后数月不换模的设备更看重读稳定和维护。不要先背一个耐久数字再选应用，应先从产品工作周期推导所需写次数、保持时间和容许误差。

5. 单器件做到，不等于阵列做到，更不等于量产做到

第一层，单器件。报告状态窗、读写曲线、温度、脉冲条件、重复次数和个体差异。漂亮的典型曲线可能只代表少数样本；要看分布和失效样本。

第二层，阵列。引入选择器、晶体管、字线/位线电阻、寄生、串扰、半选干扰及读出外围。百万个单元需要同时满足规范，尾部误差会变重要。扩大阵列也会改变供电、热和编程并行度，不能把单器件能量乘器件数就当成真实阵列能量。

第三层，工艺集成。PDK 需要可用的模型、设计规则、可靠性规则和角落数据；BEOL 集成要满足后道工艺的热预算、材料与互连兼容性。前道晶体管很好不等于新增存储层自动可靠，研发样片可工作也不等于生产流程已经稳定。

第四层，产品制造。关注晶圆内/间差异、缺陷、良率、测试时间、修复冗余和封装后变化。模拟输出还需明确每颗芯片校准多少参数、多久完成、是否写入持久存储，以及客户使用期是否重校准。

教学概率例题：假定每个单元独立失效概率为 10^{-6}，一个含 100 万单元、完全不允许坏单元的阵列，其零失效概率约为 $(1-10^{-6})^{(1,000,000)} \approx e^{-1} \approx 36.8\%$。真实缺陷并不一定独立，工程上也有冗余和容错，所以这不是良率预测；它只说明“单元成功率很高”仍不足以跳过阵列统计。

6. 三篇材料相关论文，教你不要等“神材料”

MRAM：换求和方式。2022 年 Nature 工作面对 MRAM 低电阻导致传统电流求和功耗偏大的问题，采用电阻求和架构并实现阵列。它的价值是展示器件约束可以推动电路改变；读结果还要分清实测网络与使用实测误差的仿真。MRAM 原论文

PCM：接受器件不完美，研究补偿边界。IBM 论文同时量化编程误差与时间漂移，并使用数字处理和硬件感知方法。它不能证明材料问题已完全解决，但说明产品路线可以通过器件、电路与算法共同逼近目标，而非先要求一个完美电导。IBM PCM 误差与补偿研究

FeFET：可靠性是工作条件和统计问题。最新可靠性研究提醒你检查自然老化、陷阱、脉冲工程和外推假设。一个材料名字覆盖很多结构与工艺，只读标题就给它统一寿命是不专业的。FeFET 可靠性全文

这些例子并不表示材料研究不重要；它们说明决定成败的是约束组合。架构师的任务是把器件的长处放到适合的工作负载里，把短处转化成明确的校准、冗余、精度或时序预算，再判断总成本是否仍有优势。

7. 算一个数：阵列省电 10 倍，系统可能只提升 1.22 倍

教学假设：一次推理总能量归一化为 1，其中存储计算阵列占 20%，ADC/驱动、数字处理、通信及其他部分占 80%。新材料让阵列能量降低到原来的 1/10，其他部分不变，则新总能量为 `0.8+0.2/10=0.82`，整体能效提升为 `1/0.82≈1.22 倍`，总能量只降低 18%。

因此评估器件突破时，第一步是测原系统各项占比，第二步是检查新器件是否连带改变外围。若更稳定的状态还能减少 ADC 采样或校准，系统收益可能超过单独阵列项；若它需要更高写电压、额外选择器或复杂读出，收益也可能更小。这个公式不是否定材料，而是帮助找到改进最有价值的位置。

同理，权重读取减少不等于所有内存流量消失。对于大模型，KV cache 和激活动态变化；对于小模型，唤醒、编程与待机可能占主要比例。应基于你的产品使用模式算能量，不把另一种场景中的占比拿来直接用。

一个有说服力的评审结论应写成条件句：若权重驻留达到某比例、校准频率不超过某阈值、外围能量受控，那么新材料可带来某区间收益；不满足时优先优化通信或利用率。所有阈值都应来自测量或明确教学假设。

8. “是不是都在等材料创新？”可以这样回答

> 不能把存算融合看成统一等待一种新材料。数字 SRAM CIM 主要可以在现有 CMOS 平台上推进，瓶颈是面积、布线、位宽、容量和系统利用率。模拟及非易失 CIM 确实受到器件一致性、漂移、读噪声、写入和可靠性限制，但转换器、网络、编译和算子覆盖也很关键。材料改善能扩大可行区间，电路与算法可以缓解部分非理想；最终应看同等任务质量下的系统能量、延迟和成本。

如果面试官继续问“你会押哪条路线”，先给条件：固定端侧模型和长时间权重驻留，可以重视非易失与低待机；频繁更新、复杂精度或通用算子，数字或异构路径更容易保持灵活；面向 DRAM 厂商，则还要问新增逻辑怎样影响密度、刷新、良率、热与标准接口。不要只凭论文峰值或材料热度下结论。

你可以提出下一步实验：用同一个量化模型，扫描器件噪声、漂移、写次数、ADC 精度与阵列大小，测出质量约束下的可行区域；再加入实际宏的容量、通信与校准成本。若没有芯片测量，就明确这是架构敏感性研究，不能把仿真点包装成材料已经量产的证据。

面试问答与追问

1 存算融合现在主要在等材料突破吗?

要分路线。数字 SRAM CIM 不依赖新型存储材料，主要面临逻辑、密度、连线和系统调度；模拟非易失 CIM 更受状态稳定、写行为和可靠性影响，但 ADC、通信、容量、编译与算子覆盖同样关键。材料和电路算法应协同发展，能否落地取决于目标工作负载的整体收益。

继续追问

- 给一个通过电路改变适应器件的论文例子?
- 材料能量降十倍为什么系统收益可能很小?

2 耐久一百万次与保持十年分别说明什么?

耐久说的是在指定写循环与判据下能承受多少次更新；保持说的是指定状态、温度和误差要求下能维持多久。两者不能互换，长期保持可能来自加速模型外推。还要看读干扰、循环后保持、漂移及样本分布，不能只取标题数字。

继续追问

- 自然存放五年的未编程器件是否证明编程态保持五年?
- 推理和训练更关注哪些指标?

3 为什么单器件有 256 个状态不等于阵列有 8 位有效权重?

阵列中器件分布、编程误差、读噪声、漂移、温度、线阻与外围误差都占状态间距。还要确认所有状态能在所需可靠性和速度下区分，且映射后乘加精度足够。有效权重精度必须从完整链路和统计测量判断，不能从展示的电导级数直接推断。

继续追问

- 差分编码会增加哪些成本?
- 如何设计跨器件与跨时间的测试?

4 阵列占总能量 20%，材料让阵列能量降到 1/10，整体提升多少?

新总能量为 0.8+0.2/10=0.82，因此整体能效约提升 1.22 倍，总能量下降 18%。若器件改进同时减少外围成本，需重新分解；若新增写校验或读出开销，也要扣除。这个算式要求其他部分保持不变。

继续追问

- 如果 ADC 才是最大项，应优先研究什么?
- 为什么仍不能据此说材料研究不重要?

本章自测

1 哪项主要属于耐久而非保持?

A. 断电存放后状态维持多久

B. 反复写入后是否仍满足规范

C. 同一次读取的量化位宽

参考答案 B 耐久衡量循环承受能力；保持关注时间、温度下状态维持。

2 阵列能量占比 20%，降低 10 倍，其他不变，总能量还剩多少?

A. 10%

B. 82%

C. 20%

参考答案 B 80% 的其他能量未变，阵列剩 2%，总计 82%。

3 哪项无法仅凭材料电导更稳定自动解决?

A. 部分读误差

B. 校准频率需求

C. 跨宏通信拥塞和算子覆盖不足

参考答案 C 更稳定材料能缓解误差和校准，但网络、映射及算子支持仍需架构与软件设计。

原始资料

1. Compute in-Memory with Non-Volatile Elements for Neural Networks: A Review from a Co-Design Perspective · 2023 · Advanced Materials 35, 2204944
2. A crossbar array of magnetoresistive memory devices for in-memory computing · 2022 · Nature 601, 211–216; DOI 10.1038/s41586-021-04196-6
3. Long-term reliability of naturally aged hafnium oxide ferroelectric transistors for energy-efficient embedded memory · 2026 · Cell Reports Physical Science 7, 103057; DOI 10.1016/j.xcrp.2025.103057
4. A 64-core mixed-signal in-memory compute chip based on phase-change memory for deep neural network inference · 2023 · Nature Electronics 6, 680–693; public preprint 2022
5. Mixed-precision deep learning based on computational memory · 2020 · Frontiers in Neuroscience 14, 406; author manuscript
6. A 5-nm 254-TOPS/W 221-TOPS/mm² Fully-Digital Computing-in-Memory Macro Supporting Wide-Range DVFS and Simultaneous MAC and Write Operations · 2022 · ISSCC 2022, paper 11.6

论文训练营：从机制复述进阶到架构取舍

面试不需要背八篇摘要。每篇只抓六件事：原瓶颈、改变的硬件/软件、节省的数据路径、新增成本、证据类型和失败条件。下面的“反问自己”是原创面试练习；它们与论文实际结论分开，不能伪装成作者做过的实验。

学习目标

- 用90秒解释一篇论文及其适用边界
- 区分电路仿真、架构仿真、真实平台与厂商声明
- 对两篇相反结论做假设对齐
- 提出有消融、反例和基线约束的复现计划

01 | Ambit • MICRO 2017：物理原语能否变成系统收益

抓住机制：把共享位线的电荷多数判决变成整行逻辑操作，减少外部数据搬运；论文包括电路可靠性与系统性能建模，不是普通DIMM上默认支持的应用库。原论文：第3–5及评估章节

90秒表达：“它把数据路径缩到子阵列，但我会把专用行、输入复制、结果写回和布局约束放进成本模型。目标应是大量批量位运算；不是所有矩阵乘都合适。”

原创追问：你有一张很大的用户权限位图，要执行三次AND和一次OR。若中间结果留在计算行，可以少复制几次？若两张位图分散在不同bank，要搬的数据是否吃掉收益？设计消融时，保持输入布局不变，分别比较“仅计算快”“加入复制”“加入跨子阵列移动”三档，避免只展示理想原语延迟。

读图习惯：看到32倍一类聚合加速数字，先找比较的运算集合、基线和是否端到端。不要把批量位逻辑收益变成“CPU所有任务快32倍”。

02 | SIMDRAM • ASPLOS 2021：通用性与效率要分开

抓住机制：从逻辑表示、计算行分配到命令序列执行，构造DRAM内bit-serial运算；其硬件方案用模拟评估，CPU/GPU比较并不使SIMDRAM变成硅实测。原论文：框架与第6–7节

原创练习：假设一次数据转置花6ms，本地PIM每个任务1ms，GPU每个任务3ms。若数据可直接复用k次，PIM为6+k，GPU为3k；只有k>3时PIM在这个简化模型下更快。若每次输入都变化、要重新转置，则无法沿用该结论。

用这个例子解释“数据驻留和复用”如何决定机制落地，而非只复述论文运算吞吐。再问位宽从8bit到32bit时，操作步骤、临时行容量和转置成本如何变；乘法、比较和bitcount不该强行套同一个缩放系数。

面试加分句： “支持一个指令只证明表达能力；我要分别给出指令复杂度、数据变换和被调用次数，才能判断实际加速。”

03 | Samsung HBM-PIM • ISCA 2021：接口兼容不是软件零成本

证据说明：会议论文书目与摘要已核实，但全文访问受限。本节按官方公告和官方开源PIMSimulator讲机制，不把模拟器运行当成论文实片复现。IEEE论文入口；Samsung官方实现

可从公开实现观察bank附近计算单元、寄存器和内存命令驱动执行。面试真正该分析的是：既然复用内存接口，host如何配置操作、分发输入、等待完成和读回结果？数据在哪些bank？同一条广播命令怎样让不同bank处理不同数据？

原创设计题：把矩阵按输出行分片到bank，则输入向量需要广播，各bank可独立产生部分输出；按输入列分片则各bank产生partial sum，随后需要归约。前者可能复制输入，后者可能增加归约。矩阵维度、bank数量和局部存储决定更合适的分法。

回答时同时指出CPU/GPU缓存可见性、内存区域管理与库调度的成本。所谓“沿用现有接口”首先降低集成门槛，不能等价于所有应用不调整也自动获益。

04 | UPMEM实验研究 • 2021：读懂真实硬件的负结果

抓住证据：这项工作在真实UPMEM系统上做微基准和应用测量，揭示计算能力与跨DPU通信限制。它适合训练你在硬件已经存在时，如何寻找可用边界。原论文：系统组织与Key Takeaways

原创分析：给两个任务：A逐元素过滤后返回少量命中；B每一步都需要全局shuffle。A有机会让每个DPU在本地大数据上筛选，只回小结果；B即使局部计算很快，也可能不断经host交换。你要画出每轮通信字节数，而不是只数DPU数量。

再加入一个新约束：A的选择率从1%升到90%。返回结果大小暴涨，PIM的通信优势可能变弱；如果过滤谓词变成复杂浮点函数，本地计算也可能成为瓶颈。这两次变量变化，展示了*同一个算法名不等于同一种工作负载*。

复现规则：分别报数据分发、kernel和结果收集时间；标明DPU数量、频率、容量、输入大小和CPU线程数。旧硬件上的相对收益不能直接平移到2026 GPU。

05 | LLM in a flash • ACL 2024：最值得迁移的不是“Flash”这个词

抓住机制：普通Flash存权重，按需带入DRAM；windowing复用近期神经元，row-column bundling改善连续读取。它研究软硬件协同加载，既不是FlashAttention，也不是HBF量产验证。原论文：第2–4节

原创练习：方案A随机读2GB、有效速度1GB/s；方案B连续读3GB、有效速度3GB/s。B多读50%却只需1秒，A要2秒。下一步不要立刻宣布B最优：如果缓存容量只剩2GB怎么办？如果多读部分很快被重用呢？把时延、流量和驻留容量一起优化。

讨论论文适用性时，问模型是否存在可预测激活稀疏、预测器开销及漏预测精度损失；对不具备相同稀疏性/激活模式的模型，机制不保证直接复现同样收益。

迁移到HBF的表达： “我借鉴它减少有效搬运和匹配访问粒度的思路，但重新标定HBF延迟、并行度和页大小。不能用普通SSD的最优chunk size直接指定HBF数据布局。”

06 | FLINT • 2026预印本：围绕只读权重设计控制面

抓住机制：动态合并小请求、利用Flash已有buffer、用冗余plane隐藏维护，再简化只读权重映射；评价采用trace-driven模拟。原论文：第4–6节

原创消融：保持GPU、总容量和权重布局相同，依次关掉请求合并、双buffer重叠、后台维护隔离。观察到底是有效读放大下降、总线空泡减少，还是P99停顿下降。只给总token/s看不出三者贡献。

再给出失败条件：同一page中请求不能合并，MoE路由使使用顺序变化，模型权重需要在线更新，或维护量超过冗余资源的吞吐。设计只读FTL可减少复杂度，但它是对工作负载的契约；换成通用显存后必须补回相关责任。

评价数字时问：与HBM-only相比是否改变GPU数和跨卡同步？单卡装下模型的收益，不能全部归功于介质读取更快；基线因容量被迫多卡，是另一种架构收益来源。回答应把容量收益与单访问性能拆开。

07 | KV-Centric HBF • 2026预印本：识别“更快介质，系统更慢”

抓住结论边界：研究用扩展TokenSim和trace测试特定KV卸载栈，完整HBF组织可能失利；比较还改变近端内存资源，作者明确未证明HBF介质孤立地输给SSD。原论文：方法、第7节及局限

原创推演：HBM缓存把最热prefix留下，远端层收到的恰好是很冷的尾部。总系统prefix命中率很高，HBF层自身却可能“一写几乎不读”。因此不能用整体命中率去论证Flash写入值得；要测每层conditional hit和reads-per-write。

面试官问“加速远端读有什么坏处”，可答它本身可能没有坏处，但实际方案往往同时花掉封装位置、HBM带宽、写功耗及软件预算。使用Amdahl检查：若远端读只占总时间5%，即使变成零，理想加速也仅1/0.95≈1.053。

最好的下一步实验不是争论HBF好坏，而是固定HBM资源、仅更改远端介质，再单独评估封装替换成本，分离因果。

08 | Characterizing HBF • 2026预印本：吞吐、TPOT与寿命一起看

抓住方法：GPU kernel测量用于校准系统模型，HBF部分仍为外推；研究用分层放置和带缓存余量的调度探索高吞吐服务。明确的局限包括未完整建模商业HBF控制器与全部封装功耗，延迟受限小batch不在其主要范围。原论文：方法及5.5节

原创评审练习：方案A一小时完成100万token，但某用户P99 TPOT=200ms；B完成80万token，P99=40ms。若SLO为50ms，A不一定是好方案。请区分原始throughput、goodput、TTFT、TPOT与总完成时间，并给出相同请求到达轨迹下的结果。

寿命估算同样要注明P/E、均匀磨损和WAF前提。论文的理想假设可以作为可行性边界，不能改写成供应商承诺。把“WAF从1升到3”“维护写翻倍”“冷数据命中下降”列为敏感性实验，更能说明你理解存储工程。

统一八篇的回答骨架：先解释数据为什么在移动，再解释哪段可以避免，接着列新增成本，最后说明验证方法与失败区间。不要用论文数量替代因果链。

面试问答与追问

1 给你90秒讲一篇论文，应怎么组织?

用六句：具体负载和瓶颈；改变的位置或机制；减少哪段数据移动；付出什么面积/精度/布局/软件代价；实验是仿真还是实测及基线；在哪个条件下不成立。最后提出一个你会做的消融。

继续追问

- 请用FLINT示范，不背摘要。

2 PIM原语快10倍，为何应用只快1.2倍?

可能原语覆盖率低，复制/转置/同步没有变快，或瓶颈转移到结果汇总。先用Amdahl估计理论上限，再分解端到端时间；单个kernel不能代表应用。

继续追问

- 若只有20%时间可加速10倍，上限是多少？约1.22倍。

3 真实GPU上运行的HBF模拟器，是不是比纯模拟一定可信?

它保留了一部分真实计算与异步行为，但HBF仍依赖时序、温度、容量与访问覆盖模型。应检查注册访问比例、校准设备是否真HBF、注入开销以及外推范围，不能凭“真实GPU”四字跳过验证。

继续追问

- HBFSim用什么介质校准?
- 快模拟路径与参考路径如何对齐?

4 如何公平比较HBM-only和HBM+HBF?

先说明比较约束：同GPU数、同封装面积、同容量、同成本并不同时成立。固定工作负载/SLO，逐项报告资源与能耗；把容量导致的少卡收益、介质服务性能和软件调度收益分开。

继续追问

- 如果一组8卡而另一组1卡，怎么解释吞吐和成本？

5 没有前沿项目经历，怎样讲这些论文不显得背书？

明确说这是论文学习和自主分析，不冒充真实研发经历。拿一个可复算的容量/带宽模型、一个反例、一个小实验设计讲完整；将自己GPU/架构背景连接到队列、数据布局、并行度和可验证取舍。

继续追问

- 你能在两天内做哪个最小可验证实验？

本章自测

1 论文模拟新增容量让模型从多卡变成单卡，吞吐变好，可以直接归因HBF延迟更低吗？

A. 可以

B. 不可以，应分离容量和跨卡通信因素

参考答案 B 装得下带来的通信减少与介质单次访问速度是不同因果路径。

2 转置6ms，PIM每次1ms，GPU每次3ms，输入布局可复用，至少做几次才严格更快？

A. 2次

B. 3次

C. 4次

参考答案 C 6+k<3k要求k>3，4次时10ms<12ms。

3 论文使用100k P/E、WAF=1估寿命，正确引用方式是？

A. 该HBF产品保证100k P/E

B. 在论文假设下的外推寿命，需要对真实器件重新标定

C. WAF不会影响寿命

参考答案 B 模拟假设与厂商保证不同，P/E、WAF、温度和磨损不均都影响寿命。

原始资料

1. FLINT: Efficiently Leveraging High Bandwidth Flash for Capacity-Scalable LLM Inference Acceleration · 2026 · arXiv:2608.25062 v1，2026-08-25预印本
2. LLM in a flash: Efficient Large Language Model Inference with Limited Memory · 2024 · ACL 2024 Long Papers
3. Ambit: In-Memory Accelerator for Bulk Bitwise Operations Using Commodity DRAM Technology · 2017 · MICRO 2017
4. SIMDRAM: An End-to-End Framework for Bit-Serial SIMD Computing in DRAM · 2021 · ASPLOS 2021

5. Benchmarking a New Paradigm: An Experimental Analysis of a Real Processing-in-Memory Architecture · 2021 · arXiv:2105.03814；UPMEM真实系统研究
6. Samsung Develops Industry’s First High Bandwidth Memory with AI Processing Power · 2021 · Samsung 2021-02-17官方公告
7. Samsung SAIT PIMSimulator · 2021 · Samsung Advanced Institute of Technology官方开源实现
8. Hardware Architecture and Software Stack for PIM Based on Commercial DRAM Technology: Industrial Product · 2021 · ISCA 2021 Industrial Product
9. A Full-Stack Characterization of High-Bandwidth Flash for KV-Centric LLM Serving · 2026 · arXiv:2608.11668 v1，2026-08-12预印本
10. Characterizing High Bandwidth Flash for LLM Serving · 2026 · arXiv:2609.39131 v1，2026-09-30预印本
11. HBFSim: Fast and Faithful Simulation of High-Bandwidth Flash Under Real GPU Execution · 2026 · arXiv:2609.09800 v1，2026-09-09预印本

模块 21 · 深入研读

架构案例：给 70B 推理设计一份能被评审的存储规格

这是一道完整的教学产品定义题。所有器件容量、带宽、效率和预算都是明确假设，不对应某款真实芯片。目标是练习从服务指标推导存储组织，并主动指出模型的适用范围。

学习目标

- 独立推导权重与 KV cache 容量
- 区分 prefill、decode、每用户延迟与总吞吐
- 给出容量/带宽/功耗/质量多约束方案和验收方法

1. 先定义负载和 SLA，再选存储

教学需求：一个稠密 70B 参数模型，BF16 权重，每个参数 2 B；80 层，8 个 KV head，head dimension=128，KV 用 BF16；batch=8，每请求当前上下文 8192 token。目标稳态每用户 30 token/s，即一次 decode step 同时为 8 个用户各生成一个 token，step 约不超过 33.3 ms，总吞吐 240 token/s。暂不计多卡通信。

BF16 是数值格式，不代表硬件一定以相同速度处理所有算子。80 层和 8 个 KV head 是本题配置，不能从“70B”三个字符自动推导。GQA 的多个 query head 共享较少 K/V head，因此 KV 容量应使用 KV head 数。GQA 原论文，§2

真实需求还要补：最大上下文、输入/输出长度分布、并发变化、TTFT（首 token 时间）、ITL/TPOT（生成阶段 token 间隔）、p99、质量阈值与功耗。平均 240 token/s 不等于每个用户都达到 30 token/s。

2. 权重和 KV：把每个因子写出来

权重容量 `Cw=P×bytes_per_parameter`。$70\times10^{9}\times2=140$ GB≈130.4 GiB。若理想 INT8 为 70 GB、理想 INT4 为 35 GB；实际还要计 scale、zero point、padding、可能保持高精度的层等，不能把裸位数当部署文件大小。

普通 MHA/GQA 的 KV 容量近似为 `Ckv=2×L×B×S×Hkv×D×bytes`。前面的 2 是 K 和 V。按本题：2×80×8×8192×8×128×2=21,474,836,480 B=20 GiB≈21.47 GB。每请求每 token 为 320 KiB，每请求 8192 token 为 2.5 GiB。

若把 KV head 从 8 改成 64，其它不变，KV 就变成 160 GiB；它不是“同一个模型免费开关”，模型结构/质量也要处理。若上下文翻倍，KV 容量近似翻倍；权重不随 batch 或上下文同步增长。

容量预算假设：另留 16 GB 给 workspace、运行时和暂定余量，则当前需求为 140+21.47+16=177.47 GB。若设计容量 192 GB，剩约 14.53 GB；它还不是可接受的最大服务规模，长尾上下文、动态 batch 和碎片需要再验算。PagedAttention 可以减少分配浪费，但不会改变每个有效 KV 元素本身的字节数。
PagedAttention，§3–4

3. Decode 带宽：一次权重读取服务多少 token

一阶模型：每 step 所有权重从外存读一次并在 batch 内共享；完整 dense attention 读取当前所需 KV 一次；先忽略激活、小张量、写回、重复读取和通信。则每 step 数据约为 140+21.47=161.47 GB。达到 33.3 ms，持续有效带宽至少约 4.84 TB/s。这是乐观下限，不能直接承诺 SLA。

若已知原始带宽到有效业务流量的效率仅 75%，原始接口至少约 6.46 TB/s，并且仍无计算、串行与 p99 余量。把 161.47 GB 除以 batch=8，可得每个输出 token 平均分摊约 20.18 GB；绝不能把每用户 token/s 与全模型每 step 流量混乘。

batch 增大能摊薄每 token 权重读取，但 KV 容量和 KV 总读流量也增长。长上下文、低位权重、大 batch 下，KV 占比可能显著上升。只有在测到负载的真实外存字节后，才能修正这个一阶模型。

4. Prefill 与 decode：同一模型，瓶颈会换

Prefill 一次处理较多输入 token，可形成较大的矩阵计算，提高权重复用；长序列 attention 也有大量计算与 I/O。Decode 每序列通常只前进一步，batch 小时更像低复用矩阵向量计算，并反复读取权重/KV。它们“常见的倾向”不同，不能不看形状就断言所有 prefill 算力受限、所有 decode 带宽受限。

教学计算：用稠密网络粗略估计每 step 主要线性层运算为 2PB=1.12×10^{12} FLOP；若这类算子的有效算力为 200 TFLOP/s，计算时间量级是 5.6 ms。该估算未含 attention 的上下文相关运算、激活、量化与通信，不是完整模型 FLOP 精确值。

理想可重叠时下界接近 `max(Tcompute,Tmemory,Tcommunication)`；实际依赖链和重叠不足会增加时间，不能无条件把各项相加，也不能无条件只取最大。FlashAttention 主要减少 attention 中间流量，不能免掉模型权重或 decode 所需 KV 的全部读取。FlashAttention Roofline

5. 比较三种方案：每个结论带一个成立条件

以下堆栈配置纯为教学假设，单堆栈原始带宽均设为 1.6 TB/s，效率均先设 75%。

方案	总容量	有效带宽	本题数据搬运下界	判断
A：4×48 GB，BF16 权重	192 GB	4.8 TB/s	161.47/4800≈33.64 ms	仅搬运已超目标，没有实现余量
B：6×32 GB，BF16 权重	192 GB	7.2 TB/s	约 22.43 ms	容量相同，接口并行更多；需付封装/PHY/功耗代价
C：4×48 GB，理想 INT8 权重	192 GB	4.8 TB/s	(70+21.47)/4800≈19.06 ms	可能减流量；质量、元数据与量化 kernel 需验证

不能只选最短理论时间。B 需要更多物理资源；C 需要软件和模型条件成立。若用 HBF 增容量却替换部分 HBM，还应重算剩余热层带宽；若全部权重无法常驻热层，则搬运过程加入关键路径，容量增加也可能导致延迟下降不了。

设计取舍应以“满足约束的最小成本方案”为目标，比较基线、改动、收益、代价、失效场景，而不是让某一个指标最大。

6. 功耗、面积、可靠性：不知道的量要列成待验证项

教学能量假设：若接口动态能耗为 2 pJ/bit，方案 B 按有效数据 7.2 TB/s 折算为 115.2 W；按 9.6 TB/s 满速传输折算为 153.6 W。实际能量口径须包含无用传输与编码等，并另计阵列、refresh、静态、控制器和封装。这里不能推导真实某代 HBM 功耗。

面积至少分成 DRAM die、加速器上的控制器/PHY、封装占用；它们不是同一个 mm^2。没有版图、工艺或经过校准的模型时，正确做法是列出资源需求和模型输入，不编造面积结果。

验收还要包含温度下的速率/刷新变化、错误上报、ECC 覆盖、坏资源降级、数据完整性与长期可靠性。容量“装得下”、短时“跑得快”只是两项条件。

7. 一页面试交付：让方案可以被否证

1. **需求表：**模型/数值格式、batch/上下文分布、p99 TTFT/ITL、总吞吐、功耗及成本上限。
2. **数据表：**权重、KV、临时状态、元数据分别计容量、读写量、复用、寿命。
3. **架构表：**存储类型、堆栈/通道、数据放置、映射、调度、缓冲与错误处理。
4. **模型：**容量预算、带宽下界、计算下界、重叠假设、敏感性区间。
5. **实验：**microbenchmark 校准带宽与延迟，再跑真实长度分布、混合 prefill/decode、长尾和高温场景。

6. **决策：**说明何种证据支持 B 或 C，什么结果会推翻它；保留回退配置。

可复述回答：“我先明确 batch 与上下文，再得到 177.47 GB 教学容量预算和约 4.84 TB/s 有效带宽下限。四堆栈基线在假设效率下没有时延余量，所以我比较更多并行接口与权重量化，并把质量、封装功耗及 p99 验证作为决策条件。”这是纸面分析示例，未运行实验时不要称为已经实现的产品。

面试问答与追问

1 70B BF16 模型为什么不等于只需要 140 GB 内存?

140 GB 只是按 70×10^9 参数、2 B/参数计算的纯权重。还要加 KV cache、workspace、框架与元数据、碎片、通信和服务余量。KV 还随 batch、上下文、KV head 数和数值格式变化，应单独逐因子计算。

继续追问

- GB 与 GiB 差多少?
- GQA 为什么改变 KV 预算?

2 batch 从 1 提到 8，吞吐是否一定提高 8 倍?

不会自动线性提高。权重可在 batch 内复用，但 KV 流量和计算增加，容量可能限制批量；调度等待、padding、kernel 效率和 p99 也会变化。应比较每 step 时间和每秒输出 token，而不是只看 batch 数。

继续追问

- 哪一项流量可摊薄，哪一项难以摊薄?
- 在线 SLA 下如何选择 batch?

3 方案 B 理论搬运 22.43 ms，能否承诺 30 token/s?

不能。这只是简化流量与固定效率下的下界；尚未完整计计算、额外流量、通信、调度以及尾延迟。它说明有继续验证的空间，应通过实际流量和端到端时序校准，而不是把下界当测量值。

继续追问

- 理想重叠时为何取 max?
- 哪些串行环节会破坏重叠?

4 客户说“加 HBF 就能解决容量墙”，你怎么推进?

先问哪些数据放到 HBF、是否保留足够 HBM 热层、迁移粒度与预取可预测性、写入与寿命要求。然后把容量收益和新增关键路径 I/O 同时建模，以 p99 goodput、能耗和成本评估，而不是只验证模型能装入。

继续追问

- 替换一个 HBM stack 会损失什么?

- 什么工作负载最不适合依赖预取?

本章自测

1 本题每请求每 token 的 KV 大小为何是 320 KiB?

A. 70B×2 B

B. 2×80×8×128×2 B

C. 8×8192 B

参考答案 B 逐项是 K/V 两份、层数、KV head、head dimension 和每元素字节数；batch/context 尚未乘入。

2 若模型权重不变，上下文翻倍，哪项一阶变化正确?

A. 权重与 KV 都翻倍

B. 权重不变，KV 容量近似翻倍

C. 只有权重翻倍

参考答案 B 普通 dense MHA/GQA 的 KV 元素数随序列长度增长，权重是模型常驻参数。

3 本题每 step 为 batch=8 各生成一 token，step=40 ms，总吞吐是多少?

A. 25 token/s

B. 200 token/s

C. 320 token/s

参考答案 B 8/0.04=200 token/s，总吞吐；每用户则为 25 token/s。

原始资料

1. Roofline: An Insightful Visual Performance Model for Floating-Point Programs and Multicore Architectures · 2008 · UC Berkeley 技术报告 UCB/EECS-2008-134；2009 CACM 工作的原始报告
2. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness · 2022 · NeurIPS 2022
3. Efficient Memory Management for Large Language Model Serving with PagedAttention · 2023 · SOSP 2023
4. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints · 2023 · EMNLP 2023
5. FlexGen: High-Throughput Generative Inference of Large Language Models with a Single GPU · 2023 · ICML 2023
6. Sandisk HBF Fact Sheet · 2025 · Sandisk 2025-07 技术简介

面试实验室：三个能诚实复现、能讲设计取舍的小项目

这里提供的是可执行的实验设计与复现路线，不是已经完成的实验结果。你可以先把它作为论文阅读和方案分析讲清楚，再实际运行、记录和复核，逐步升级为可展示项目。面试最有分量的是可追溯证据和失败解释。

学习目标

- 设计有对照、可复现的 DRAM 调度实验
- 用 GPU 微基准分离合并访问、缓存、MLP 影响
- 建立 HBM/HBF 参数化模型并解释其有效边界

1. 项目表述分三档：读过、复现过、改进过

阅读分析： “我读了原论文，复核了容量/带宽推导，设计了待运行的对照实验。” **完成复现：** “我在指定 commit、配置与机器上跑了基线，保留了输入、日志和结果。” **验证改进：** “我修改某机制，在固定条件下得到差异，并分析统计波动、开销和失效场景。” 没有做到哪一档，就不提前使用那一档的动词。

每个项目建立相同证据包：README、环境/版本、固定随机种子、输入生成脚本、配置、原始日志、汇总 CSV、作图脚本、已知限制。图上写清单位、误差与样本。论文的收益数字只能标“论文报告”，不得替代自己的测量。

建议先完成项目一的单 bank/双 bank 诊断，再做项目二；项目三适合把你现有架构能力与 HBF 前沿连接起来。三者追求“能解释一张图”，而不是堆很多未经校准的曲线。

2. 项目一：用 Ramulator 研究“吞吐优先”何时伤害延迟

问题：FR-FCFS、朴素按年龄选择、带年龄保护的策略，在相同 DRAM 与流量下如何改变吞吐和公平性？以 Ramulator 2.0 论文理解模块化设计；本次查到的官方仓库已是 2.1。下面固定的是本轮核验的 2.1 提交，属于复现起点，尚未在此材料中编译或跑通。

```
git clone https://github.com/CMU-SAFARI/ramulator2.git
cd ramulator2
git checkout c6c7a5de0cef35935d6fad7369e6271406798865
git rev-parse HEAD
```

在 Linux/WSL 或仓库支持的开发容器中，按固定版本 README 安装 C++20、CMake、Python 依赖，先运行官方 example 与对应时序/调度测试。入口是 `examples/example_config.py`；测试包括 `tests/device_timings` 和 `tests/controller_scheduling`。不要把旧 2.0 YAML 教程与新 2.1 Python API 混用。固定提交与 README 2.0 论文

输入：固定一个 DDR4 preset 作为教学基线，保持组织、速率、映射、队列和刷新不变。先构造已知地址向量，验证同一打开行、同 bank 不同行、不同 bank 三类序列；再生成连续流、均匀随机、bank 热点以及“行命中流 + 延迟敏感流”。随机种子设 1、2、3；每组预热与正式请求数明确记录。

3. 项目一如何形成可信结果

变量：先只换调度器；再单独扫描注入间隔、读写比、队列深度和年龄阈值。不存在的策略要明确自己实现，并保持同地址依赖与命令合法性。不要同时改映射、频率和调度，然后把收益都归给调度。

输出：完成字节/模拟时间、平均/p95/p99 请求延迟、每来源服务量与等待上界、row hit/conflict、ACT/PRE 次数、队列占用。若需要 p99 或来源统计而默认计数器没有，就记录请求到达/完成时间及 source ID。只统计已完成请求时，还应报告未完成/被截断请求，避免把最慢请求排除。

关键对照：低负载与接近饱和分别画 latency-throughput 曲线；双流与各流单独运行比较。请求流必须反映注入和反压，固定 trace 的离线重放不能自动代表应用 IPC；需要系统收益时再接有依赖/计算间隔的 frontend。重排基础 公平性案例

失败解释：策略几乎无差异，可能是负载太轻、全是行命中或总线已饱和；行命中提升但吞吐下降，可能是 bank 并行被牺牲；平均延迟改善但 p99 变差，可能有偏置/饥饿。先检查地址与命令 trace，再谈架构结论。

4. 项目二：GPU 访存微基准，把“快慢”拆成可测因素

问题：保持有效读取工作量时，stride、偏移、工作集、并发如何影响有效字节/s 和实际事务？使用自己有权限的 CUDA GPU，记录 GPU 型号、驱动、CUDA、编译选项、频率/功耗状态和 profiler 版本；没有 GPU 时先做代码与预测，不报硬件结果。

```
// 示意核心；输入至少覆盖 (n-1)*stride+offset+1 个元素
__global__ void gather(const float* in, float* out,
                       size_t n, size_t stride, size_t offset) {
    size_t i = blockIdx.x * (size_t)blockDim.x + threadIdx.x;
    if (i < n) out[i] = in[i * stride + offset];
}
```

输入和对照：stride=1/2/4/8/16/32，offset=0/1/2/4 个 float；另做小于与显著大于 L2 的工作集扫描。固定有效读取数时 stride 会改变总覆盖空间，要另设固定覆盖空间的对照并说明两种口径。比较连续流与单依赖链/多独立链，分离带宽与 MLP。数组先初始化，保留输出并检查正确性，避免编译器删去工作。

测量：预热后用 CUDA event 计 kernel 区间，重复多次报告中位数和分位数，单独记录主机计时；copy kernel 的有效读写字节按实际操作计算。收集 DRAM bytes、L2 命中/流量、sector/request、stall 等可用指标，再按当前 GPU 文档解释。冷/热缓存实验分开；Nsight replay 默认清缓存，不能与自然热缓存运行混为一谈。CUDA 合并访问 Nsight 测量条件

失败解释：带宽“超过 HBM 峰值”先查是否命中 L2、字节分母是否重复计数；stride 没明显影响先查缓存复用或其它瓶颈；小 kernel 波动大先查启动/计时与动态频率。不能仅凭 stride 曲线反推出未公开的完整 bank 映射。

5. 项目三：HBM/HBF 两层预取模型，检验容量是否能转成吞吐

问题：在固定封装资源/成本情景下，保留多少 HBM 热层、将什么放到 HBF，才能满足 SLA？这是自行编写的 Python 离散事件或时间线模型，不是 HBF 实物测量，也不是标准认证模拟器。灵感来自分层放置与传输重叠；FlexGen 的对象是 GPU/CPU/磁盘，并未验证 HBF。FlexGen，§4

输入：按层的权重/KV块大小、访问次序、计算时间；HBM/HBF 容量、读写带宽、首数据延迟、并发服务额度、缓冲大小、预取距离、命中率和链路竞争。分别设全 HBM、仅按需迁移、固定距离预取、知道未来的理想预取四条基线。理想预取只作为上界。

教学假设：HBF 首数据延迟 20 µs、链路上限 1 TB/s。4 MiB 单块的串行完成约 20+4.19=24.19 µs，对应约 173 GB/s；4 KiB 单块串行只有约 0.205 GB/s。都远低于 1 TB/s，说明需要粒度和并行。按带宽—延迟积，仅覆盖 20 µs 延迟就需约 20 MB 在途，即至少约 5 个 4 MiB 请求；内部资源限制还可能需要更多或无法达到。

```
对每个访问块：
  若已驻留 HBM：排队等待 HBM 读服务，计入共享带宽与首数据延迟
  否则：排队等 HBF 服务、链路、缓冲及 HBM 填充；记录数据可用时间
  消费开始 = max(依赖完成，数据已到达计算侧可消费的位置)
  预取下一批可预测块；按容量执行驱逐
  记录 stall、搬运字节、无用预取与写入量
```

HBM 命中只表示不必从冷层取回，不代表数据能零成本到达计算单元。权重读、KV 读写与冷层填充若共享 HBM 端口，要在同一资源队列计费；可重叠的是独立资源上的操作。如果 Tcompute 已包含经校准的 HBM 服务，则应写清统计边界并避免重复计数。

输出：每 step 时间、p99、满足 SLA 的 goodput、HBM 占用、迁移放大、预取准确率/覆盖率、HBF 读写量及能耗模型范围。扫顺序读取、随机专家选择、长尾上下文和混合写入。预取失败应能归因于地址不可知、队列拥塞、热层太小、带宽不足或无用搬运，而不是一律再加缓冲。

6. 每张图配一段“为什么”，不能只说提高了多少

统一回答模板：**目标与约束 → 机制假设 → 对照与指标 → 结果/目前状态 → 代价 → 失效条件 → 下一步证据。**没有跑结果时，把“结果”明确替换为“预计可区分的现象”，例如：“若 bank 并行是瓶颈，分散地址应减少服务等待；若总线已饱和，收益应变小。我准备通过命令占用和完成吞吐验证。”

可复述的阅读版自我介绍：“我原有 GPU/SoC 架构基础，近期重点补齐了 DRAM 的 row-buffer 与时序约束。我围绕控制器公平性、GPU 事务效率和 HBM/HBF 数据放置设计了三组对照，当前完成了文献和一阶模型；尚未完成的硬件实验我不会报收益。下一步优先校准实际访问流量和 latency-throughput 曲线。”完成实验后，再替换成自己的具体配置和可追溯结论。

论文阅读顺序：Rixner 建立重排直觉；BLISS 学公平性；Ramulator 学怎么建模；Roofline 学怎么给上界；FlashAttention/PagedAttention 学不同层级的流量与容量优化；FlexGen 学分层放置的吞吐—延迟代价。

7. 一轮 20 分钟模拟面试

1. **3 分钟：**画一次 GPU load 到 DRAM 的路径，指出 coalescing、L2 hit、row hit 分别在哪。
2. **4 分钟：**给定行命中流和随机流，设计控制器 QoS；说清什么状态要加、何时会不如基线。
3. **5 分钟：**口算 70B 权重和 KV，区分总吞吐、用户 token/s、prefill 与 decode。
4. **4 分钟：**给一个 HBF 高带宽声明，说明还缺哪些数据才能判断是否适合这个模型。
5. **4 分钟：**面试官改变 batch、上下文或功耗约束，你现场更新判断，并提出最小验证实验。

优秀回答的标志不是名词最多，而是每个结论都能回答“因为什么、在什么条件下、怎么测”。遇到不知道的内部细节，可以明确公开资料边界，再说明可观测量与验证方法；这通常比编造具体 bank 位或芯片时序更专业。

面试问答与追问

1 你的研究是不是只读论文，没有真实项目？

按实际完成度回答。若目前只读和建模，就明确这是专项调研与实验设计，展示推导、对照和版本记录；如果已跑通，再展示原始日志与复现方法。阅读研究本身可以证明分析能力，但不能替代 RTL、硅后或硬件测量经历。

继续追问

- 你亲手完成了哪一部分？
- 哪个结果与你的预期相反？

2 你的新调度器平均吞吐提高 10%，是否就该采用？

先确认 10% 是自己的实测还是题设，然后看实验可比性、每流 slowdown、p99、饥饿与实现时序/面积成本。还要测低负载和未见工作负载，确认不是牺牲少数任务或改变了其它条件。

继续追问

- 如何检查统计和截断偏差？
- 若收益只来自一个负载怎么办？

3 HBF 模型预测很好，怎样避免模型只是证明自己的假设?

把结果拆成可校准的首数据延迟、粒度、并发、链路和计算重叠；同时跑按需与理想上界，给敏感性范围和不利工作负载。未知的器件参数标为假设，之后用公开测量或硬件数据校准。若只有理想预测有效，就不能宣称产品可行。

继续追问

- 固定封装面积下会减少多少 HBM?
- 随机访问和写入如何建模?

4 微基准带宽异常高，你先做什么?

先检查字节口径、计时区间、异步完成、编译器消除和数据正确性，再检查缓存命中与工作集。固定 profiler 的缓存/replay 设置，保留自然运行结果作对照。只有定位流量走到哪一级，带宽数字才有意义。

继续追问

- CUDA event 与主机计时分别包含什么?
- 怎样避免用 cache 测量冒充 DRAM 带宽?

本章自测

1 未运行的 HBF 参数模型在面试中最准确的名称是?

A. 已量产验证的 HBF 架构

B. 已取得硬件加速收益的项目

C. 待校准的架构分析与拟议实验

参考答案 C 将设计、模拟、硬件验证与产品化区分开，结论才可信。

2 比较两种调度器时，哪种实验最有说服力?

A. 同时改变频率、映射和调度器

B. 固定其它参数，扫描负载并报告每流和尾延迟

C. 只挑最好的一条 trace

参考答案 B 需要单变量对照、负载覆盖和多指标，才可解释因果及适用范围。

3 20 µs 首数据延迟的 4 KiB 串行读取，能否仅凭 1 TB/s 峰值达到 1 TB/s?

A. 可以，峰值就是任何请求的速率

B. 不可以；首数据与并发限制会主导

C. 把时钟乘 2 就一定可以

参考答案 B 单请求约 20 μs 才完成，4 KiB/20 μs 约 0.205 GB/s；需要并行与粒度才能接近聚合带宽。

原始资料

1. Memory Access Scheduling · 2000 · ISCA 2000
2. The Blacklisting Memory Scheduler: Achieving High Performance and Fairness at Low Cost · 2014 · ICCD 2014
3. Ramulator 2.0: A Modern, Modular, and Extensible DRAM Simulator · 2023 · 2023 arXiv 版本；IEEE Computer Architecture Letters
4. CMU-SAFARI/ramulator2 官方仓库 · 2026 · CMU SAFARI；2026-10-30 访问
5. CUDA Programming Guide：Writing SIMT Kernels · 2026 · NVIDIA 持续更新官方文档；2026-10-30 访问
6. Roofline: An Insightful Visual Performance Model for Floating-Point Programs and Multicore Architectures · 2008 · UC Berkeley 技术报告 UCB/EECS-2008-134；2009 CACM 工作的原始报告
7. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness · 2022 · NeurIPS 2022
8. Efficient Memory Management for Large Language Model Serving with PagedAttention · 2023 · SOSP 2023
9. FlexGen: High-Throughput Generative Inference of Large Language Models with a Single GPU · 2023 · ICML 2023
10. Nsight Compute Profiling Guide · 2026 · NVIDIA 持续更新官方文档；2026-10-30 访问
11. Sandisk HBF Fact Sheet · 2025 · Sandisk 2025-07 技术简介

论文案例比较与复述

以下卡片便于面试前回顾。机制解释与案例分析均为原创教学整理；完整实验配置和数值结论以链接原文为准。

A Case for Exploiting Subarray-Level Parallelism (SALP) in DRAM

2012 · ISCA · 原文链接

问题 Bank内不同Subarray的请求被保守串行化。

机制 逐级交叠局部PRE/ACT与写恢复；MASA将活动与全局输出选择解耦。

取舍 需要Subarray映射与协议/局部控制支持，外部总线仍共享。

证据边界 结构和时序分析、面积估计、系统模拟；未证明量产实现。

面试复述 SALP不是凭空增加DQ，而是释放Bank内部本来被共享控制掩盖的局部并行。

Tiered-Latency DRAM: A Low Latency and Low Cost DRAM Architecture

2013 · HPCA · 原文链接

问题 长位线有利于感放面积摊销，却增加感测与驱动负担。

机制 隔离晶体管把位线分成近段和远段，以数据管理利用小而快的近段。

取舍 远段时序、近段容量、迁移与替换开销；不能把近段收益推广到所有访问。

证据边界 电路级模型及系统模拟，非商用器件实测。

面试复述 TL-DRAM把密度和延迟矛盾转成数据放置问题，因此热点稳定性决定价值。

Adaptive-Latency DRAM: Optimizing DRAM Timing for the Common-Case

2015 · HPCA · 原文链接

问题 统一最坏情况时序使典型模块与温度下的余量未被利用。

机制 表征真实模块并根据运行条件适配部分时序参数。

取舍 需要条件监测、保守边界、覆盖测试及回退；不能超出器件保证外推。

证据边界 115个模块表征并有真实系统评估；结论限定于测试条件。

面试复述 AL-DRAM利用典型条件余量，我会追问如何持续证明当前条件仍在可靠边界内。

RAIDR: Retention-Aware Intelligent DRAM Refresh

2012 · ISCA · 原文链接

问题 统一刷新使保持能力强的行承担不必要维护。

机制 按行保持需求分箱，Bloom filter保存弱行集合，安排不同刷新频率。

取舍 依赖可靠profile；DPD/VRT、显式行恢复命令和管理成本需计入。

证据边界 模型与系统模拟，非直接量产功能；教学分箱例子与论文实参分开。

面试复述 RAIDR允许Bloom假阳性保守多刷，却不能让profile漏判或VRT风险自动消失。

An Experimental Study of Data Retention Behavior in Modern DRAM Devices

2013 · ISCA · 原文链接

问题 刷新优化需要可靠保持时间地图，但保持行为并不简单固定。

机制 通过温度受控平台与多种图样表征DPD和VRT。

取舍 更全面测试消耗时间，有限观测仍不能保证永不切换。

证据边界 248颗DDR3、五家厂商的真实器件研究。

面试复述 读RAIDR后必须读retention实测，因为可维护的正确profile才是难点。

RowClone: Fast and Energy-Efficient In-DRAM Bulk Data Copy and Initialization

2013 · MICRO · 原文链接

问题 大块复制和清零让数据往返外部内存通道。

机制 FPM在共享感放的行间复制，PSM使用内部共享数据路径。

取舍 布局与粒度限制、缓存一致性、权限和完成语义；需专门器件支持。

证据边界 架构提案、时延/能量建模和系统评估；原语加速不等于应用加速。

面试复述 RowClone的核心不是多造算力，而是让无需离开DRAM的数据别绕CPU一圈。

Flipping Bits in Memory Without Accessing Them: An Experimental Study of DRAM Disturbance Errors

2014 · ISCA · 原文链接

问题 重复激活可能破坏邻近行数据，违背地址隔离假设。

机制 真实器件与系统表征扰动，并讨论选择性/概率性恢复等防护。

取舍 防护状态、额外维护、性能及最坏情况保证；历史阈值不可通用。

证据边界 真实系统与FPGA表征；这是物理可靠性证据。

面试复述 RowHammer说明控制器的访问行为会影响其他地址的数据，因此可靠性需要跨层设计。

RowPress: Amplifying Read Disturbance in Modern DRAM Chips

2023 · ISCA · 原文链接

问题 仅用ACT计数描述读扰动可能遗漏开行时长因素。

机制 扫描行打开时间，测量不同条件下的扰动行为。

取舍 计数型防护和追逐row-hit策略需要考虑驻留时间与器件条件。

证据边界 真实DDR4芯片实验；原论文研究条件不能直接作产品安全阈值。

面试复述 RowPress让我在分析open-page策略时同时看激活次数和行打开时间。

Understanding and Modeling On-Die Error Correction in Modern DRAM: An Experimental Study Using Real Devices

2019 · DSN · 原文链接

问题 on-die ECC遮蔽原始错误，外部观测无法直接对应单元缺陷。

机制 用统计模型及真实设备实验推断内部纠错和纠错前错误性质。

取舍 依赖可验证统计假设，器件差异和未知ECC使解释复杂。

证据边界 真实LPDDR4数据结合统计推断与ECC模拟。

面试复述 外部零错误不等于内部零错误，先画ECC的保护和观测边界。

Design-Induced Latency Variation in Modern DRAM Chips

2017 · SIGMETRICS / POMACS · 原文链接

问题 器件内部不同位置需要不同可靠访问时间。

机制 识别设计布局引起的慢区域，研究有针对性的profiling和数据组织。

取舍 地址扰乱与冗余映射遮蔽物理位置；测试和适配需保证可靠性。

证据边界 96条DIMM的真实表征与后续机制评估，不能按逻辑地址线性猜测位置。

面试复述 DIVA提醒我DRAM不是延迟均匀的抽象数组，距离与外围布局会产生结构化差异。

Ramulator 2.0: A Modern, Modular, and Extensible DRAM Simulator

2024 · IEEE Computer Architecture Letters（2023预印本） · 原文链接

问题 研究新命令与控制器机制需要可扩展、可验证的模型。

机制 分离组织、命令、状态、约束与控制器策略，使用模块化接口。

取舍 模型配置与实现覆盖决定结论；不模拟的模拟电路行为不能由协议正确性背书。

证据边界 论文用Micron DDR4 Verilog模型检查命令轨迹等验证；本教程未声称亲自运行。 核读 §2；2023 预印本与当前 2.1 仓库需要分版本。本材料给了核验过的 2.1 提交作为新实验起点。

面试复述 我会用Ramulator验证调度效果，再用器件证据支持被改变的物理时序。 说明你修改了哪个模块、保持哪些参数不变，并用短 trace 与命令合法性检查证明模型没被改坏。

SoftMC: A Flexible and Practical Open-Source Infrastructure for Enabling Experimental DRAM Studies

2017 · HPCA · 原文链接

问题 普通CPU benchmark难精确控制DRAM命令与数据条件。

机制 FPGA控制器提供可编程命令与时序，配合真实模块进行实验。

取舍 硬件/PHY支持、温控、器件覆盖和测量可见性需要说明。

证据边界 真实器件实验基础设施，原论文还报告某些模型预期在实物不可观察。

面试复述 SoftMC的启示是：能在模型里说通的电荷假设，还需真实器件检验。

Memory Access Scheduling

2000 · ISCA · 原文链接

问题 同一 DRAM 的有效带宽随 bank/row/column 访问模式变化，简单按序服务会浪费可并行机会。

机制 观察未完成请求，按命令就绪、行状态与年龄重排访问；论文区分 first-ready 和更积极的列/行优先策略。

取舍 局部性与并行可能改善，但请求等待、顺序语义、控制逻辑和公平性需要另行处理。

证据边界 阅读重点为 §3 调度设计空间、§5 不同调度比较。论文使用当时媒体负载，不把其百分比当现代 GPU 预测。

面试复述 先给出“同 bank 换行”和“另一 bank 已就绪”的小例子，再解释重排为什么能填补空闲。

The Blacklisting Memory Scheduler

2014 · ICCD · 原文链接

问题 行命中和高强度流可能持续占用共享内存，让其它应用被不公平地拖慢。

机制 以连续服务计数把来源暂分为两组，优先非黑名单请求，再看行命中和年龄，周期清除分类。

取舍 用较小状态换取公平性，但阈值、恢复周期和来源粒度需要适配工作负载。

证据边界 核读 §III–V，包括优先级和硬件存储项。论文包含性能、公平性和实现复杂度比较；这里不复用其阈值为最佳值。

面试复述 回答“行命中最高是不是最好”时，主动补每来源 slowdown、p99 与硬件仲裁开销。

Improving DRAM Performance by Parallelizing Refreshes with Accesses

2014 · HPCA · 原文链接

问题 刷新会占用 bank/rank，固定刷新次序可能与业务访问冲突。

机制 DARP 寻找空闲 bank 与写回时段；SARP 借助子阵列级内部支持，使刷新和访问进一步重叠。

取舍 调度灵活性受保持/刷新规则约束，子阵列并行涉及 DRAM 内部改动和共享资源限制。

证据边界 核读 DARP/SARP 机制与论文对标准背景的说明；其 2014 年器件背景不能代替今天标准原文。

面试复述 将提案拆成控制器可改变部分和需要 DRAM 电路/协议支持部分，明确验证不漏刷新。

Roofline: An Insightful Visual Performance Model

2008 · UC Berkeley 原始技术报告；后续 CACM 2009 · 原文链接

问题 只看 FLOPS 或时钟无法判断程序为什么慢，也难以决定优化计算还是数据搬运。

机制 选定层级，用 FLOP/byte 与持续带宽构造斜线，用对应计算峰值构造水平上限。

取舍 模型易解释但属于上界；实际依赖、并发、调度和多层缓存需要进一步分析。

证据边界 核读 operational intensity 定义、公式及 L2 roofline 讨论。应对齐运算精度、数据口径和带宽层级。

面试复述 先写 min(P,B×I)，再说明观测点为何可能远在屋顶下方，以及要补哪组计数器。

FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness

2022 · NeurIPS · 原文链接

问题 注意力中间矩阵在 HBM 与片上存储之间反复搬运，FLOP 少不一定执行快。

机制 分块、在线 softmax 统计、融合和反向重算，避免完整中间矩阵的外存物化。

取舍 片上容量、tile、寄存器和并发互相约束；保持标准 attention 不代表消除全部 KV 或线性化算术复杂度。

证据边界 核读 §3，特别是 tiling/recomputation。论文测量来自特定模型/硬件，不以最高加速比代替用户实验。

面试复述 用“多做部分计算，少搬大量字节”解释取舍，再区分 SRAM/HBM 之间的 I/O 与 DRAM 芯片内部时序。

Efficient Memory Management for Large Language Model Serving with PagedAttention

2023 · SOSP · 原文链接

问题 KV 随序列增长，连续预留和碎片浪费容量，限制可服务并发。

机制 逻辑 KV 块映射到不连续物理块，按需分配，并支持块共享等机制。

取舍 块小降低碎片但可能增加管理/访问开销；块大利于部分执行效率却增加浪费。

证据边界 核读 §3–4、§7 的块尺寸讨论。它优化管理与服务效率，不是自动压缩每个 KV 元素，也不是 DRAM row policy。

面试复述 将“容量利用率改善可容纳更多 batch”与“单条请求变快”分开说明。

GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints

2023 · EMNLP · 原文链接

问题 多头 attention 的 K/V 状态和 decode 读取有显著容量与带宽代价。

机制 多个 query head 分组共享较少 K/V head，在 MHA 与 MQA 之间选择结构。

取舍 KV 减少需要结合模型质量、训练/适配与 kernel 实现评估，不是存储控制器能独自改变的开关。

证据边界 核读 §2 的 head 分组与端点定义；容量推导使用 Hkv 而非 Hquery。

面试复述 现场写出 2×层数×batch×长度×KV heads×head dim×字节数，解释每个因子。

FlexGen: High-Throughput Generative Inference of Large Language Models with a Single GPU

2023 · ICML · 原文链接

问题 GPU 容量有限，大模型需要把数据分布在更慢、更大的层级中。

机制 联合选择张量放置与层/批次执行顺序，并将未来数据传输与当前计算重叠。

取舍 以可接受更长延迟的离线吞吐为目标；更大 batch、更多搬运和复杂调度未必满足在线 p99。

证据边界 核读 §4 放置、遍历和重叠。该论文研究 GPU/CPU/磁盘，不是 HBF 器件或系统实测。

面试复述 借其思路设计 HBM/HBF 模型，同时明确新介质的延迟、粒度、写入和封装约束必须另行校准。

Ambit: In-Memory Accelerator for Bulk Bitwise Operations Using Commodity DRAM Technology

2017 · MICRO 2017 · 原文链接

问题 大位向量运算在外部处理器与DRAM之间搬运，受外部带宽限制。

机制 借三行激活的多数函数与NOT原语组合bulk bitwise操作。

取舍 专用行、复制和局部性约束；不是所有现成DRAM可直接执行。

证据边界 电路与架构层评估，不能当量产芯片实测。

面试复述 若位图跨subarray，先估复制与分配成本；给出计算快而端到端收益小的反例。

SIMDRAM: An End-to-End Framework for Bit-Serial SIMD Computing in DRAM

2021 · ASPLOS 2021 · 原文链接

问题 少量DRAM内逻辑原语如何系统化支持更复杂的运算。

机制 逻辑综合、计算行分配与命令执行，配合bit-serial数据布局。

取舍 位宽和复杂操作带来串行步骤；转置、复制、编程与系统集成都要计账。

证据边界 主要PIM方案用gem5/电路等建模，真实CPU/GPU基线不等于PIM硅已实现。

面试复述 讲清可表达性与效率区别；用转置成本的摊薄阈值解释什么负载适合。

Hardware Architecture and Software Stack for PIM Based on Commercial DRAM Technology: Industrial Product

2021 · ISCA 2021 Industrial Product · 原文链接

问题 存储厂商如何让bank附近计算接入已有主机体系。

机制 官方实现展示寄存器、bank数据及内存命令驱动的数字运算流程。

取舍 布局、广播、模式切换、同步和缓存可见性不能因接口复用而忽略。

证据边界 论文书目/摘要核验，全文访问受限；具体机制依据Samsung官方公告及PIMSimulator，未复现实片。

面试复述 比较按矩阵行分片与列分片：分别需要输入广播还是partial-sum归约。

Benchmarking a New Paradigm: An Experimental Analysis of a Real Processing-in-Memory Architecture

2021 · arXiv:2105.03814；UPMEM真实系统研究 · 原文链接

问题 真实通用PIM系统能跑哪些应用，瓶颈在哪里。

机制 用DPU微基准和应用测量拆解本地计算、内存及跨DPU通信。

取舍 有限指令与算力、数据分发、host中转通信；结果依配置和负载。

证据边界 真实UPMEM平台实验；应区分kernel时间与包含传输的端到端时间。

面试复述 构造过滤选择率改变及全局shuffle反例；不要从更多DPU推导线性加速。

LLM in a flash: Efficient Large Language Model Inference with Limited Memory

2024 · ACL 2024 Long Papers · 原文链接

问题 普通设备DRAM容量放不下模型，按需加载的Flash读成本过高。

机制 近期神经元复用与连续权重布局，降低新增搬运并改善读粒度。

取舍 模型稀疏性、预测成本/精度、DRAM缓存预算及布局重排限制可迁移性。

证据边界 ACL 2024真实平台实验；是普通Flash加载研究，不是HBF实测。

面试复述 为什么多读一点连续数据可能更快？先比较数据量/有效带宽，再加入缓存容量。

FLINT: Efficiently Leveraging High Bandwidth Flash for Capacity-Scalable LLM Inference Acceleration

2026 · arXiv:2608.25062 v1，2026-08-25预印本 · 原文链接

问题 HBF权重容量层需要隐藏读延迟，且维护和通用FTL会带来额外成本。

机制 动态burst合并、复用局部buffer、维护plane和只读映射特化。

取舍 依赖权重读密集假设；映射、维护资源与运行时请求分布仍有限制。

证据边界 2026-08-25 arXiv v1；trace-driven系统模拟，非HBF器件实测。

面试复述 用消融分开容量、合并、重叠和维护收益；说明什么时候只读特化不再适用。

A Full-Stack Characterization of High-Bandwidth Flash for KV-Centric LLM Serving

2026 · arXiv:2608.11668 v1，2026-08-12预印本 · 原文链接

问题 直接把SSD式KV层换成更快Flash是否能自动提高服务质量。

机制 完整服务trace与分层模型，检查近端资源损失、远端复用与写成本。

取舍 比较包含HBM/GDDR资源变化；不能将完整架构负结果归因于单一介质。

证据边界 2026-08-12预印本，扩展TokenSim模拟；作者明确尚无商业HBF硅验证。

面试复述 整体prefix命中率高，远端层为什么仍写多读少？分层条件命中率是关键。

Characterizing High Bandwidth Flash for LLM Serving

2026 · arXiv:2609.39131 v1，2026-09-30预印本 · 原文链接

问题 大容量HBF如何通过放置和调度转化为高吞吐服务收益。

机制 HBM–HBF–host分层及预留KV增长空间的调度；比较权重/专家放置。

取舍 吞吐与TPOT不等价；寿命依P/E、WAF及均匀磨损假设，模型不覆盖所有控制器和功耗。

证据边界 2026-09-30预印本；GPU kernel校准+HBF系统建模，主要面向高吞吐场景。

面试复述 解释总完成时间改善但TPOT变差的情况，并以SLO goodput选择方案。

A 5-nm 254-TOPS/W 221-TOPS/mm² Fully-Digital Computing-in-Memory Macro Supporting Wide-Range DVFS and Simultaneous MAC and Write Operations

2022 · ISSCC 2022, paper 11.6 · 原文链接

问题 数字 CIM 怎样在保留确定性计算的同时，减少权重搬运，并缓解写入与 MAC 竞争？读题时先固定 4 位输入/4 位权重/14 位输出这一官方口径。

机制 作者摘要给出 5 nm 全数字宏、专门 12T 位单元和同时 MAC/写入能力，并强调动态电压频率配置。可以据此理解端口、电路拓扑与流水线的协同，不能凭摘要补出未公开核读的每条信号路径。

取舍 增加并发能力通常会影响位单元面积、布线和控制。宏级峰值能效还需对应电压、精度、活动率和统计边界，不能直接与高精度 GPU 系统数字比较。

证据边界 本卡是原始摘要及 ISSCC 官方 press kit 的定向阅读，不是全文复现。官方简介提供 4b/4b/14b，论文标题给出的 254 TOPS/W 应始终带上宏与精度条件。

面试复述 按“并发冲突是什么→专用单元怎样改变约束→付出哪些成本→怎样定义验证条件”复述。追问若无法回答电压/活动率细节，应承认尚未读到完整曲线。可做位串行 MAC 的软件功能模型，但不能把它称为晶体管级复现。

CAP-RAM: A Charge-Domain In-Memory Computing 6T-SRAM for Accurate and Precision-Programmable CNN Inference

2021 · IEEE JSSC / author manuscript · 原文链接

问题 怎样既保留较高 SRAM 存储密度，又利用电荷域并行计算，并在真实工艺失配下维持可用精度？它直接纠正“SRAM 必然数字计算”的误解。

机制 标准 6T SRAM 保存位，由共享电荷计算电路形成模拟乘加，后接转换与数字处理。阅读重点是位存储与计算域分离，以及校准对整体传递曲线的作用。

取舍 电容、共享结构和 ADC 带来面积、并行度、精度与存储利用率取舍。可配置位宽和模型压缩有助于映射，但意味着工作负载条件必须交代。

证据边界 已核读公开全文和测量章节，论文有 65 nm 原型。它既测量不同切片/芯片的误差，也进行网络验证；部分网络测试使用 500 张图像，不能将这些结果描述为任意任务上的完整标准评测。

面试复述 先画“SRAM 位→开关/电容→电荷共享→ADC→校准”，再解释偏置、增益、非线性为什么要分开测。可提出复现实验：分别关闭增益校准、偏置校准和量化感知训练，观察 MVM 误差与任务准确率；没有实际运行就标为实验计划。

A compute-in-memory chip based on resistive random-access memory

2022 · Nature 608, 504–512 · 原文链接

问题 RRAM 模拟计算怎样从孤立点积扩展到可重配置、多层模型，并使芯片非理想进入算法设计？

机制 NeuRRAM 采用可重配置阵列与电压模式神经元读出，配合硬件特征驱动的校准及训练。重点看数据流和误差共同决定可映射任务，而不只是器件能够做乘法。

取舍 提高数值位宽不保证提高有效精度，输入幅度、噪声、并行度和读出方式都可能改变误差；算法适配也增加部署流程。

证据边界 已核读 Nature 2022 正文、Methods 与扩展数据。输入位宽与 RMSE 的例子来自期刊版本，避免同早期预印本混用数字。任务精度接近某量化基线不表示每个点积逐位精确。

面试复述 围绕“位宽增加为何可能不降误差”回答：格式分辨率只是一个环节，物理信噪比与转换链同样约束结果。可设计输入幅度×位宽×并行核心数扫描，并同时观察误差和模型质量；不得用独立高斯噪声自动替代全部硬件行为。

A 64-core mixed-signal in-memory compute chip based on phase-change memory for deep neural network inference

2023 · Nature Electronics 6, 680–693; public preprint 2022 · 原文链接

问题 怎样把多个 PCM 模拟计算核、数字后处理及通信组合起来，同时量化时间相关误差，并展示网络推理？

机制 多核 PCM 提供模拟矩阵操作，数字单元完成若干激活、缩放和归并；研究区分编程误差、剩余电路误差与漂移。阅读时把数据路径画到 FPGA/主机边界。

取舍 改善器件编程误差可能改变读出工作区与其他误差；数字功能扩展系统能力，但通信与不支持算子仍影响完整应用。

证据边界 核读的是 2022 作者全文，期刊版发表于 2023。预印本演示的层间激活经 FPGA 传输，max-pooling/embedding 有未支持部分；因此此处不把宏峰值当独立系统实测。

面试复述 用“接口位宽≠有效精度”和“片上算子多≠全部端到端自包含”两条原则分析。若要产品化，提出补齐层间缓冲/互连与算子覆盖，再测真实任务能量；不声称已经复现这颗芯片。

A mixed-precision memristor and SRAM compute-in-memory AI processor

2025 · Nature 639, 617–623 · 原文链接

问题 统一低精度或统一高精度为什么都可能浪费？怎样同时依据模型敏感性选择数值格式和 memristor/SRAM 计算路径？

机制 公开扩展数据展示 memristor 局部模拟计算、5 位 ADC、数字合并以及 SRAM 数字路径，并描述精度配置与浮点对齐。把它理解为多层次协同选择，比把标题简化为固定高低位拆分更准确。

取舍 异构获得精度与能量的配置空间，却增加映射、转换、配置和调度复杂度；位宽、工作负载与统计范围必须同时报告。

证据边界 主文订阅受限，已核读作者摘要与公开 Extended Data 图注，不标为完整全文深读或可公开代码复现。浮点对齐相关能量比较属于模拟，任务报告与组件模拟分开处理。

面试复述 设计一张“层→数值格式→执行引擎→误差→通信量”的表，说明如何选择配置。先以同质量约束比较，再讨论能效；若面试官问详细电路，明确自己目前可验证的公开证据范围。

Mixed-precision deep learning based on computational memory

2020 · Frontiers in Neuroscience 14, 406; author manuscript · 原文链接

问题 模拟器件更新粗糙且有随机性时，怎样保留小梯度的信息，避免每一步都因直接舍入或不准确脉冲而丢失？

机制 模拟存储参与主要加权和，数字端维护高精度更新残差；累积到可写粒度才发出更新脉冲，再保留未实现部分。它把精度放在学习状态而非要求所有模拟操作都精确。

取舍 数字状态需要容量、计算和通信；器件脉冲仍不理想，方案是否有效需用训练质量验证。不能把降低更新频率当作耐久与收敛问题全部消失。

证据边界 已读作者全文。MNIST 是硬件/软件结合训练实验，其他网络评估包含行为仿真，系统能效提升为模型研究，不能写成一颗完整训练芯片的实测功耗。

面试复述 手推步长 0.01、每步 0.003 的残差例子，然后说明随机更新与状态依赖为什么还需验证。可复现软件残差算法并扫描非线性/不对称；没有 PCM 硬件就把结论限定为行为模型。

A crossbar array of magnetoresistive memory devices for in-memory computing

2022 · Nature 601, 211–216; DOI 10.1038/s41586-021-04196-6 · 原文链接

问题 MRAM 具有实际存储优势，但其低电阻会让传统电流求和 crossbar 产生较大电流；是否只能等更高阻材料？

机制 作者改变求和方式，以电阻求和架构实现 MRAM 阵列计算，并集成读出电路。这里的关键是让架构适应器件工作区，而非机械套用一种 crossbar 模型。

取舍 改变求和与读出方式会引入自己的映射、动态范围和外围限制，不能据此宣称 MRAM 普遍优于 RRAM/PCM；应用精度与规模仍需验证。

证据边界 已读作者机构公开全文，包含 64×64 阵列。小网络实测与利用实测误差进行更深网络仿真分别报告；仿真准确率不当作完整深网络的硬件实测。

面试复述 回答“都在等材料吗”时给出这一反例：材料约束也可以驱动电路创新。随后追问扩展到大阵列的线阻、面积、读出和网络映射，避免从单阵列演示直接跳到通用大模型产品。

完整来源索引

1. **Micron TN-40-40: DDR4 Point-to-Point Design Guide, Rev. H** · 2020 · Micron Technical Note（授权分销商镜像）
   阅读第2页8n prefetch、第6–8页Bank Group与时序、第9页PPR。为Micron原始技术文档；具体器件仍须核对datasheet。
   https://www.mouser.com/pdfdocs/micron_ddr4_design_guide.pdf
2. **A Case for Exploiting Subarray-Level Parallelism (SALP) in DRAM** · 2012 · ISCA
   阅读§2逻辑/物理组织、§4 SALP-1/SALP-2/MASA。电路组织分析与系统仿真；非量产收益。
   https://people.inf.ethz.ch/omutlu/pub/salp-dram_isca12.pdf
3. **Tiered-Latency DRAM: A Low Latency and Low Cost DRAM Architecture** · 2013 · HPCA
   阅读§2电荷过程、§3位线长度权衡、§4近远段。电路模型与系统模拟；不是商用器件测试。
   https://people.inf.ethz.ch/omutlu/pub/tldram_hpca13.pdf
4. **Adaptive-Latency DRAM: Optimizing DRAM Timing for the Common-Case** · 2015 · HPCA
   阅读§2.2激活/恢复/预充电、FPGA表征与真实系统评估。实测支持特定DDR3样品与条件，不构成所有DRAM降时序的保证。
   https://users.ece.cmu.edu/~omutlu/pub/adaptive-latency-dram_hpca15.pdf
5. **RAIDR: Retention-Aware Intelligent DRAM Refresh** · 2012 · ISCA
   阅读§3分箱、Bloom filter及刷新实现。评估为模型/仿真，对保持时间准确获知有前提。
   https://research.ece.cmu.edu/safari/pubs/2012_raidr.pdf
6. **An Experimental Study of Data Retention Behavior in Modern DRAM Devices** · 2013 · ISCA
   阅读测试平台、数据模式与VRT章节。248颗DDR3实测；揭示DPD与VRT，不应外推固定比例到新器件。
   https://people.inf.ethz.ch/omutlu/pub/dram-retention-time-characterization_isca13.pdf
7. **RowClone: Fast and Energy-Efficient In-DRAM Bulk Data Copy and Initialization** · 2013 · MICRO
   阅读§2内部数据路径、§3 FPM/PSM与系统支持。架构提案/模型和系统模拟，不代表普通DDR可直接执行非法命令。
   https://people.inf.ethz.ch/omutlu/pub/rowclone_micro13.pdf
8. **Flipping Bits in Memory Without Accessing Them: An Experimental Study of DRAM Disturbance Errors** · 2014 · ISCA
   真实系统演示与FPGA器件表征；重复激活导致附近行扰动，防护讨论包含概率邻行刷新。历史样本阈值不是现代器件安全阈值。
   https://people.inf.ethz.ch/omutlu/pub/dram-row-hammer_isca14.pdf
9. **RowPress: Amplifying Read Disturbance in Modern DRAM Chips** · 2023 · ISCA
   阅读§4.1实验方法：扫行打开时长及最少扰动激活数。真实DDR4表征，强调激活次数之外的驻留时间。
   https://people.inf.ethz.ch/omutlu/pub/RowPress_isca23.pdf
10. **Understanding and Modeling On-Die Error Correction in Modern DRAM: An Experimental Study Using Real Devices** · 2019 · DSN
    阅读§1、§4及真实LPDDR4研究。外部可见错误已被on-die ECC变换，不能当作原始bitcell错误率。
    https://people.inf.ethz.ch/omutlu/pub/understanding-and-modeling-in-DRAM-ECC_dsn19.pdf
11. **Design-Induced Latency Variation in Modern DRAM Chips** · 2017 · SIGMETRICS / POMACS
    阅读阵列/Mat分层、与外围电路距离造成的时延差异。96条DIMM表征；不是简单按逻辑地址线性推断快慢。
    https://people.inf.ethz.ch/omutlu/pub/DIVA-low-latency-DRAM_sigmetrics17-paper.pdf
12. **Ramulator 2.0: A Modern, Modular, and Extensible DRAM Simulator** · 2024 · IEEE Computer Architecture Letters（2023预印本）
    阅读§II层级状态机和命令约束、§III用Micron DDR4 Verilog模型检查命令轨迹。协议通过不等于模拟了硅片所有模拟行为。
    https://arxiv.org/html/2308.11030v2
13. **Ramulator 2.1: A Composable Memory System Simulator for Modern DRAM Systems** · 2026 · arXiv预印本
    阅读§2现代协议特征、§4验证。说明截至2026-10-30已有2.1；这里只建议固定版本实验，不宣称通过了本项目未执行的仿真。
    https://arxiv.org/html/2606.13844v1

14. **Micron DRAM Power Calculators** · 2026 · Micron官方设计工具（访问于2026-10-30）
官方页面说明按实际系统访问行为估算功耗，并列DDR4等计算器及技术说明；估算不能代替测量。
https://www.micron.com/sales-support/design-tools/dram-power-calculator

15. **Micron DDR5 SDRAM Product and Feature Overview** · 2026 · Micron官方产品页（访问于2026-10-30）
官方DDR4/DDR5对照包含Bank、burst及on-die ECC；器件配置随位宽/密度变化，不把一个表格推广到全部产品。
https://www.micron.com/products/memory/dram-components/ddr5-sdram

16. **SoftMC: A Flexible and Practical Open-Source Infrastructure for Enabling Experimental DRAM Studies** · 2017 · HPCA
阅读FPGA可控命令平台及验证案例。实验平台支持物理表征，原论文还报告某些预期时延收益在现有芯片不可观察，提醒区分模型和实测。
https://people.inf.ethz.ch/omutlu/pub/softMC_hpca17.pdf

17. **Memory Access Scheduling** · 2000 · ISCA 2000
已核读第 3 节设计空间与第 5 节 first-ready/aggressive reordering。支持按 DRAM bank/row/column 状态重排请求；历史评测数字不能直接外推到 DDR5/HBM。
https://groups.csail.mit.edu/cag/PSRG/papers/rixner_isca27.pdf

18. **DFI：控制器与 PHY 接口官方说明** · 2026 · DFI Group；2026-10-30 访问
官方明确 DFI 的接口边界；页面记录 DFI 6.0 于 2026-05 发布并首次正式加入 HBM 支持。训练分工仍应查所用版本及 IP。
https://ddr-phy.org/

19. **The Blacklisting Memory Scheduler: Achieving High Performance and Fairness at Low Cost** · 2014 · ICCD 2014
已核读第 III–V 节：连续服务计数、黑名单分类、周期清除，以及非黑名单/行命中/年龄优先级。用于学习公平性与硬件成本，不引用其阈值为通用最佳值。
https://people.inf.ethz.ch/omutlu/pub/bliss-memory-scheduler_iccd14.pdf

20. **Improving DRAM Performance by Parallelizing Refreshes with Accesses** · 2014 · HPCA 2014
已核读机制：DARP 调整 per-bank 刷新次序并与写回批次重叠；SARP 需 DRAM 内部支持子阵列访问与刷新并行。论文中的当代标准背景是 2014 年。
https://people.inf.ethz.ch/omutlu/pub/dram-access-refresh-parallelization_hpca14.pdf

21. **Ramulator 2.0: A Modern, Modular, and Extensible DRAM Simulator** · 2023 · 2023 arXiv 版本；IEEE Computer Architecture Letters
已核读第 2 节：将控制器与 DRAM 标准解耦、接口/实现模块化、命令与时序约束。2023 是预印本年份；当前仓库引用条目列 CAL 2024。
https://arxiv.org/pdf/2308.11030

22. **CMU-SAFARI/ramulator2 官方仓库** · 2026 · CMU SAFARI；2026-10-30 访问
已核读 README 的配置、统计及测试章节。当前默认文档为 Ramulator 2.1，不能把它的 Python 配置命令照搬到 2.0；实验须固定提交。 本轮 git ls-remote 与固定 README 核验提交为 c6c7a5de0cef35935d6fad7369e6271406798865。
https://github.com/CMU-SAFARI/ramulator2

23. **CUDA Programming Guide：Writing SIMT Kernels** · 2026 · NVIDIA 持续更新官方文档；2026-10-30 访问
已核读 Coalesced Global Memory Access：warp 请求合并为 32-byte 事务，强调使用字节/传输字节比例。事务口径不能直接当作 DRAM burst。
https://docs.nvidia.com/cuda/cuda-programming-guide/02-basics/writing-cuda-kernels.html

24. **NVIDIA Hopper Architecture In-Depth** · 2022 · NVIDIA 官方架构说明
已核读 GH100/H100 配置与架构图、L2/HBM 控制器、TMA 说明。区分完整 die 与 SKU，不将其中早期性能规格当最新产品承诺。
https://developer.nvidia.com/blog/nvidia-hopper-architecture-in-depth/

25. **Roofline: An Insightful Visual Performance Model for Floating-Point Programs and Multicore Architectures** · 2008 · UC Berkeley 技术报告 UCB/EECS-2008-134；2009 CACM 工作的原始报告
已核读第 2 节和讨论：以选定层级实际流量定义 operational intensity，性能上界为计算峰值和带宽乘强度的较小者；可为 L2 等层级建立 roofline。
https://www2.eecs.berkeley.edu/Pubs/TechRpts/2008/EECS-2008-134.pdf

26. **FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness** · 2022 · NeurIPS 2022
已核读第 3 节：分块、在线 softmax 统计、反向重算、融合减少 HBM 中间流量。exact 表示标准注意力计算，不意味着所有浮点执行逐位一致。
https://arxiv.org/pdf/2205.14135

27. **Efficient Memory Management for Large Language Model Serving with PagedAttention** · 2023 · SOSP 2023
已核读第 3、4、7 节：KV 分块映射、动态分配、共享；块尺寸在内存碎片和执行效率间取舍。其软件 block 不是 DRAM row。
https://arxiv.org/html/2309.06180v1

28. **Choosing the Right DDR SDRAM Memory for Your Application** · 2019 · Synopsys 官方白皮书（历史分类资料）
已核读 DDR/LPDDR/GDDR/HBM 分类和应用取舍。只用于稳定分类与 PPA 思路，文中 DDR5 仍属未来的表述已过时，不沿用其路线图。
https://www.synopsys.com/dw/doc.php/wp/which_DDR_SDRAM_memory_to_use_and_when_LI.pdf

29. **Micron LPDDR5X 官方产品与应用说明** · 2026 · Micron 持续更新页面；2026-10-30 访问
已核读 mobile、data center 应用及标准说明。证明 LPDDR 应用不限手机；厂商功耗优势必须限定测试口径。
https://www.micron.com/products/memory/lpddr-components/lpddr5x

30. **NAND Flash Organization** · 2024 · Microchip 官方在线技术文档
已核读 NAND block/page/spare 区、坏块管理与有限寿命说明。未套用该页旧器件寿命数值到 HBF 或现代 TLC/QLC。
https://onlinedocs.microchip.com/oxy/GUID-87101AE8-6003-4D1F-B624-895C158CCD3E-en-US-1/GUID-F3BE160C-4AEC-4F0A-92A7-39025FDDEE0B.html

31. **GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints** · 2023 · EMNLP 2023
已核读第 2 节：多个 query head 分组共享 K/V head。KV 容量公式应使用 KV head 数，不能不加区分地代入 query head 数。
https://aclanthology.org/2023.emnlp-main.298.pdf

32. **FlexGen: High-Throughput Generative Inference of Large Language Models with a Single GPU** · 2023 · ICML 2023
已核读第 4 节：分层放置、批次/层遍历、传输与计算重叠。目标是延迟不敏感的吞吐量，不能将离线收益等同聊天 p99 延迟改善，也不是 HBF 实测。
https://arxiv.org/pdf/2303.06865

33. **Nsight Compute Profiling Guide** · 2026 · NVIDIA 持续更新官方文档；2026-10-30 访问
已核读采集模式、Cache Control、指标说明。默认 replay 之间清缓存可改变实验语义；应记录 replay/cache-control 配置。
https://docs.nvidia.com/nsight-compute/ProfilingGuide/index.html

34. **Sandisk HBF Fact Sheet** · 2025 · Sandisk 2025-07 技术简介
已核读三页简介：HBF 基于 NAND、面向 AI 推理。容量/带宽/性能表述属于厂商设计说明，不作为本学习项目实测；最新进展见独立 HBF 模块。
https://documents.sandisk.com/content/dam/asset-library/en_us/assets/public/sandisk/collateral/company/Sandisk-HBF-Fact-Sheet.pdf

35. **What Designers Need to Know About HBM3** · 2022 · Synopsys technical bulletin
已读Channel Architecture及RAS：HBM3 16×64-bit channel、32×32-bit pseudo-channel、BL8；规格解释，不等于所有GPU实际效率。
https://www.synopsys.com/articles/hbm3-ip-dwtb.html

36. **Micron HBM3E product information** · 2026 · Micron（网页检索于2026-10-30）
官方产品口径：24GB 8-high/36GB 12-high、1024 I/O、16 channel/32 pseudo-channel、超过1.2TB/s；厂商产品指标。
https://www.micron.com/products/memory/hbm/hbm3e

37. **Micron HBM4 product information** · 2026 · Micron（网页检索于2026-10-30）
2048 I/O、12-high 36GB、16-high 48GB samples等产品状态；用于区分接口宽度、容量与堆叠高度。
https://www.micron.com/products/memory/hbm/hbm4

38. **CoWoS advanced packaging** · 2026 · TSMC 3DFabric（网页检索于2026-10-30）
已读CoWoS-S/R/L结构：SoC与HBM通过硅/RDL/局部硅互连集成；2.5D封装不等于GPU上垂直叠HBM。
https://3dfabric.tsmc.com/english/dedicatedFoundry/technology/cowos.htm

39. **What is NAND Flash Memory?** · 2026 · KIOXIA技术教育（检索于2026-10-30）
已读储电薄膜、阈值电压读出、串联cell及掉电保持机制；非易失不等于无限保持。
https://www.kioxia.com/en-jp/rd/technology/nand-flash.html

40. **Managing Garbage Collection Operations Efficiently to Improve e-MMC and UFS Device Performance** · 2024 · KIOXIA technical brief
已读6页技术说明：L2P、异地更新、有效页迁移、GC与WAF；文档例子不是所有设备的固定WAF。
https://americas.kioxia.com/content/dam/kioxia/en-us/business/memory/mlc-nand/asset/KIOXIA_Efficient_Garbage_Collection_Operations_Tech_Brief.pdf

41. **Micron FAQs — NAND ECC and read disturb** · 2026 · Micron technical support（检索于2026-10-30）
NAND问答证实ECC要求依器件而异、重复读引起read disturb、需刷新/迁移；不采旧产品速度作2026通用值。
https://www.micron.com/sales-support/sales/faqs

42. **Sandisk and SK hynix Advance Global Standardization of HBF with First OCP Technical Specification** · 2026 · Sandisk 2026-08-03官方公告
首版技术规范发布、标准化工作组与参与者；发布标准不证明产品量产或独立性能实测。
https://www.sandisk.com/company/newsroom/press-releases/2026/2026-08-03-Sandisk-and-sk-hynix-advance-global-standardization-of-hbf

43. **SK hynix Unveils First HBF Standard Specifications at FMS 2026** · 2026 · SK hynix 2026-08-04官方公告
8/16-high NAND dies、最高512GB、约0.4–3.0TB/s三档、UCIe接口。数值属公布的规格范围，不拼成某现售单型号。
https://news.skhynix.com/en/hbf-at-fms-2026/

44. **Memory-Centric AI: Sandisk’s High Bandwidth Flash Will Redefine AI Infrastructure** · 2025 · Sandisk roadmap blog
2025公告提出HBF memory于2026下半年送样、推理设备2027年初送样的目标；截至检索不能据此断言目标已完成。
https://www.sandisk.com/company/newsroom/blogs/2025/memory-centric-ai

45. **FLINT: Efficiently Leveraging High Bandwidth Flash for Capacity-Scalable LLM Inference Acceleration** · 2026 · arXiv:2608.25062 v1，2026-08-25预印本
已读机制4.1–4.4、方法5、评估6：trace-driven模拟、burst-buffer/phantom-plane/read-only FTL；不可当HBF硅实测。
https://arxiv.org/html/2608.25062v1

46. **LLM in a flash: Efficient Large Language Model Inference with Limited Memory** · 2024 · ACL 2024 Long Papers
已读2–4节及实验/附录：普通flash到DRAM的稀疏权重加载、windowing和row-column bundling；真实平台实验，非HBF论文。
https://aclanthology.org/2024.acl-long.678.pdf

47. **Ambit: In-Memory Accelerator for Bulk Bitwise Operations Using Commodity DRAM Technology** · 2017 · MICRO 2017
已读TRA、NOT、临时行与命令序列、系统集成和评估：电路/架构仿真方案；不是通用DRAM现成指令。
https://www.microsoft.com/en-us/research/wp-content/uploads/2017/09/MICRO-50_347.pdf

48. **SIMDRAM: An End-to-End Framework for Bit-Serial SIMD Computing in DRAM** · 2021 · ASPLOS 2021
已读bit-serial映射、控制单元、系统集成、gem5方法和评估；SIMDRAM是仿真，CPU/GPU基线部分为真实测量。
https://people.inf.ethz.ch/omutlu/pub/SIMDRAM_asplos2021.pdf

49. **Benchmarking a New Paradigm: An Experimental Analysis of a Real Processing-in-Memory Architecture** · 2021 · arXiv:2105.03814；UPMEM真实系统研究
已读DPU/WRAM/MRAM结构、微基准和第6节：真实UPMEM系统，适配简单运算且少跨DPU通信的负载；结果绑定当代配置。
https://arxiv.org/pdf/2105.03814

50. **Samsung Develops Industry’s First High Bandwidth Memory with AI Processing Power** · 2021 · Samsung 2021-02-17 官方公告
官方披露HBM-PIM在memory bank附近部署可编程计算能力；公布效果需绑定演示负载。
https://news.samsungsemiconductor.com/global/samsung-develops-industrys-first-high-bandwidth-memory-with-ai-processing-power/

51. **Samsung SAIT PIMSimulator** · 2021 · Samsung Advanced Institute of Technology官方开源实现
已读架构、指令/模式及GEMV使用说明：CRF、GRF、bank row buffer、ADD/MUL/MAC/MAD；模拟器本身不是实片测量。
https://github.com/SAITPublic/PIMSimulator

52. **Hardware Architecture and Software Stack for PIM Based on Commercial DRAM Technology: Industrial Product** · 2021 · ISCA 2021 Industrial Product
已核实会议官网及IEEE摘要书目；论文全文访问受限。本学习包细节依据Samsung公开实现和官方公告，不声称复现该论文全部实片结果。
https://ieeexplore.ieee.org/document/9499894/

53. **A Full-Stack Characterization of High-Bandwidth Flash for KV-Centric LLM Serving** · 2026 · arXiv:2608.11668 v1，2026-08-12预印本
已读固定v1的4方法、7写流量、9.3局限：TokenSim及生产trace衍生模拟，比较完整架构同时改变近端容量/带宽；不能归纳成HBF介质必输SSD。2026-10-30复核arXiv已有v4（2026-09-14），标题增加HBF Sucks?；本教程的章节定位和分析仍对应所链v1，不混用新版数值。
https://arxiv.org/html/2608.11668v1

54. **Characterizing High Bandwidth Flash for LLM Serving** · 2026 · arXiv:2609.39131 v1，2026-09-30预印本
已读固定v1的3方法、4分层与调度、5结果局限：GPU kernel校准+HBF模拟；着重高吞吐；寿命依均匀磨损/WAF=1/100k P/E假设，非保证。2026-10-30复核arXiv已有v2（2026-10-05）；本教程分析对应所链v1，避免把新版摘要百分比与v1配置混合。
https://arxiv.org/html/2609.39131v1

55. **HBFSim: Fast and Faithful Simulation of High-Bandwidth Flash Under Real GPU Execution** · 2026 · arXiv:2609.09800 v1，2026-09-09预印本
已读固定v1的4机制及6.1–6.2：真实GPU执行但HBF为注入模型；校准flash/thermal设备为普通NVMe SSD；注册访问覆盖范围有限。2026-10-30复核arXiv已有v3（2026-09-30），论文与SlugLab/HBFSim官方代码仓库互链且题名一致；本教程中的具体校准/覆盖批评仅针对所链v1，不代替对新版代码的审计。
https://arxiv.org/html/2609.09800v1

56. **A Review of SRAM-based Compute-in-Memory Circuits** · 2024 · arXiv review
已读作者公开全文，用于计算域、SRAM 电路路线与术语地图；具体芯片结果另引原始论文。
https://arxiv.org/html/2411.06079v1

57. **A 5-nm 254-TOPS/W 221-TOPS/mm² Fully-Digital Computing-in-Memory Macro Supporting Wide-Range DVFS and Simultaneous MAC and Write Operations** · 2022 · ISSCC 2022, paper 11.6
核对出版方摘要及 ISSCC 官方 press kit；IEEE 正文受访问限制，未声称阅读全文。摘要支持 12T、64 kb、同时 MAC/写入；精度口径由官方会议信息交叉确认。
https://ieeexplore.ieee.org/document/9731754/

58. **ISSCC 2022 Press Kit — Session 11, paper 11.6** · 2022 · IEEE ISSCC
已打开官方 PDF 并读 paper 11.6 条目：5 nm 数字 CIM、4 位输入/4 位权重/14 位输出；会议预览信息不替代完整论文实现细节。
https://www.isscc.org/s/ISSCC2022PressKit.pdf

59. **Colonnade: A Reconfigurable SRAM-Based Digital Bit-Serial Compute-In-Memory Macro for Processing Neural Networks** · 2021 · IEEE JSSC 56(7), 2221–2233
已读作者机构页面所载完整摘要与书目信息。支持 1–16 位可配置、XNOR/全加器、位串行面积与时延取舍；未获得全文，不据此推断每条数据通路。
https://pure.kaist.ac.kr/en/publications/colonnade-a-reconfigurable-sram-based-digital-bit-serial-compute-/

60. **CAP-RAM: A Charge-Domain In-Memory Computing 6T-SRAM for Accurate and Precision-Programmable CNN Inference** · 2021 · IEEE JSSC / author manuscript
已读公开全文，包括电荷域结构、ADC、校准、芯片测量与网络映射。区分真实宏测试、网络测试规模及模拟对照。
https://arxiv.org/pdf/2107.02388

61. **A compute-in-memory chip based on resistive random-access memory** · 2022 · Nature 608, 504–512
已读期刊全文、Methods 和 Extended Data。采用 2022 期刊版本；未混抄 2021 预印本数字。重点核读电压模式、噪声感知训练和输入位宽与 RMSE。
https://www.nature.com/articles/s41586-022-04992-8

62. **A 64-core mixed-signal in-memory compute chip based on phase-change memory for deep neural network inference** · 2023 · Nature Electronics 6, 680–693; public preprint 2022
已读 2022 作者全文的架构、误差、推理与实验方法；最终期刊 DOI 为 10.1038/s41928-023-01010-1。本文具体实验边界依公开预印本描述，明确 FPGA 层间通信及不支持的算子。
https://arxiv.org/pdf/2212.02872

63. **A mixed-precision memristor and SRAM compute-in-memory AI processor** · 2025 · Nature 639, 617–623
已读出版方摘要及公开 Extended Data 图注 1–9；主文需订阅，未宣称全篇可复现。图注支持 memristor/SRAM 异构、5 位 ADC 加数字累加、量化配置；浮点对齐能量图为模拟。
https://www.nature.com/articles/s41586-025-08639-2

64. **Fusion of memristor and digital compute-in-memory processing for energy-efficient edge computing** · 2024 · Science 384, 325–332; DOI 10.1126/science.adf5538
已读作者摘要及出版信息，出版社正文访问受限；只用于验证 memristor 与数字 CIM 融合的研究方向，不引用未核读的电路细节。
https://pubmed.ncbi.nlm.nih.gov/38669568/

65. **Compute in-Memory with Non-Volatile Elements for Neural Networks: A Review from a Co-Design Perspective** · 2023 · Advanced Materials 35, 2204944
已读开放全文器件、外围和系统协同章节。综述用于范围梳理；表格典型值不当作材料普适上限，关键案例另引原始研究。
https://advanced.onlinelibrary.wiley.com/doi/full/10.1002/adma.202204944

66. **A crossbar array of magnetoresistive memory devices for in-memory computing** · 2022 · Nature 601, 211–216; DOI 10.1038/s41586-021-04196-6
已读作者机构公开全文：64×64 MRAM 阵列以电阻求和应对低阻大电流；区分实测小网络与使用实测误差的更深网络仿真。
https://ciqm.harvard.edu/uploads/2/3/3/4/23349210/jung2022.pdf

67. **Long-term reliability of naturally aged hafnium oxide ferroelectric transistors for energy-efficient embedded memory** · 2026 · Cell Reports Physical Science 7, 103057; DOI 10.1016/j.xcrp.2025.103057
已读开放全文；自然老化超过五年与编程态保持测试是不同概念，长期保持涉及外推。单批器件的耐久、温度与脉冲条件不推广为全部 FeFET 极限。
https://publica-rest.fraunhofer.de/server/api/core/bitstreams/3a789816-dabd-4f94-9ae5-6d4e69da0d49/content

68. **Mixed-precision deep learning based on computational memory** · 2020 · Frontiers in Neuroscience 14, 406; author manuscript
已读架构、数字梯度残差与 PCM 更新方法。MNIST 为硬件/软件结合实验，其余网络含行为仿真，系统能效为建模结果。
https://arxiv.org/pdf/2001.11773

69. **Ambit: In-Memory Accelerator for Bulk Bitwise Operations Using Commodity DRAM Technology** · 2017 · MICRO 2017
已读作者官方摘要并核对原论文入口。三行激活与感放反相为提案机制，证据含电路模拟及系统评估；不能说所有标准 DDR 支持这些操作。
https://www.microsoft.com/en-us/research/publication/ambit-memory-accelerator-bulk-bitwise-operations-using-commodity-dram-technology/